

Notas de Teoría de la Computación

Miguel Angel Norzagaray Cosío

25 de abril de 2023

Índice general

1. Lenguajes Regulares	7
1.1. Lenguajes	8
1.1.1. Alfabetos y cadenas	8
1.1.2. Lenguajes	13
1.1.3. Operaciones con lenguajes	14
1.2. Autómatas Finitos Determinísticos	15
1.2.1. Representación de AFD	16
1.2.2. Completitud de la función de transición	17
1.2.3. Autómata Procesando Cadenas	18
1.2.4. El lenguaje de un AFD	20
1.2.5. Descripción instantánea de un AFD	22
1.2.6. AFD mínimos	22
1.2.7. Análisis Léxico	22
1.2.8. Autómata finito no determinístico	25
1.2.9. Autómata finito con transiciones ϵ	30
1.3. Expresiones Regulares	36
1.3.1. Conversión de expresiones regulares a AFD	40
1.3.2. Conversión de AFD a expresiones regulares	42
1.3.3. Simplificación de expresiones regulares	49
1.4. Propiedades de los lenguajes regulares	51
1.4.1. ¿Cómo saber que un lenguaje es regular?	51
1.4.2. Propiedades de cerradura	52
1.4.3. Propiedades de decisión	55
1.5. Limitaciones de los lenguajes regulares	55
2. Lenguajes Libres de Contexto	57
2.1. Gramáticas Libres de Contexto	57
2.1.1. Derivaciones izquierdas y derivaciones derechas	59
2.1.2. Árboles de derivación	60
2.1.3. Formas normales	61
2.1.4. Simplificación de gramáticas	62
2.1.5. Ejemplos de aplicaciones	63
2.1.6. Gramáticas ambiguas	63
2.2. Autómatas de Pila	65

2.2.1.	Descripción instantánea de los AP	67
2.2.2.	Lenguaje de un Autómata de Pila	68
2.2.3.	Equivalencia entre GLL y AP	69
2.2.4.	Limitaciones de los AP	70
2.2.5.	Autómatas de Pila Determinísticos (APD)	71
2.3.	Lenguajes y compiladores	71
2.3.1.	Eliminación de recursividad izquierda	72
2.3.2.	Analizadores Recursivos Descendentes	73
2.3.3.	Analizadores LL(1)	73
2.4.	Propiedades de los LLC	74
2.4.1.	Propiedades de decisión	74
2.4.2.	Propiedades de cerradura	75
2.5.	Limitaciones de los lenguajes libres de contexto	75
3.	Lenguajes Recursivamente Enumerables	77
3.1.	Máquinas de Turing	77
3.1.1.	Descripción instantánea de una MT	80
3.1.2.	Lenguaje de una MT	81
3.1.3.	MT como calculadoras de funciones	82
3.1.4.	Máquinas de Turing no determinísticas	85
3.1.5.	Máquinas de Turing con múltiples cintas	86
3.1.6.	Lenguajes sensibles al contexto	88
3.2.	Problemas de decisión	89
3.2.1.	Máquinas de Turing Universales	90
3.2.2.	Máquina de Turing Universal restringida	91
3.2.3.	Lenguajes recursivos	94
3.2.4.	Lenguajes diagonal y universal	95
3.2.5.	El problema de la parada	97

Introducción

La Teoría de la Computación constituye los cimientos que sostienen con firmeza los conocimientos del área de la computación. Comprende dos grandes rubros, íntimamente ligados:

1. la teoría de los lenguajes y
2. la computabilidad y factibilidad.

Estas notas introductorias es material para usarse en el curso de Teoría de la Computación, en las carreras del Departamento Académico de Sistemas Computacionales de la UABCS. Abordan sólo la teoría de los lenguajes, aunque aportan los principios para el estudio de la computabilidad. Es de gran fortuna que muchos de los resultados importantes de la teoría de lenguajes que se utilizan en la práctica tiene demostraciones que son constructivas, lo que proporciona no sólo su veracidad, sino un algoritmo que permite realizar una tarea.

Cada vez que se define un concepto se *enfatiza* utilizando un tipo de letra que lo distinga. En algunos párrafos se ha colocado un símbolo $\star$ al margen, indicando que la idea es importante y que vale la pena su estudio cuidadoso.

Los ejercicios aparecen aparte en el Cuaderno de Ejercicios de Teoría de la Computación, indispensable para el buen aprovechamiento de esta asignatura.

Los conceptos y proposiciones sobre teoría de conjuntos e inducción matemática, pueden consultarse en el suplemento de Fundamentos Matemáticos. Dicho texto ofrecen el material necesario para estudiar lenguajes y autómatas. Es necesario para entender las proposiciones presentadas (y demostradas), que, en muchos casos, son medulares para el estudio de la computabilidad y el desarrollo del pensamiento abstracto del estudiante.

Estos materiales corresponden a un curso de Matemáticas Discretas y por tanto pueden emplearse ahí como material complementario. Como es de esperarse, muchos libros y notas sobre matemáticas discretas pueden ser de utilidad para este curso.

Material de utilidad fueron muchas referencias de todo tipo, desde libros clásicos, sus actualizaciones, así como notas de otros cursos. De manera particular, los Apuntes y Ejercicios de Gonzalo Navarro, de la Universidad de Chile, ofrecieron una perspectiva nueva para algunos temas.

Es ampliamente recomendable utilizar software como JFLAP (en este caso se utilizó la versión 7.0) para hacer ejercicios y experimentos.

Estas notas fueron elaboradas (y son mantenidas) utilizando $\text{\LaTeX}$, la distribución MikTeX 2.9, editando con TeXnicCenter 2.0 Beta y VauCanSon-G 0.4 para los diagramas de los autómatas.

El código fuente en $\text{\LaTeX}$ está disponible para otros docentes y estudiantes solicitándolo por correo electrónico a: `manc@uabcs.mx`.

Capítulo 1

Lenguajes Regulares

La teoría de lenguajes aporta al mundo de la computación no solo teoría elegante y formalismo indispensable para confiar en el trabajo del diseño de lenguajes y programación de computadoras, sino que además provee herramientas poderosas para el diseño mismo de lenguajes y su procesamiento para el trabajo de compilación.

Introducción

Desde hace mucho existen máquinas que hacen cálculos de algún tipo. El mecanismo de Antikitera, la Pascalina, el telar de Jackard (dirigido por tarjetas perforadas), las máquinas de Babagge y de Hollerit (electromecánica), hasta la época moderna, con la llegada de las ideas de Turing y el diseño y construcción de ENIAC, EDVAC y ACE (o su versión reducida la Pilot ACE).

Cuando se pensaba en el diseño de verdaderas computadoras (como la ACE, ya con programas residentes en memoria) se analizaba la manera óptima de escribir instrucciones. En un principio esto se hizo un tanto artesanalmente, aunque la descripción de algoritmos ya ocurría. Así, se llegó a la abstracción de los lenguajes, que son conjuntos de cadenas.

La descripción de lenguajes mediante gramáticas ya ocurría desde muchos siglos antes, pero no es hasta la década de 1950 cuando se desarrolla la Teoría de los Lenguajes Formales. Desde los primeros años surgieron diversos modelos matemáticos bien formalizados para la descripción de lenguajes. De todos, los más importantes son las gramáticas y las máquinas de estado finito.

Las *gramáticas* son un modelo matemático estructurado que definen la manera como las cadenas de un lenguaje se deben formar. La forma de definirlo es mediante reglas (gramaticales) de la forma $\alpha \rightarrow \beta$, donde tanto α como β pueden ser símbolos terminales o variables. Con tales reglas, la gramática genera cadenas. Se emplean en el diseño de todo lenguaje de programación y son esenciales en el proceso de compilación, cuando se verifica que el código de un programa es correcto gramaticalmente y se puede pasar a traducir a lenguaje

máquina.

Las *máquinas de estado finito*, llamadas más cortamente autómatas, se representan como grafos dirigidos en los que las aristas están etiquetadas por los símbolos del alfabeto con los que se forman las cadenas, que son rutas en cada máquina, de un (único) estado inicial a un estado final. Dada una cadena, el autómata parte del estado inicial y la cadena se acepta si se llega a un estado final. Todo compilador es un autómata especializado en uno o varios lenguajes.

Gramáticas y autómatas son modelos equivalentes. Cambiando sus propiedades es posible pasar de lenguajes sencillos a lenguajes con mayor poder de expresión.

1.1. Lenguajes

1.1.1. Alfabetos y cadenas

Sea Σ un conjunto finito no vacío de símbolos al que llamaremos *alfabeto*.

Aunque cualquier caracter puede ser considerado como símbolo, en general utilizaremos letras minúsculas y dígitos, como el conjunto $\{0, 1\}$ o $\{a, b, c\}$.

Ejemplo 1. *Los alfabetos ASCII o Unicode son ejemplos de alfabetos, de uso muy difundido en el mundo de la computación.*

Ejemplo 2. *El código morse, que asocia una secuencia única no vacía de rayas y puntos a cada letra y número, es un alfabeto.*

Algunas propiedades y operaciones

Una *cadena* o *palabra* es el resultado de concatenar símbolos de un alfabeto. Por ejemplo, las cadenas 00101 y 11001 son cadenas sobre el alfabeto $\Sigma = \{0, 1\}$. Más formalmente, tenemos nuestra primera definición:

Definición 1. *La concatenación de dos símbolos de un alfabeto se escribe como $a \cdot b$.*

La concatenación, definida por ahora para dos símbolos, se extenderá para concatenar un símbolo con una cadena y luego dos cadenas, utilizando siempre la misma notación, que por lo común se omite.

Aunque los alfabetos sean finitos, se puede generar una infinidad de cadenas a partir de ellos.

El sencillo concepto de cadena pueden formalizarse de manera inductiva, como se muestra a continuación.

Definición 2. *Una cadena es vacía, denotada por ϵ , o es un símbolo a del alfabeto, concatenado con una cadena, esto es, la cadena w se forma concatenando el símbolo a con la subcadena x , es decir, $w = a \cdot x$.*

Con esta definición, se construyen cadenas uniendo símbolos por la izquierda a cadenas ya formadas. Es importante notar que no se habla de agregar símbolos por la derecha, sólo se agregan por la izquierda.

Ejemplo 3. La palabra *hola* se expresa formalmente como $h \cdot (o \cdot (l \cdot (a \cdot \epsilon)))$, o más simplemente $h \cdot o \cdot l \cdot a \cdot \epsilon$.

Es razonable, a partir del ejemplo anterior, que sea preferible omitir el símbolo de concatenación y su asociatividad, escribiendo simplemente **hola**. Aunque cada cadena tiene la cadena vacía al final, se acostumbra omitirla, a menos que se requiera explícitamente, por ejemplo **hola** ϵ .

Definición 3. La longitud de una cadena es la cantidad de símbolos que la componen y se denota por $|w|$ la longitud de la cadena w .

Por ejemplo la cadena $w = 01110$ tiene longitud 5. La cadena vacía es la cadena que se compone de cero símbolos y se denota por ϵ (algunos autores prefieren λ u otros símbolos). Como se espera, la cadena vacía tiene longitud 0.

Ahora se procede a definir la concatenación de dos cadenas, a partir de la definición anterior. De nuevo se usa el método inductivo.

Definición 4. La concatenación de dos cadenas r y s se define como

$$r \cdot s = \begin{cases} s & \text{si } r = \epsilon \\ a \cdot (w \cdot s) & \text{si } r = a \cdot w \end{cases} \quad (1.1)$$

Esta definición establece que la concatenación de cadenas $r \cdot s$ se realiza separando uno a uno los símbolos iniciales de r y concatenando con s al terminar, mediante la definición 2. Nótese: siempre es de izquierda a derecha. El ejemplo siguiente lo ilustra.

Ejemplo 4. Para concatenar «*abc*» con «*def*» se procede paso a paso como sigue.

$$\begin{aligned} ab\epsilon \cdot def &= a \cdot (b\epsilon \cdot def) \\ &= a \cdot (b \cdot (\epsilon \cdot def)) \\ &= a \cdot (b \cdot (c \cdot (\epsilon \cdot def))) \\ &= a \cdot (b \cdot (c \cdot def)) \\ &= a \cdot (b \cdot (cdef)) \\ &= a \cdot (bcdef) \\ &= abcdef \end{aligned}$$

La definición 2 se aplica a partir de la tercera línea. El símbolo ϵ se ha colocado al final de la cadena inicial para poder emplear la definición. En la segunda cadena, está de manera implícita.

Es sencillo ver que la concatenación de cadenas es asociativa y lo podemos anunciar como una proposición formal.

Proposición 1. Sean r , s y t cadenas de Σ^* . Se cumple que

$$(r \cdot s) \cdot t = r \cdot (s \cdot t) \quad (1.2)$$

Demostración. Para probar, basta definir cada cadena con símbolos como

$$r = r_1 r_2 \cdots r_m$$

concatenar y reasociar. Los detalles se dejan como ejercicio. $\square$

Ejemplo 5. Dadas las cadenas $w_1 = aab$ y $w_2 = bcab$, su concatenación se escribe, de manera más económica como $w_1 \cdot w_2 = aabbcab$, evitando el detalle de las operaciones, evitando tanto los paréntesis como el operador $\cdot$, a menos que sea necesario para hacer algo explícito.

En las definiciones anteriores se establece la forma de construir cadenas, concatenando símbolos a cadenas, comenzando (o terminando) con la cadena vacía. Es natural pensar que la cadena vacía pudiera concatenarse con cualquiera y dar como resultado la cadena original, es decir, $w\epsilon = \epsilon w = w$, pero esto debe probarse y no obrar por intuición.

Proposición 2. Para toda cadena w , se cumple que $w \cdot \epsilon = w$.

Demostración. Es sencillo aplicar inducción sobre la longitud de w .

Como caso base, $|w| = 0$ cuando se trata de la cadena vacía y por la definición 2, $w \cdot \epsilon = \epsilon \cdot \epsilon = \epsilon$.

Como hipótesis de inducción, suponemos que para toda cadena w de longitud menor o igual que n se cumple que $w\epsilon = w$. Lo que hay que demostrar es que toda cadena de longitud $n + 1$ también lo cumple.

Sea $w = a \cdot x$ una cadena de longitud $n + 1$, con a un símbolo y x una subcadena de longitud n .

$$\begin{aligned} w \cdot \epsilon &= (a \cdot x) \cdot \epsilon \\ &= a \cdot (x \cdot \epsilon) && \text{Definición 1.1} \\ &= a \cdot x && \text{Hip. de Inducción} \\ &= w \end{aligned}$$

$\square$

Si bien no es directamente asociado con las cadenas, es importante detenerse para analizar una la técnica de definición y demostración muy común en el estudio de los lenguajes formales. Ya se ha empleado la definición y demostración inductiva. Esto se puede aplicar junto con la definición de operaciones cerradas y será fácil de ver con un ejemplo práctico.

Ejemplo 6. Considérense las cadenas que representan expresiones aritméticas con paréntesis bien emparejados, como $(a - (b + a))/(c - a)$. El alfabeto consta de dígitos, letras, operadores y paréntesis. Concentrémonos exclusivamente en los paréntesis. Este conjunto de cadenas, que se denotará por P . Los paréntesis bien emparejados se definen inductivamente de la siguiente manera.

1. $\epsilon \in P$

2. Todo dígito o letra pertenece a P
3. Si la cadena $w \in P$ entonces $(w) \in P$
4. Si r y s son cadenas de P , entonces su concatenación $r \cdot s$ también es cadena de P .

Se trata de operaciones cerradas, es decir, cada que hacemos algo válido con cadenas del lenguaje, concatenarlas o ponerle paréntesis, la nueva cadena sigue estando en el lenguaje, por lo que los paréntesis están correctamente anidados. Puede demostrarse por inducción que cada cadena generada tiene paréntesis bien emparejados.

Demostración. Sea $w \in P$. Nuestra inducción se hace sobre la longitud de w . El caso base es cuando $|w| \leq 1$, es decir, cuando w es la cadena vacía o un símbolo. Por definición, w pertenece al lenguaje P y además, al no tener paréntesis, estos están bien emparejados¹.

Por hipótesis de inducción, supongamos que cada cadena w con $|w| \leq n$ pertenece al lenguaje. Ahora falta realizar las operaciones de la definición inductiva, cada una por separado, revisando que el resultado sigue siendo una cadena de P con paréntesis bien emparejados. Sean r y s dos cadenas con longitud n o menor.

3. Debe ser evidente que, como r tiene los paréntesis bien emparejados, la cadena (r) también debe tener los paréntesis bien emparejados.
4. La concatenación $r \cdot s$ no modifica la cantidad de paréntesis, por lo que también siguen correctamente emparejados.

□

Definición 5. Se define y denota como Σ^k al conjunto de todas las cadenas de longitud k sobre el alfabeto Σ .

Ejemplo 7. Por ejemplo, si $\Sigma = \{0, 1\}$, entonces

$$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}.$$

Definición 6. El conjunto de todas las cadenas que se pueden formar a partir de un alfabeto Σ se denota por Σ^* , conocida como la estrella de Kleen o cerradura de Kleen. De esta manera,

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \dots$$

Dado que Σ^* incluye la cadena vacía, si se le desea excluir del lenguaje se escribe Σ^+ , lo que elimina Σ^0 de Σ^* . Debe entenderse que el lenguaje vacío no contiene cadenas, por lo que la cadena vacía no pertenece al lenguaje vacío.

¹Este tipo de argumento es por vacuidad.

Ejemplo 8. Si el alfabeto Σ son las letras de la A a la Z, entonces Σ^* son todas las combinaciones de letras, incluyendo (de manera mayoritaria) las que no tienen significado en ningún lenguaje natural de ningún país.

A una cadena $w \in \Sigma^*$ se le pueden invertir sus símbolos, de tal forma de convertir **abcd** en **dcba**. Esta operación es llamada reversa y se define formalmente a continuación.

Definición 7. Sea w una cadena. Su reversa se define y denota como

$$\text{rev}(w) = \begin{cases} \epsilon & \text{si } w = \epsilon \\ \text{rev}(x) \cdot a & \text{si } w = a \cdot x \end{cases} \quad (1.3)$$

donde $a \in \Sigma$ es un símbolo y $x \in \Sigma^*$ es una subcadena (sufijo) de w que contiene todos los símbolos menos 1.

Ejemplo 9. Calculemos la reversa de **abc**:

$$\begin{aligned} \text{rev}(abc) &= \text{rev}(a \cdot bc) \\ &= \text{rev}(bc) \cdot a \\ &= \text{rev}(b \cdot c) \cdot a \\ &= (\text{rev}(c) \cdot b) \cdot a \\ &= (\text{rev}(c \cdot \epsilon) \cdot b) \cdot a \\ &= ((\text{rev}(\epsilon) \cdot c) \cdot b) \cdot a \\ &= ((\epsilon \cdot c) \cdot b) \cdot a \\ &= cba \end{aligned}$$

Proposición 3. Dadas dos cadenas r y s , se cumple que

$$\text{rev}(r \cdot s) = \text{rev}(s) \cdot \text{rev}(r).$$

Demostración. Se procederá por inducción en la longitud de las cadenas. Primero el caso base. Cuando ambas cadenas miden 0 de longitud, el resultado se cumple trivialmente. Podría usarse el caso en que la longitud es uno e igualmente es sencillo de la definición de la función rev . Consideraremos dos casos: cuando la cadena r aumenta en un símbolo y cuando la cadena s es la que aumenta su longitud en 1.

Como hipótesis de inducción, supongamos que $\text{rev}(r \cdot s) = \text{rev}(s) \cdot \text{rev}(r)$ es cierto para cadenas con longitud n o menor.

Caso 1: Sea $|r| = n + 1$ y $r = a \cdot x$, con $a \in \Sigma$ y $x \in \Sigma^*$.

$$\begin{aligned} \text{rev}(r \cdot s) &= \text{rev}(a \cdot x \cdot s) \\ &= \text{rev}(x \cdot s) \cdot a && \text{Definición 1.3} \\ &= \text{rev}(s) \text{rev}(x) \cdot a && \text{Hipótesis de inducción} \\ &= \text{rev}(s) \cdot \text{rev}(r) && \text{Definición 1.3} \end{aligned}$$

Con este caso queda claro que si las cadenas de n símbolos o menos cumplen con la propiedad, entonces también ocurre cuando la primera de las cadenas, r es mayor que n .

Caso 2: Sea ahora $|s| = n + 1$ y $s = x \cdot a$, con a y x como en el primer caso.

$$\text{rev}(r \cdot s) = \text{rev}(r \cdot (x \cdot a)) \quad (1.4)$$

$$= \text{rev}((r \cdot x) \cdot a) \quad \text{Proposición 1} \quad (1.5)$$

$$= \text{rev}(a) \text{rev}(r \cdot x) \quad \text{Por el caso anterior} \quad (1.6)$$

$$= \text{rev}(a) \text{rev}(x) \text{rev}(r) \quad \text{Por el caso anterior} \quad (1.7)$$

$$= \text{rev}(x \cdot a) \text{rev}(r) \quad (1.8)$$

$$= \text{rev}(s) \text{rev}(r) \quad (1.9)$$

que es lo que se quería demostrar. $\square$

1.1.2. Lenguajes

Con el concepto de cadenas bien entendido, entramos a las amplias aguas de los lenguajes y comenzamos con su definición.

Definición 8. *Un lenguaje es un conjunto arbitrario de cadenas definidas sobre un alfabeto fijo Σ .*

Es claro que Σ^* , todas las cadenas que es posible construir a partir de los símbolos de Σ , es un lenguaje, pero generalmente un lenguaje (de interés) L es un subconjunto propio de Σ^* que cumple con características especiales. Se dice que $L \subsetneq \Sigma^*$ es un lenguaje sobre Σ . El conjunto vacío es un lenguaje sin cadenas y es distinto de $\{\epsilon\}$, el lenguaje que posee sólo a la cadena vacía. Ambos se consideran lenguajes sobre cualquier alfabeto.

Por sencillo que parezca, el concepto siguiente es fundamental en la Teoría de la Computación.

Definición 9. *Un problema es toda pregunta que pretende decidir si una cadena pertenece o no a un lenguaje.* ★

Todo problema en computación (y en matemáticas) puede traducirse a esta definición que se formaliza diciendo: sea Σ un alfabeto y L un lenguaje sobre Σ . Entonces, un *problema* en L es:

$$\text{Dada una cadena } w \in \Sigma^* \text{ decidir si ocurre o no que } w \in L. \quad (1.10)$$

De hecho, hablar sobre lenguajes o problemas es la misma cosa, todo depende del punto de vista como se estén abordando. Si se estudian lenguajes, las cadenas importan por sí mismas, mientras que en computabilidad, cada cadena representa un problema. Décadas de uso han probado que esta es una manera adecuada de enfrentar las preguntas importantes de la Teoría de la Computación, que permite además clasificarlas según la dificultad para determinar su veracidad o no.

Para definir un lenguaje sobre el alfabeto Σ es posible utilizar diversas notaciones. Una vez definido el lenguaje, se puede emplear cualquiera de las formas de descripción de conjuntos, como a continuación se muestra.

Ejemplo 10. *Varios lenguajes sobre alfabetos distintos.*

$$\begin{aligned} L_1 &= \{aab, abb, aca, bac\} \text{ sobre el alfabeto } \Sigma = \{a, b, c, d\} \\ L_2 &= \{w \mid w \text{ es una cadena binaria que inicia con } 1\} \text{ sobre } \Sigma = \{0, 1\} \\ L_3 &= \{w \mid w \text{ es un programa válido en lenguaje C}\} \\ L_4 &= \{a^n b^{n+1} \mid a \text{ y } b \text{ son dígitos}\} \\ L_5 &= \{for, goto, gosub, next, return\} \\ L_6 &= \{100, 105, 110, 115, \dots, 190, 195\} \text{ sobre } \Sigma = \{0, 1, \dots, 9\} \end{aligned}$$

El primero es un lenguaje finito, mientras que los siguientes tres son infinitos². En el caso de L_3 , las cadenas son programas en lenguaje C, por lo que los caracteres con que se forman no son símbolos simples, sino compuestos, como palabras reservadas u operadores como `+=`. L_5 es una colección de palabras reservadas de una versión antigua de Basic y puede suponerse que el alfabeto son los caracteres individuales o que las palabras mismas son parte del alfabeto, que es lo que generalmente se supone en lenguajes de programación.

Con la definición dada de lenguaje, ahora es posible proceder a clasificarlos, partiendo de los más simples a los más complejos. Es posible hacer esto de dos formas:

- definiendo las reglas que generan el lenguaje L o
- definiendo una máquina que sólo acepta las cadenas de L .

Ambos enfoques son equivalentes, necesarios y complementarios.

1.1.3. Operaciones con lenguajes

Siendo los lenguajes conjuntos, es natural que se hereden las operaciones unión, intersección y complemento, considerando el lenguaje Σ^* como el universo. Estas operaciones se aplican tal y como se acostumbran. Para cada operación, en el contexto de los lenguajes se le da una interpretación y notación conveniente.

En el caso de la unión de dos lenguajes L_1 y L_2 , se acostumbra utilizar como notaciones equivalentes $L_1 \cup L_2$ y $L_1 + L_2$, preferentemente esta última en teoría de lenguajes formales.

Adicionalmente, para lenguajes se definen la concatenación y la cerradura. Sean L_1 y L_2 lenguajes.

²Aunque en la práctica, los programas válidos en C son un conjunto finito debido a las limitaciones técnicas de los compiladores, pero en la teoría de lenguajes siempre se supone medios de almacenamiento infinitos.

Definición 10. La concatenación de los lenguajes L_1 y L_2 se define como la concatenación de cada pareja posible de cadenas de ambos.

$$L_1 L_2 = \{r \cdot s / r \in L_1 \text{ y } s \in L_2\}$$

En caso de concatenar un lenguaje consigo mismo, equivale a escribir $LL = L^2$. Debe ser claro que tanto L como L^2 son subconjuntos de Σ^* . De aquí deriva la definición de

Definición 11. La potencia n de un lenguaje L es la concatenación de n copias de L .

Ejemplo 11. Si $\Sigma = \{0, 1\}$ y $L = \{00, 1\}$, entonces

$$L^0 = \{\epsilon\} \tag{1.11}$$

$$L^1 = \{00, 1\} \tag{1.12}$$

$$L^2 = \{0000, 001, 100, 11\} \tag{1.13}$$

$$\vdots \tag{1.14}$$

Definición 12. La cerradura del lenguaje L se define como la unión de todas sus potencias:

$$L^* = \bigcup_{i=0}^{\infty} L^i$$

Es importante entender que el conjunto vacío $\emptyset$ es distinto del lenguaje que sólo posee la cadena vacía $\{\epsilon\}$.

1.2. Autómatas Finitos Determinísticos

Un *autómata finito determinístico* (AFD) es un formalismo o modelo matemático que simula una máquina que lee caracteres desde una cinta de entrada de longitud infinita y avanza sobre cada uno. La máquina *acepta o no* la cadena en la cinta de entrada, significando que dicha cadena pertenece o no al lenguaje. De esta forma, las cadenas aceptadas pertenecen al lenguaje y las no aceptadas están fuera de este.

Formalmente, un AFD consta de:

1. Q un conjunto de estados,
2. Σ un alfabeto fjo,
3. $\delta : Q \times \Sigma \rightarrow Q$ la función de transición,
4. $q_0 \in Q$ el estado inicial y
5. $F \subset Q$ el conjunto no vacío de los estados finales o de aceptación.

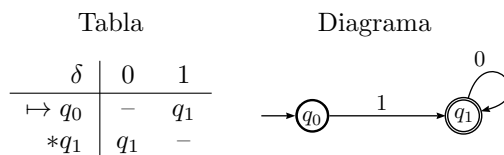


Tabla 1.1: Dos formas de representar las transiciones para reconocer las potencias de 2 en binario.

De esta manera, un AFD es una quintupla $(Q, \Sigma, \delta, q_0, F)$ junto con una cinta de entrada donde se escribe una cadena de caracteres que será procesada.

La palabra determinístico se refiere a que δ es una función, pues a cada pareja (q, x) le asocia un único estado en Q . De esta manera, la transición siempre es fija desde un estado con el mismo símbolo. Las restricciones en la definición del AFD determinan el tipo de lenguajes que es capaz de procesar. Más adelante, se definirán autómatas más poderosos que son capaces de aceptar una cantidad más amplia de lenguajes.

Si el estado inicial q_0 es a su vez uno de los estados finales ($q_0 \in F$), entonces eso significa que el AFD acepta la cadena vacía ϵ pues no se requiere ningún símbolo para llegar a un estado de aceptación.

Es posible que existan estados a los que no es posible llegar a partir del estado inicial. Estos se consideran *estados inalcanzables* y pueden ser omitidos del autómata.

1.2.1. Representación de AFD

Hay dos maneras de representar AFD, con *tablas de transición*, escribiendo una matriz donde a cada fila le corresponde un estado y a cada columna un símbolo de Σ , o con un *diagrama de transición*, dibujando un grafo que represente los estados y las transiciones entre ellos.

La tabla 1.1 muestra las dos versiones de representación del mismo autómata. Se usa la convención de que el estado inicial se indica con $\mapsto$ y los estados finales con $*$. El guión (símbolo —) se emplea para las indicar transiciones no definidas. En el diagrama, la flecha volante que llega a q_0 indica el estado inicial y el círculo de doble línea indica estados finales.

Las tablas de transiciones son las adecuadas al momento de programar computadoras que simulen autómatas. Un archivo de texto con la tabla de transiciones es más práctico, pero el diagrama es más efectivo para reconocimiento humano. Unos ejemplos mostrarán más sobre las representaciones.

Ejemplo 12. Sea el AFD $(Q, \Sigma, \delta, q_0, F) = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_1\})$, con dos transiciones definidas como $\delta(q_0, 1) = q_1$ y $\delta(q_1, 0) = q_1$. Este autómata de 2 estados acepta las cadenas de ceros y unos que comienzan con un 1 y terminan con puros ceros, es decir, las potencias de 2 en binario. Este autómata es el que se representa en la tabla 1.1. El estado q_0 es inicial y q_1 el único final.

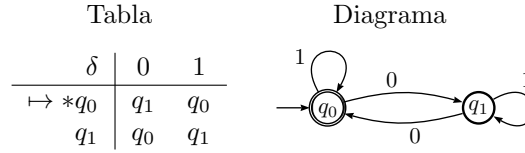


Tabla 1.2: Tabla y diagrama de transiciones de un autómata que reconoce cadenas de ceros y unos que tienen una cantidad par de ceros.

En este ejemplo, si se intenta con las cadenas 1, 10, 100, 1000 y todas las de esa forma, se llega al estado q_1 y se aceptan las cadenas. Si la cadena de entrada es 10010, al intentar procesar el segundo 1 el autómata queda sin poder proceder, pues no hay transición en el estado final para el símbolo 1, por lo que la cadena no se acepta.

Ejemplo 13. Sea el AFD $(Q, \Sigma, \delta, q_0, F) = (\{q_0, q_1\}, \{0, 1\}, \delta, q_0, \{q_0\})$, con dos transiciones definidas como $\delta(q_0, 1) = q_0$, $\delta(q_0, 0) = q_1$, $\delta(q_1, 1) = q_1$ y $\delta(q_1, 0) = q_0$. Este autómata acepta las cadenas de ceros y unos que tienen una cantidad par de ceros. Este autómata es el que se representa en la tabla 1.2. De los dos estados, el q_0 es inicial y final a la vez.

Para este caso, las cadenas 00, 100, 010, 001 y todas sus combinaciones serán aceptadas. Incluso la cadena vacía se acepta, pues el estado inicial es final a la vez.

El ejemplo 13 es interesante pues muestra una técnica común en el diseño de autómatas. El estado q_0 identifica cadenas con cantidad par de ceros, mientras que el estado q_1 reconoce las de cantidad impar. Así, es posible tener estados para identificar cadenas con varias características a la vez. El siguiente ejemplo es común en el reconocimiento de lenguajes.

Ejemplo 14. Consideremos el lenguaje formado por todas las cadenas de símbolos en el alfabeto $\Sigma = \{a, b, c, \dots, y, z\}$ que contienen la subcadena aei al menos una vez. Por ejemplo las cadenas $kaeiyl$, aei , $aaeeaeiaei$, $kihraejaei$ y similares.

En este caso, el AFD que acepta tal lenguaje debe tener algún camino de estados que asegure la presencia de la subcadena de interés, al tiempo de permitir la formación de cualquier cadena válida. La figura 1.1 muestra el AFD, donde la ruta $q_0 \rightarrow q_1 \rightarrow q_2$ permite identificar la subcadena aei .

1.2.2. Completitud de la función de transición

Como se observa de la representación en la tabla 1.1, la función de transición no está definida para los casos $\delta(q_0, 0)$ y $\delta(q_1, 1)$. Estas entradas en la tabla de transición solo tienen un guión. Eso motiva una definición importante.

Definición 13. Una función de transición es completa si está definida para toda combinación de valores de su dominio $Q \times \Sigma$.

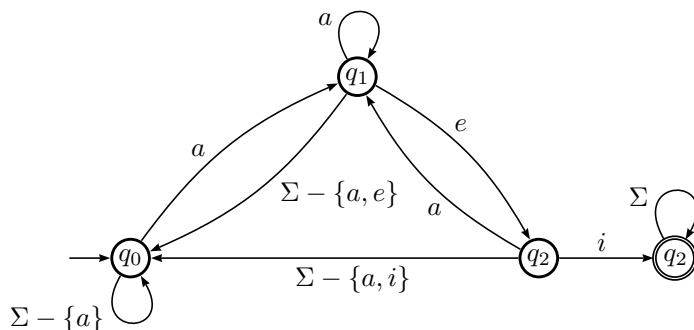


Figura 1.1: AFD que acepta cadenas que contienen la subcadena «aei».

Tabla			Diagrama		
δ	0	1			
$\mapsto q_0$	q_2	q_1			
$*q_1$	q_1	q_2			
q_2	q_2	q_2			

Tabla 1.3: Autómata completo, presentado incompleto en el ejemplo 12.

Retomemos el ejemplo 12 y completemos la función de transición en el ejemplo 15.

Ejemplo 15. Al AFD del ejemplo 12 se le completa agregando un estado no final q_2 y las transiciones

$$\delta(q_0, 0) = q_2$$

$$\delta(q_1, 1) = q_2$$

$$\delta(q_2, 0) = q_2$$

$$\delta(q_2, 1) = q_2.$$

Las últimas dos transiciones se agregan debido a que de no hacerlo, δ estaría incompleta para el estado q_2 . Este nuevo AFD se representa en la tabla 1.3.

Completar la función de transición de un AFD es importante para ciertas operaciones sobre lenguajes que se estudiarán más adelante.

1.2.3. Autómata Procesando Cadenas

Sea la cadena 11010 escrita en la cinta de entrada. Un AFD inicia su ejecución en el estado q_0 , y lee el primer símbolo en la cadena de entrada, el 1. Si $\delta(q_0, 1) =$

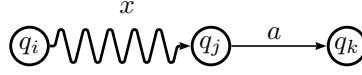


Figura 1.2: La regla (1.16) procesa la cadena x desde q_i , pasando por $|x|$ transiciones hasta llegar al estado q_j . Luego transita con $\delta(q_j, a) = q_k$.

q_i , entonces se cambia al estado q_i en espera de la lectura del siguiente símbolo, el segundo 1. Si no está definida la transición $\delta(q_0, 1)$ entonces la cadena se rechaza. El proceso continúa, leyendo cada símbolo en la cinta de entrada y cambiando de estado conforme la función de transición, hasta que la cadena termina. Si la cadena se termina y el estado actual, digamos q_f , es uno de los estados finales, es decir, $q_f \in F$, entonces la cadena 11010 es *aceptada* por el AFD.

Si en el transcurso de la ejecución se llega a un estado a partir del cuál no está definida la transición para el símbolo que se lee, el autómata se detiene y rechaza la cadena. Si la cadena se termina pero el autómata no está en uno de los estados finales, la cadena tampoco es aceptada.

Aunque la función de transición procesa caracter por caracter, en general es más cómodo pensar que procesa cadenas completas, por ejemplo $\delta(q_0, 11010)$ y que el resultado sea el estado al que se llega con la cadena. Esto puede formalizarse inductivamente. Sea w una cadena formada por símbolos de Σ tal que

$$w = xa$$

con $a \in \Sigma$ el último símbolo de w . A la subcadena x se le llama prefijo. Sea $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ una función auxiliar definida recursivamente como

$$\hat{\delta}(q, \epsilon) = q \quad (1.15)$$

$$\hat{\delta}(q, w) = \delta(\hat{\delta}(q, x), a) \quad (1.16)$$

El resultado de procesar la cadena w es el estado $q_k = \hat{\delta}(q_i, w)$. Lo que hace la regla (1.16) es procesar la subcadena x a partir de q_i con $\hat{\delta}$ para llegar al estado q_j y luego utilizar δ para procesar el símbolo a y llegar al estado destino con la transición normal $\delta(q_j, a) = q_k$. Esta transición genérica se esquematiza en la figura 1.2.

Ejemplo 16. *Considérense de nuevo el lenguaje L_6 del ejemplo 10, las cadenas de 3 dígitos que representan múltiplos de 5 desde 100 hasta 195. Sea el autómata AFD $A = (Q, \Sigma, \delta, q_0, F)$ que acepta cadenas de ese tipo. Del estado q_0 , se requiere una transición que procese el primer dígito, que debe ser un 1, con $\delta(q_0, 1) = q_1$. Luego del estado q_1 debe ser posible pasar al estado q_2 con cualquier dígito. Del estado q_2 se debe poder pasar al estado final q_3 con un 0 o con un 5.*

Es deseable pensar en las transiciones que sucesivamente tienen que darse para una cadena, por ejemplo 125. $\hat{\delta}(q_0, 125)$ se calcula como sigue, utilizando las reglas (1.15) y (1.16).

La cadena $w = 125$ se descompone primero como $x = 12$ y $a = 5$, luego se repite el proceso, descomponiendo la cadena 12 como $x = 1$ y $a = 2$. Finalmente, la cadena $w = 1$ se descompone como $x = \epsilon$ y $a = 1$ siendo el caso base. A partir de allí, sólo se sustituyen las transiciones definidas para δ .

Los pasos son:

$$\begin{aligned}
 \hat{\delta}(q_0, 125) &= \delta(\hat{\delta}(q_0, 12), 5) & x = 12 \quad a = 5 \\
 &= \delta(\delta(\hat{\delta}(q_0, 1), 2), 5) & x = 1 \quad a = 2 \\
 &= \delta(\delta(\delta(\hat{\delta}(q_0, \epsilon), 1), 2), 5) & x = \epsilon \quad a = 1 \\
 &= \delta(\delta(\delta(q_0, 1), 2), 5) & \text{por la regla (1.15)} \\
 &= \delta(\delta(q_1, 2), 5) & \delta(q_0, 1) = q_1 \\
 &= \delta(q_2, 5) & \delta(q_1, 2) = q_2 \\
 &= q_3
 \end{aligned}$$

★

Viendo que es formalmente posible definir cómo un AFD procesa cadenas, en lo sucesivo nos referiremos indistintamente a δ como si procesara símbolos individuales o cadenas completas.

1.2.4. El lenguaje de un AFD

De manera empírica se ha mencionado que el AFD acepta cadenas, sin mencionar el lenguaje que esto define. Es momento de formalizar.

Definición 14. El lenguaje que acepta el AFD $A = (Q, \Sigma, \delta, q_0, F)$, denotado con $L(A)$, se define como

$$L(A) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F\} \quad (1.17)$$

que son todas las cadenas del alfabeto que al ser procesadas, iniciando en el estado q_0 , terminan en algún estado final del AFD. Se dice que $L(A)$ es un lenguaje regular.

Con esta definición, entonces el lenguaje definido en el ejemplo 12 es un lenguaje regular pues es el aceptado por un AFD. Cualquiera que sea el AFD definido, el conjunto de cadenas que partiendo del estado inicial alcancen un estado final al procesar el último símbolo es un lenguaje regular.

Ejemplo 17. El autómata que reconoce el lenguaje $\{Hola, mundo\}$ es el de la figura 1.3. De esta manera, es posible definir autómatas que acepten las palabras reservadas de un lenguaje.

Los AFD se utilizan constantemente en la práctica. Desde la verificación de la entrada válida de un usuario o los primeros pasos que todo compilador da, al realizar lo que se conoce como el análisis léxico, que es el reconocimiento de cada elemento individual en el código de un programa, como operadores,

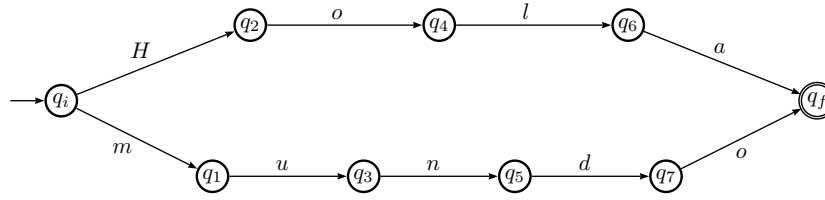


Figura 1.3: Autómata que reconoce un lenguaje con sólo dos cadenas.

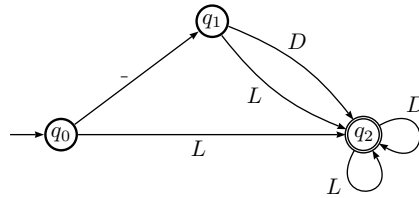


Figura 1.4: Autómata que acepta identificadores de un lenguaje de programación. La D representa al conjunto de los dígitos y L el de las letras.

palabras reservadas, identificadores del usuario, constantes, delimitadores de ámbito y todo aquello de que consta cierto lenguaje. Cada elemento de algún tipo es llamado token.

Ejemplo 18. *Un identificador es una cadena alfanumérica que se utiliza para nombrar una función, variable, etiqueta o macro. Todo identificador comienza con una letra o el símbolo $_$ y puede continuar con letras y números. Si comienza con $_$ simple o doble, debe tener al menos una letra³. Suponiendo que no hay restricciones en la longitud de los identificadores, se puede hacer un AFD T que reconozca sólo cadenas válidas.*

Sea L el conjunto de las letras válidos para usarse en un identificador y D el conjunto de los números del 0 al 9. Sea el alfabeto $\Sigma = \text{Letra} \cup \text{Dígito} \cup \{-\}$, es decir, toda letra, dígito (del cero al nueve) y el guión bajo.

$T = (\{q_0, q_1, q_2\}, \Sigma, \delta, q_0, \{q_2\})$ y $\delta(q_0, -) = q_1$, $\delta(q_0, x) = q_2$ para toda letra $x \in \text{Letra}$ y además $\delta(q_1, x) = q_2$ para toda x elemento de $\text{Letra} \cup \text{Dígito}$. A este autómata le corresponde el diagrama de transiciones de la figura 1.4.

³Existe toda una variedad de reglas y convenciones bien establecidas, además de las variantes que cada vendedor de compiladores ofrece, sobre cómo nombrar identificadores. Por ejemplo, en C las variables que comienza con doble guión bajo sólo las debe usar el compilador mismo.

1.2.5. Descripción instantánea de un AFD

Cuando un autómata está procesando una cadena, es posible conocer el estado que guarda la lectura de los símbolos. Sea la cadena que se procesa

$$w = a_1 a_2 a_3 \cdots a_r$$

con $a_i \in \Sigma$ y con i de 1 a r . La descripción instantánea del autómata se compone del estado actual q_j y el resto de la cadena que está por ser procesada.

Es claro que al inicio, en el estado inicial q_0 y con toda la cadena w por ser procesada, la descripción instantánea es

$$q_0 a_1 a_2 a_3 \cdots a_r$$

y si existe la transición $\delta(q_0, a_1) = q_k$, entonces la siguiente descripción instantánea es

$$q_k a_2 a_3 \cdots a_r$$

donde el símbolo inicial a_1 ya fue procesado.

Para un AFD y cierta cadena w , hay una única sucesión de descripciones instantáneas, que termina con $q_f \epsilon$, con q_f uno de los estados finales, en caso de que la cadena sea aceptada.

1.2.6. AFD mínimos

Es posible construir muchos AFD que acepten el mismo lenguaje. Lo ideal es hacer uso del más sencillo de estos. Por lo general, se da preferencia a los que tienen menos estados, por lo que se requiere un criterio para reducir el número de estados de un AFD sin afectar el lenguaje que acepta. El criterio es el siguiente.

Definición 15. *Dado un AFD y dos de sus estados que no son finales, p y q , se dice que son equivalentes si para toda cadena de entrada, $\delta(p, w) \in F$ si y sólo si $\delta(q, w) \in F$.*

Esto significa que para cada cadena w , llegamos a un estado final partiendo de cualquiera de los dos estados. Si la cadena no es aceptada, ninguno nos llevará a un estado de aceptación.

Cuando dos estados son equivalentes, pueden ser sustituidos por un único estado, que equivale a eliminar uno de los dos. Además, se puede probar que la equivalencia de estados es transitiva, así que si hay más de un estado que es equivalente, todos pueden ser sustituidos por un único estado.

1.2.7. Análisis Léxico

En análisis léxico es un problema al que se enfrenta tanto un compilador como un editor de textos, un corrector ortográfico, un buscador de bases de datos, un navegador de internet y toda aplicación que utiliza cadenas de caracteres para identificar patrones o similitudes. Es la aplicación más directa de los AFD.

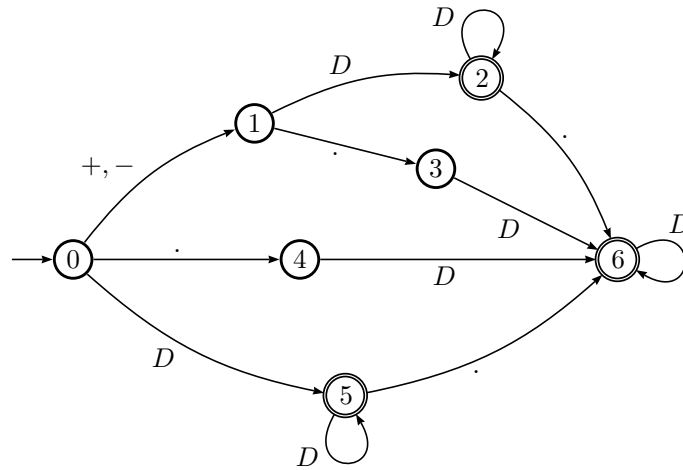


Figura 1.5: Autómata Finito Determinístico que acepta números flotantes, del ejemplo 19. En la transición del estado 0 al 1, por etiqueta $+, -$ debe entenderse que cualquier signo es válido para la transición. La D hace referencia a cualquier dígito.

El código de un analizador léxico se basa directamente en la tabla de transiciones del AFD, solo que esta debe completarse con las salidas adecuadas para cada entrada de la tabla donde no hay transición definida, lo que hasta ahora se ha marcado con el signo $-$.

Lo que hace el analizador léxico es asignar a una variable el índice del estado actual y actualizarla cada ciclo hasta que se termine la cadena y comprobar que se terminó en un estado final.

Ahora ya se ve la primera aplicación práctica de las teorías descritas hasta el momento. Programas sencillos pero con gran poder se pueden desarrollar haciendo uso de teoría bien desarrollada.

Ejemplo 19. En muchos lenguajes se utilizan números de punto flotante, tales como 2.3 , -7.23 , $+0.15$, 456.23 o $.12$ y son consideradas constantes. Esas cadenas son un lenguaje regular, ya que son aceptadas por el AFD de la figura 1.5.

A partir de la tabla de transiciones se construye el analizador del código 1.1. La tabla de transiciones ha sido codificada en el arreglo bidimensional **Tabla**. El código es simple, ya que todo el conocimiento de cómo son las cadenas del lenguaje está codificado en la tabla de transiciones (el arreglo **Tabla**).

En el programa completo, cuya entrada es un arreglo de cadenas que representan flotantes, ante la entrada `123.45 -.56 rt45 6y4 2*3 654`, el programa muestra:

Analizando 123.45 ... se acepta

Código 1.1: Analizador léxico para números flotantes simples reconocidos por el autómata de la figura 1.5. La tabla de transiciones se ha ampliado para poder considerar la salida en caso de un error. Encontrar el fin de cadena (caracter nulo) es lo que da fin al ciclo, es decir, el **case** de la línea 27 manda directo fuera del ciclo (línea 32).

```

1  int Tabla[][3]={ {5, 1, 4}, {2,-1, 3},
2                    {2,-1, 6}, {6,-1,-1},
3                    {6,-1,-1},{5,-1,6},
4                    {6,-1,-1}};
5  int AnalizarFlotante(const char *Cad)
6  {
7      char c;
8      int Estado=0, Simbolo, Error=-1;
9      int Digito=0, Signo=1, Punto=2;
11
12      do {
13          c = *Cad++;
14          switch(c) {
15              case '0': case '1':
16              case '2': case '3':
17              case '4': case '5':
18              case '6': case '7':
19              case '8': case '9':
20                  Simbolo = Digito;
21                  break;
22              case '-': case '+':
23                  Simbolo = Signo;
24                  break;
25              case '.':
26                  Simbolo = Punto;
27                  break;
28              case '\0':
29                  continue;
30              default:
31                  return NO_SE_ACEPTA;
32          }
33          Estado = Tabla[Estado][Simbolo];
34          if ( Estado == Error )
35              return NO_SE_ACEPTA;
36      } while ( c != '\0' );
37
38      if ( Estado == 2 || Estado == 5 ||
39          Estado == 6 )
40          return SE_ACEPTA;
41
42      return NO_SE_ACEPTA;

```

```

Analizando -.56 ... se acepta
Analizando rt45 ... no se acepta
Analizando 6y4 ... no se acepta
Analizando 2*3 ... no se acepta
Analizando 654 ... se acepta

```

Claro, este es un ejemplo con fines didácticos, bastante simple. Un compilador toma en cuenta notación científica, sufijo, longitud de cadena y detalles que oscurecerían la idea del analizador, por lo que se omiten en el ejemplo.

Otra forma de terminar, en el caso de un AFD que tenga varios estados finales, es definiendo una transición para el caracter que simula el fin de la cadena, el caracter nulo $\backslash 0$, que lleva a un único estado final. El código del analizador habría que cambiarlo ligeramente y agregar una columna más a la tabla de transiciones.

En el caso más general de un analizador léxico que de procesar un archivo completo, las posibilidades del `switch` deben considerar los todos los caracteres, además de que los símbolos $+$ y $-$ pueden ser entendidos como operadores unarios y no parte del número.

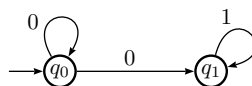
Lo que es importante notar es que si se define otro lenguaje regular, con su propia tabla de transiciones, lo único que hay que cambiar en el código del analizador es el arreglo `Tabla` y los casos del `switch`, así que cada analizador léxico tiene esa misma forma. Es un proceso tan mecánico que existen generadores automáticos de analizadores léxicos, por ejemplo Lex, Flex o Ragel. No se necesita escribir códigos como el del ejemplo anterior, sólo describir el lenguaje con una expresión regular, para que el generador construya un código que es bastante eficiente. ★

Un buen ejercicio es extender este lenguaje para que acepte notación científica.

1.2.8. Autómata finito no determinístico

Los AFD son el modelo más básico de una máquina que acepta cadenas para definir un lenguaje. Con la intención de darle poder, se pueden cambiar algunas cosas, particularmente el comportamiento de la función de transición δ . Las variantes que más interesan son las siguientes.

Consideremos el siguiente fragmento de un autómata que tiene transiciones distintas para $\delta(q_0, 0) = q_0$ y $\delta(q_0, 0) = q_1$.



¿Cómo decide el autómata qué hacer? Si la cadena inicia con 0, ¿continúa la ejecución en q_0 o se pasa a q_1 ? Es mejor pensar que el autómata no decide qué camino seguir, sino que continúa al mismo tiempo por ambas transiciones. Así, lo correcto es pensar que la transición es $\delta(q_0, 0) = \{q_0, q_1\}$, llegando a los dos estados con el mismo símbolo. ★

Este tipo de autómeta finito es NO determinístico y es similar al AFD, pero con la capacidad de estar al mismo tiempo en varios estados. Esta habilidad es la que permite modelar el *paralelismo*.

Además, para evitar conflictos con la definición de función, se define $\delta(q, 0) = \{p, q\}$, es decir, hay una única transición de $(q, 0)$ al conjunto $\{p, q\} \in 2^Q$.

Para hacer esto es necesario modificar la definición de AFD, particularmente la definición de la función de transición δ de la página 15. Ahora, como de un estado podemos ir a varios con el mismo símbolo, se define otro tipo de función de transición, una que puede ir a un conjunto de estados, por lo que su rango es el conjunto potencia de Q :

$$\delta : Q \times \Sigma \rightarrow 2^Q \quad (1.18)$$

Como ahora no está determinado a qué estado pasa el AFD con cada símbolo, llamamos al modelo *Autómata Finito No Determinístico*, o AFN. La nueva función de transición permite ir a cualquier cantidad de estados con el mismo símbolo como en

$$\delta(q, a) = \{p_1, p_2, \dots, p_k\} \subset Q$$

por eso es que el rango de δ es 2^Q , el conjunto de todos los subconjuntos de Q .

Habiendo cambiado la naturaleza de la función de transición, es necesario extenderla de nuevo para que procese cadenas de caracteres, como se hizo en la sección 1.2.3 para poder definir el lenguaje aceptado por un AFD. La extensión que se hizo con $\hat{\delta}$ lleva por un único camino de estados con una cadena, terminando en un sólo estado, pero en este caso no es así. A partir de cada estado es posible seguir varias transiciones, llegando a un conjunto de estados tras procesar la cadena.

Para que la función de transición extendida procese cadenas, sea como antes $w \in \Sigma^*$ una cadena formada por símbolos de Σ tal que $w = xa$, con $a \in \Sigma$ el último símbolo de w . Este caso se ilustra en la figura 1.6.

Sea $\hat{\delta} : Q \times \Sigma^* \rightarrow 2^Q$ la función de transición extendida, definida inductivamente, con el caso base para la cadena vacía:

$$\hat{\delta}(q, \epsilon) = \{q\}$$

y como cada transición resulta en un conjunto de estados, sea

$$\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$$

tal conjunto de estados a los que es posible llegar cuando se procesa la cadena x . Finalmente se tiene

$$\begin{aligned} \hat{\delta}(q, w) &= \bigcup_{i=1}^k \delta(p_i, a) \\ &= \{r_1, r_2, \dots, r_m\} \end{aligned} \quad (1.19)$$

El lenguaje aceptado por un AFN $N = (Q, \Sigma, \delta, q_0, F)$ es similar al de un AFD, pero en este caso, como con cada cadena se pueden seguir diversas rutas

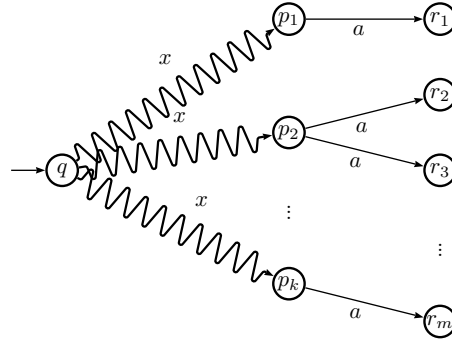


Figura 1.6: Paso inductivo de la extensión de δ a $\hat{\delta}$. Para procesar $\hat{\delta}(q, w)$ se reduce la cadena $w = xa$ en un símbolo y primero se procesa la subcadena $\hat{\delta}(q, x)$ y luego se procesa $\delta(p_i, a)$.

con las transiciones, se considera que una cadena es aceptada si al menos una de estas rutas llega a un estado final al terminar de procesar la cadena. De esta forma,

$$L(N) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\} \quad (1.20)$$

Con esta variante que modela el paralelismo los AFN parecen más poderosos que los AFD. Si esto fuera cierto, entonces, al ser más poderosos, los lenguajes aceptados por los AFD serían menos que los lenguajes aceptados por los AFN. Habría al menos un lenguaje aceptado por un AFN que no podría ser aceptado por un AFD.

Aunque el razonamiento anterior nos haría avanzar de los Lenguajes Regulares (los aceptados por los AFD) a un conjunto mayor, lo cierto es que para cada lenguaje que es aceptado por algún AFN, existe un AFD que también lo acepta, por lo que ambos modelos, el determinístico y el no determinístico son equivalentes. Si bien no son más poderosos, sí son más fáciles de construir. ★

La prueba de la equivalencia entre AFD y AFN es un algoritmo para la conversión de uno en otro. Dado un AFD, construir un AFN que acepta exactamente el mismo lenguaje y viceversa, dado el AFN, construir el AFD.

Equivalencia entre AFD y AFN

El hecho de que los AFN NO son más poderosos que los AFD, es decir, NO amplían el conjunto de lenguajes que se pueden reconocer es muy importante. Su equivalencia se plasma de manera formal.

Proposición 4. *El conjunto de lenguajes aceptados por los AFD es el mismo que los aceptados por los AFN.* ★

Demostración. Lo que hay que hacer para mostrar que $L(AFD) = L(AFN)$ es mostrar equivalencias entre los dos autómatas, es decir, dado un AFN N

arbitrario mostrar un AFD A equivalente (que acepte el mismo lenguaje) y viceversa. Es necesario construir, en ambos sentidos, un autómata adecuado y demostrar que sí acepta cada cadena que el original acepta y ni una más.

PRIMER CASO: Es evidente que todo AFD A es un AFN N en el que no hay transiciones múltiples para un mismo estado y símbolo, es decir, cada transición $\delta(q, a) = p$ se convierte en $\delta(q, a) = \{p\}$ en el nuevo AFN, pues cada estado q en el AFD se convierte en el conjunto $\{q\}$ para el AFN N . Las transiciones son las mismas y los estados finales conservan su condición. Por ello, el lenguaje $L(A) = L(A_N)$.

SEGUNDO CASO: La construcción de un AFD a partir de un AFD requiere proceder con más cuidado: Sea $N = (Q_N, \Sigma, \delta_N, q_N, F_N)$ un AFN. El nuevo AFD $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$ se construye de la siguiente manera.

- El alfabeto debe ser el mismo para ambos, pues se trata del mismo conjunto de cadenas.
- El estado inicial coincide, $q_D = \{q_N\}$, como es de esperarse.
- Q_D es el conjunto potencia de Q_N , $Q_D = 2^{Q_N}$. Es posible que no todos los subconjuntos se conviertan en estados alcanzables y serán eliminados del autómata final.
- Los estados finales del AFD F_D son todos aquellos estados de 2^{Q_N} que incluyen un estado final del AFN, es decir, $F_D = 2^{Q_N} \cap F_N$.
- Para cada subconjunto de estados $S \subset Q_N$ y símbolo de entrada $a \in \Sigma$,

$$\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a) \quad (1.21)$$

y esto representa la manera de construir (y seleccionar) los estados alcanzables. Con palabras: para cada subconjunto S de estados, se revisa cada estado $p \in S$ y a qué estados se llega con $\delta_N(p, a)$ y se calcula su unión. Obsérvese que dada una cadena w aceptada por N , será necesario probar que

$$\delta_D(q_D, w) =^? \delta_N(q_N, w) \quad (1.22)$$

donde ambas funciones de transición regresan un conjunto de estados, que debe incluir el estado final que acepta la cadena w . La diferencia es que δ_D interpreta cada subconjunto de estados de A_N como un sólo estado de A_D , mientras que δ_N sí los interpreta como un subconjunto sobre Q_N .

Ahora falta demostrar que el autómata D construido acepta el mismo lenguaje que N . Para ello, se tomará una cadena arbitraria w aceptada por N y se demostrará que también es aceptada por D . La cadena será aceptada cuando haga un recorrido en A por los estados equivalentes y que llegue a un estado final en A que incluya al correspondiente estado final de N . Se procederá por inducción sobre la longitud de w .

Caso base: $|w| = 0$, es decir, la cadena vacía. En este caso, $\delta_D(\{q_N\}, w) = \{q_D\}$ y $\delta_N(q_N, w) = \{q_N\}$. Si q_N es un estado final, tanto D como N aceptarán la cadena y la rechazarán en caso contrario, lo que prueba el caso base.

Hipótesis inductiva: Supongamos que cualquier cadena $w \in L(N)$ de longitud menor o igual a n , cumple con $\delta_D(q_D, w) = \delta_N(q_N, w)$. Equivalentemente, w es aceptada también por A , es decir, $w \in L(A)$.

Sea ahora $w \in L(N)$ con $|w| = n + 1$. La cadena w puede descomponerse como $w = xa$, con $a \in \Sigma$ el último símbolo de w . Por hipótesis de inducción,

$$\delta_D(q_D, x) = \delta_N(q_N, x)$$

son el mismo conjunto de estados. Sea $\delta_N(q_N, x) = \{p_1, p_2, \dots, p_k\}$ tal conjunto. Por una parte, la definición de δ_N nos dice que

$$\delta_N(q_N, w) = \bigcup_{i=1}^k \delta_N(p_i, a). \quad (1.23)$$

Además, de la construcción de subconjuntos (1.21) se tiene que

$$\delta_D(\{p_1, p_2, \dots, p_k\}, a) = \bigcup_{i=1}^k \delta_N(p_i, a). \quad (1.24)$$

y finalmente se puede calcular $\delta_D(\{q_D\}, w)$:

$$\begin{aligned} \delta_D(\{q_D\}, w) &= \delta_D(\delta_D(q_D, x), a) && \text{pues } w = x \cdot a \\ &= \delta_D(\delta_N(q_N, x), a) && \text{por hipótesis inductiva} \\ &= \delta_D(\{p_1, p_2, \dots, p_k\}, a) && \text{por definición} \\ &= \bigcup_{i=1}^k \delta_N(p_i, a) && \text{por la construcción} \\ &= \delta_N(q_N, w) && \smile \end{aligned}$$

La cadena w será aceptada si $\delta_D(\{q_D\}, w)$ o $\delta_N(q_N, w)$ poseen al menos un estado de F_D y F_N respectivamente. $\square$

Reiteremos la importancia del anterior resultado: pese a que parece ganarse poder con el NO determinismo, un autómata no gana poder para aceptar lenguajes, en el sentido de que el conjunto de lenguajes que se pueden reconocer no es mayor que el que ya se reconocía con los AFD.

Ejemplo 20. Considere el AFN N representado en la figura 1.7 y a partir de allí se construirá un AFD equivalente. Los estados q_0 , q_1 y q_2 tienen transiciones no deterministas pues con algún símbolo caen en más de un estado.

El nuevo autómata se forma a partir de los estados y transiciones originales, agregando algunos más de la siguiente manera. De cada transición no determinista surgen los nuevos estados, por ejemplo a partir de $\delta(q_0, 0) = \{q_0, q_1\}$ se define el un estado nuevo con etiqueta $\{q_0, q_1\}$, que se agrega en una nueva fila de la tabla de transiciones. En la tabla 1.7b ya se ha completado este proceso, agregando 4 estados más.

Luego, de cada nuevo estado, por decir, $\{q_0, q_1\}$, se calcula qué estados es posible alcanzar estando en q_0 y q_1 con el símbolo 0. Del primer renglón de la tabla se ve que de nuevo se llega a $\{q_0, q_1\}$, pues q_1 no tiene transición con 0. Con el símbolo 1, de q_0 se llega a q_2 mientras que de q_1 se llega a $\{q_1, q_3\}$, por lo que se completa el quinto renglón de la tabla:

$$\begin{aligned}\delta(\{q_0, q_1\}, 1) &= \delta(\{q_0\}, 1) \cup \delta(\{q_1\}, 1) \\ &= \{q_2\} \cup \{q_1, q_3\} \\ &= \{q_1, q_2, q_3\}\end{aligned}$$

Es importante notar que durante la creación del nuevo autómata, algunos de los estados originales no se requieren más. El mismo estado final q_3 ya no se necesita, al igual que el estado q_1 pues partiendo del estado inicial ya no es alcanzado por ninguna transición.

Los nuevos estados que contienen alguno de los estados finales del autómata original se convierten en estados finales, en este caso, cualquiera que contenga a q_3 . El AFD final, equivalente al de la figura 1.7a se muestra en la figura 1.8.

1.2.9. Autómata finito con transiciones ϵ

Otra forma de darle versatilidad a un AFD es permitir que pueda pasar de un estado a otro sin procesar símbolos en la cadena de entrada. Esto se logra modificando de nuevo la función de transición δ para que permita transiciones con la cadena vacía, como $\delta(q, \epsilon) = p$. Este tipo de transiciones permite describir autómatas de manera sencilla.

Ejemplo 21. Un sencillo AFD- ϵ se muestra en la figura 1.9. Este autómata acepta cadenas de ceros y unos que comienzan siempre con un único 1, pueden terminar con cero y tienen posiblemente un segundo 1 en última o penúltima posición.

Ejemplo 22. Considérese en autómata que acepta números reales como 123.32, -0.098 o $+15.$, todos válidos como constantes numéricas en el código de un lenguaje de programación. El alfabeto es $\Sigma = \{0, 1, 2, \dots, 9, ., +, -\}$. Sea D el conjunto de los dígitos, como en el ejemplo 18. El diagrama de transiciones de este autómata se muestra en la figura 1.10.

En los autómatas con transiciones ϵ o AFN- ϵ , la definición formal es la casi la misma que en los AFD, excepto que la función de transición es $\delta : Q \times \{\Sigma \cup \{\epsilon\}\} \rightarrow Q$, que permite pasar de un estado a otro con la cadena vacía, es decir, sin procesar ningún símbolo de la cinta de entrada.

Para formalizar la definición de los AFN- ϵ , particularmente para definir el lenguaje que aceptan, es necesario extender la función de transición, no solo definir de nuevo su dominio. Para esto, primero se define inductivamente la cerradura de un estado q , escrita como $\text{Clos}\{q\}$.

Caso base $q \in \text{Clos}\{q\}$

Inducción $p \in \text{Clos}\{q\} \Rightarrow \delta(q, \epsilon) = p$

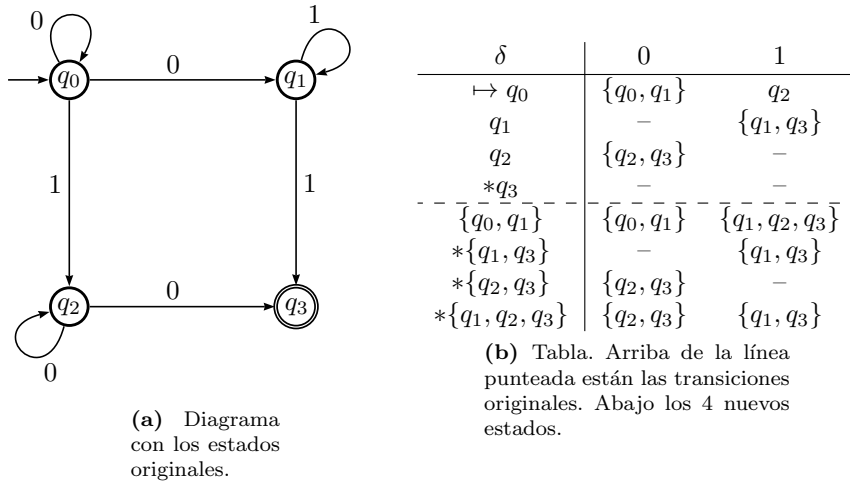


Figura 1.7: Autómata Finito No Determinístico. En la tabla de transiciones, debajo de la línea punteada, aparecen los nuevos estados que se requieren.

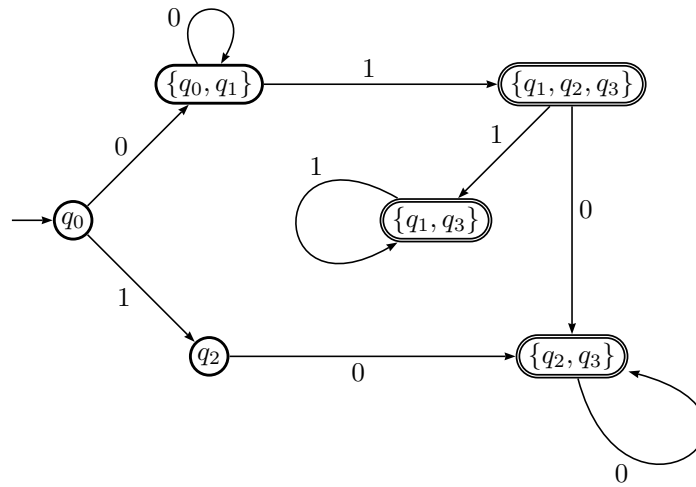


Figura 1.8: Autómata Finito Determinístico equivalente al de la figura 1.7a. Su tabla de transiciones se obtiene de la tabla 1.7b luego de eliminar los estados inaccesibles: q_1 y q_3 .

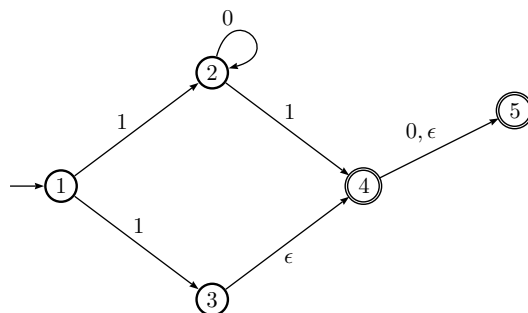


Figura 1.9: Autómata Finito No determinístico con transiciones ϵ . Acepta cadenas de ceros y unos que comienzan siempre con un único 1, pueden terminar con cero y tienen posiblemente un segundo 1 en última o penúltima posición.

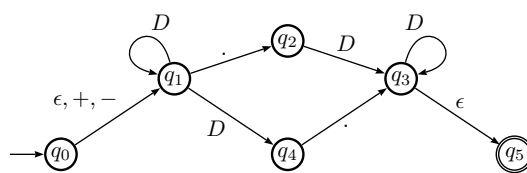


Figura 1.10: Autómata finito determinístico con transiciones ϵ que acepta cadenas que representan constantes reales.

De esta manera, la cerradura⁴ de un estado q es el conjunto de estados a los que se puede llegar desde q mediante transiciones épsilon.

La figura 1.10 tiene dos transiciones épsilon, una a partir de q_0 y otra de q_3 . Por lo tanto, la cerradura de estos estados es $\text{Clos}\{q_0\} = \{q_0, q_1\}$ y $\text{Clos}\{q_3\} = \{q_3, q_5\}$, para el resto de los estados, su cerradura tiene sólo el mismo estado. Si las transiciones épsilon son muchas en un autómata, entonces las cerraduras serán mayores en general.

¿Qué significa que un AFN- ϵ acepte su entrada? Primero es necesario extender la función de transición a $\hat{\delta}$, modificando δ para que procese cadenas y no sólo símbolos. Ya se han realizado dos extensiones similares, una para los AFD y otra para los AFN, en este caso la extensión debe tomar en cuenta las transiciones ϵ sin que estas contribuyan a la cadena w que se procesa.

El caso base de la definición es $\hat{\delta}(q, \epsilon) = \text{Clos}(q)$, con lo que sólo se avanza por las transiciones etiquetadas con ϵ .

Ahora, para el paso inductivo, sea $w \in \Sigma^*$ una cadena tal que $w = xa$, con $x \in \Sigma^*$ y a el último símbolo de w . Nótese que $a \neq \epsilon$ pues $\epsilon \notin \Sigma$. Entonces, $\hat{\delta}$ se calcula de la siguiente manera.

1. Sea $\hat{\delta}(q, x) = \{p_1, p_2, \dots, p_k\}$ el conjunto de estados que se alcanzan desde q siguiendo la cadena etiquetada con x . Este camino puede tener varias transiciones ϵ .
2. Sea $\bigcup_{i=1}^k \delta(p_i, a) = \{r_1, r_2, \dots, r_m\}$, es decir, se siguen todas las posibles transiciones con el símbolo a desde los estados en $\hat{\delta}(q, x)$. Esto significa que cada r_j es un estado al que se puede llegar mediante un camino etiquetado con x y luego con a .
3. Por último, se siguen las transiciones ϵ a partir de cada r_j , con

$$\hat{\delta}(q, w) = \bigcup_{j=1}^m \text{Clos}(r_j).$$

Ahora ya se puede definir el lenguaje aceptado por un AFN- ϵ , como

$$L(A) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\} \quad (1.25)$$

Aunque en apariencia, los lenguajes definidos en (1.25) y (1.20) son el mismo, no deben confundirse, pues cada uno utiliza una $\hat{\delta}$ distinta.

Equivalencia entre AFD y AFN- ϵ

Antes de demostrar que los AFD son equivalentes a los AFN- ϵ , estableceremos la manera de eliminar las transiciones- ϵ y luego la equivalencia será más sencilla de probar. Se asemeja

Sea $E = (Q_E, \Sigma, \delta_E, q_E, F_E)$ un AFN- ϵ y construiremos un AFD $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$ empleando la cerradura de los estados para eliminar las transiciones. Como antes, se trata del mismo alfabeto Σ . La construcción es:

⁴Este es un caso particular de la *cerradura transitiva*, definida para relaciones binarias.

1. $Q_D = 2^{Q_E}$, el conjunto de todos los subconjuntos de Q_E . Es posible que no todos sean necesarios, sólo aquellos que están en la cerradura de otros estados.
2. $q_D = \text{Clos}\{q_E\}$, el subconjunto de todos los estados a los que se puede llegar desde q_E mediante transiciones ϵ .
3. F_D es el subconjunto de estados en los que al menos uno es un estado final de E , es decir:

$$F_D = \{S \in 2^{Q_E} \text{ y } S \cap F_E \neq \emptyset\}$$

4. La función de transición se calcula de la siguiente forma. Sea $a \in \Sigma$ y $S = \{p_1, p_2, \dots, p_k\}$ un subconjunto de Q_D .

Sea $\{r_1, r_2, \dots, r_m\} = \bigcup_{i=1}^k \delta(p_i, a)$, todos los estados a los que se puede llegar partiendo los p_i con el símbolo a . La función de transición puede ahora definirse como

$$\delta_D(S, a) = \bigcup_{j=1}^m \text{Clos}(r_j)$$

Con esta construcción podemos establecer la equivalencia entre los AFD y los AFN- ϵ . Como antes, en el caso de los AFN, es necesario probar que dado un AFN- ϵ arbitrario, es posible construir un AFD equivalente y viceversa.

Proposición 5. *Un lenguaje L es aceptado por un AFD D si y sólo si es aceptado por algún AFN- ϵ E .*

Demostración. Sean dos autómatas, $E = (Q_E, \Sigma, \delta_E, q_E, F_E)$ un AFN- ϵ y un AFD $D = (Q_D, \Sigma, \delta_D, q_D, F_D)$. La demostración debe hacerse en dos partes, como en toda proposición si y sólo si. La primera parte, donde si un lenguaje es aceptado por un AFD entonces existe un AFN- ϵ es fácil.

$\Rightarrow$) A partir del AFD D es fácil construir un AFN- ϵ E que acepta el mismo lenguaje. El conjunto de estados es el mismo, el estado inicial sigue siendo igual así como los estados finales. La función de transición es casi la misma: cada transición $\delta_E(q, a) = \{\delta_D(q, a)\}$ pues δ_E tiene como dominio a 2^Q . Una cadena $w \in \Sigma$ hará el mismo recorrido por los estados de cada autómata, llegando a un estado de F_E si $w \in L(E)$, por lo que E acepta el mismo lenguaje que D .

$\Leftarrow$) En esta parte, dado el AFN- ϵ E hay que construir el AFD D que acepta el mismo lenguaje. Para esta parte aplicaremos la construcción de subconjuntos previa y se procederá por inducción en la longitud de la cadena w aceptada por E para probar que también lo es por D . Obtendremos que $L(D) = L(E)$ demostrando que las funciones de transición tienen el mismo comportamiento.

Caso base: Si $|w| = 0$ entonces $w = \epsilon$. Sabemos que $\delta(q_0, \epsilon) = \text{Clos}(q_E)$, $q_D = \text{Clos}(q_E)$ y que $\delta_D(p, \epsilon) = p$ para todo estado p , por lo que

$$\delta_E(q_E, \epsilon) = \delta(q_D, \epsilon).$$

Caso inductivo: Supongamos que la proposición se cumple para una cadena x , es decir, es cierto que $\delta_E(q_E, x) = \delta(q_D, x)$, siendo este el conjunto de estados $\{p_1, p_2, \dots, p_k\}$. Ahora hay que calcular $\delta_E(q_E, xa)$, con $a \in \Sigma$.

δ	0	1
$\mapsto 1$	—	$\{2, 35\}$
2	2	45
*35	5	—
*45	5	—
*5	—	—

Tabla 1.4: Tabla de transiciones de un AFN sin transiciones épsilon equivalente al de la figura 1.9. El estado 35 representa los estados 3, 4 y 5.

Sean $w = xa$,

$$\{r_1, r_2, \dots, r_m\} = \bigcup_{i=1}^k \delta(p_i, a) \quad (1.26)$$

y

$$\delta(q_0, w) = \bigcup_{j=1}^m \text{Clos}(r_j). \quad (1.27)$$

Examinando la manera de eliminar las transiciones ϵ con la construcción de subconjuntos, sabemos que la transición $\delta(\{p_1, p_2, \dots, p_k\}, a)$ se construye precisamente con las ecuaciones (1.26) y (1.27).

Por lo tanto, $\delta_D(q_D, w)$, que es $\delta_D(\{p_1, p_2, \dots, p_k\}, a)$ resulta ser el conjunto $\delta(q_E, w)$, lo que concluye la demostración. $\square$

Como ejemplos, tomaremos los autómatas de las figuras 1.9 y 1.10 para eliminar las transiciones épsilon.

Ejemplo 23. En el caso del autómata de la figura 1.9, sólo dos estados tienen transiciones épsilon, de tal manera que las cerraduras quedan:

$$\begin{aligned} \text{Clos}(3) &= \{3, 4, 5\} \\ \text{Clos}(4) &= \{4, 5\}. \end{aligned}$$

Si estamos en el estado 1 y se recibe un símbolo 1, se puede pasar al estado 2 o a la cerradura del estado 3, por lo que

$$\delta(1, 1) = \{2, \text{Clos}(3)\}$$

y de la misma manera se tiene que $\delta(2, 1) = \{4, 5\}$, al procesar el símbolo 1 estando en el estado 2. De esta manera, la tabla de transiciones resultante se muestra en la tabla 1.4.

Ejemplo 24. Considérese de nuevo el autómata del ejemplo 22 que acepta números reales. El diagrama de transiciones de este autómata se muestra en la figura 1.10. Es necesario observar que las cerraduras importantes son:

$$\begin{aligned}\text{Clos}(q_0) &= \{q_0, q_1\} \quad (\text{el nuevo estado inicial}) \\ \text{Clos}(q_3) &= \{q_3, q_5\}\end{aligned}$$

Por facilidad las vamos a referir como $q_{0,1}$ y $q_{3,5}$. Obsérvese la figura 1.10. Estando en el estado inicial, con un punto se puede llegar únicamente al estado q_2 , con un signo al estado q_1 y con un dígito (D) no hay transición. Pero en el nuevo estado inicial $q_{0,1}$ se deben revisar las transiciones posibles para cada estado de la cerradura. Así, con un dígito ahora puede quedarse en q_1 (donde llegó por la transición ϵ) o avanzarse a q_4 , por lo que se obtiene

$$\delta(q_{0,1}, D) = q_{1,4}$$

y $q_{1,4}$ es un nuevo estado que se agrega al autómata que se está construyendo. De la misma manera se obtienen

$$\begin{aligned}\delta(q_2, D) &= q_{3,5} \\ \delta(q_{1,4}, \cdot) &= q_{2,3,5} \\ \delta(q_{1,4}, D) &= q_{1,4} \\ \delta(q_{2,3,5}, D) &= q_{3,5}.\end{aligned}$$

Los nuevos estados finales son los que incluyen al estado final original. Al eliminar las transiciones ϵ del autómata aplicando la cerradura con respecto a ϵ se obtiene el diagrama de transiciones de la figura 1.11.

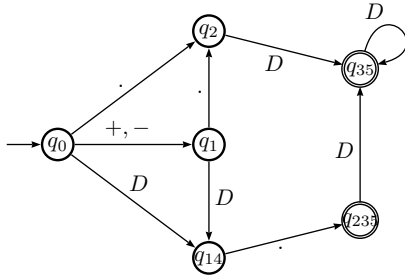
Las transiciones épsilon no agregan poder a los autómatas, pero son útiles para facilitar el diseño, tal como ocurre con el no determinismo.

1.3. Expresiones Regulares

Así como un lenguaje L se ha definido especificando la máquina (AFD) que acepta sólo cadenas $w \in L$, también es posible definirlo especificando una regla (o reglas) que indiquen cómo se pueden formar las cadenas de L .

Al ser un lenguaje un conjunto formado por cadenas, es válido realizar operaciones entre conjuntos o lenguajes, incluyendo operaciones especiales para conjuntos de cadenas. En particular, interesan 3 operaciones, que ya habían sido presentadas en la sección 1.1.3 (pág. 14). Sean L , L_1 y L_2 lenguajes regulares.

1. La *unión* de 2 lenguajes L_1 y L_2 es al conjunto formado por las cadenas que están en alguno de los dos lenguajes, es decir, $L_1 \cup L_2 = \{w | w \in L_1 \text{ o } w \in L_2\}$.
2. La *concatenación* de lenguajes se denota por L_1L_2 y se define como el conjunto de cadenas que se forman concatenando cualquier cadena de L_1 con otra cualquier cadena de L_2 , es decir, $L_1L_2 = \{w = w_1w_2 | w_1 \in L_1 \text{ y } w_2 \in L_2\}$.



(a) Autómata finito no determinístico SIN transiciones ϵ que acepta constantes reales.

δ	.	S	D
$\mapsto q_{0,1}$	2	1	$q_{1,4}$
q_1	2	—	—
q_2	—	—	$q_{3,5}$
$q_{1,4}$	$q_{2,3,5}$	—	$q_{1,4}$
$*q_{3,5}$	—	—	$q_{3,5}$
$*q_{2,3,5}$	—	—	$q_{3,5}$

(b) Arriba de la línea punteada están las transiciones originales. Abajo los 4 nuevos estados alcanzados por las cerradoras.

Figura 1.11: AFD equivalente al de la figura 1.10. En la tabla de transiciones, debajo de la línea punteada, aparecen los nuevos estados, que son las cerraduras correspondientes. Las cerraduras se han representado como subíndices.

3. La *cerradura* de un lenguaje, denotada por L^* , se forma seleccionando de manera arbitraria un subconjunto de las cadenas de L y concatenándolas todas, posiblemente con repeticiones, es decir, $L^* = \bigcup_{i \geq 0} L^i$, donde $L^0 = \{\epsilon\}$, $L^1 = L$ y $L^i = LLL \cdots L$ (la concatenación de i veces L), para $i > 1$.

Ejemplo 25. Si $L_1 = \{a, ab, aab\}$ y $L_2 = \{bb, bba\}$, entonces

$$L_1 \cup L_2 = \{a, ab, aab, bb, bba\} \quad (1.28)$$

$$L_1 L_2 = \{abb, abba, abbb, abbba, aabbb, aabbba\} \quad (1.29)$$

$$L_2 L_1 = \{bba, bbab, bbaab, bbaa, bbaab, bbaaab\} \quad (1.30)$$

$$L_1^* = \{w\alpha \mid w \in L_1 \text{ y } \alpha \in L_1^*\} \quad (1.31)$$

En el caso de (1.31), se trata de un conjunto infinito pues es una concatenación arbitraria, es decir, la cantidad que sea de cadenas tomadas de L_1 .

Siguiendo con la idea de los lenguajes como conjuntos de cadenas, se acostumbra representar la unión de dos lenguajes L_1 y L_2 como $L_1 \cup L_2 = L_1 + L_2$.

Un *lenguaje regular*, definido en (1.17), es el que es aceptado por un AFD. Siempre es posible encontrar una manera de describir algebraicamente el mismo lenguaje, con lo que se conoce como *expresiones regulares*, o ER, utilizando ciertas reglas algebraicas bien definidas. La definición es de tipo inductivo, por lo que se requieren casos base (tres) y luego los casos inductivos (cuatro), donde se ve la manera de combinar ER. Sea Σ un alfabeto.

1. **(Caso base 0)** $\emptyset$ es una expresión regular.

$$L(\emptyset) = \emptyset \quad (1.32)$$

2. **(Caso base 1)** La cadena vacía, ϵ , es una expresión regular.

$$L(\epsilon) = \{\epsilon\} \quad (1.33)$$

3. **(Caso base 2)** Cada símbolo de Σ es una expresión regular.

$$L(a) = \{a\} \text{ con } a \in \Sigma \quad (1.34)$$

4. Si E es una expresión regular, entonces (E) también lo es.

$$L((E)) = L(E) \quad (1.35)$$

5. Si E es una expresión regular, su repetición arbitraria E^* también lo es.

$$L(E^*) = (L(E))^* \quad (1.36)$$

6. Si E_1 y E_2 son expresiones regulares, entonces su concatenación E_1E_2 también lo es.

$$L(E_1E_2) = L(E_1)L(E_2) \quad (1.37)$$

7. Si E_1 y E_2 son expresiones regulares, entonces $E_1 + E_2$ también lo es.

$$L(E_1 + E_2) = L(E_1) \cup L(E_2) \quad (1.38)$$

El operador $+$, que también se utiliza para la unión de lenguajes, tiene la precedencia más baja. Se utiliza para indicar que se toma una de las dos expresiones, es decir, la expresión regular $01 + 11$ produce dos cadenas, ya sea la 01 o bien la 11 , pero no la cadena 0111 , que es el resultado de la concatenación. El operador $+$ es conmutativo.

El operador $*$ tiene la máxima precedencia y afecta a la expresión regular inmediata que le antecede, por lo que ab^* produce las cadenas a , ab , abb , $abbb$ y no $abab$, $ababab$ que se generan con la expresión $(ab)^*$.

El lenguaje producido por la expresión regular E se denota por $L(E)$. Por economía, muchas veces se escribe sólo la expresión regular para dar a entender el lenguaje que genera. Se dice que $L(E) \in ER$, es decir, el lenguaje generado por E pertenece al conjunto de los lenguajes generados por expresiones regulares.

Ejemplo 26. ¿Cuál es el lenguaje generado por la expresión $a(b + c)$? Apliquemos las reglas de la definición.

$$\begin{aligned} L(a(b + c)) &= L(a)L(b + c) \\ &= L\{a\} (L(b) \cup L(c)) \\ &= \{a\} (\{b\} \cup \{c\}) \\ &= \{a\} (\{b, c\}) \\ &= \{a\}\{b, c\} \\ &= \{ab, ac\} \end{aligned}$$

Ejemplo 27. ¿Cuál es el lenguaje generado por la expresión $(10)^*00$? Hay que generar las cadenas de $(01)^*$ y luego concatenarles 00:

$$\begin{aligned} L((10)^*00) &= L((10)^*)L(00) \\ &= \{\epsilon, 10, 1010, 101010, \dots\} \cup \{00\} \\ &= \{00, 1000, 101000, 10101000, \dots\} \end{aligned}$$

Aunque en el ejemplo anterior se hizo de manera detallada, lo común es irse de manera directa a las cadenas. A fin de cuentas, $(10)^*00$ son las repeticiones arbitrarias de la cadena 01 concatenadas con 00, pero el detalle ayuda a entender.

Ejemplo 28. ¿Cuál es el lenguaje generado por la expresión $(0+1)^*$?

$$\begin{aligned} L((0+1)^*) &= (L(0+1))^* \\ &= \{0, 1\}^* \\ &= \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\} \end{aligned}$$

Se trata de todas las cadenas que se pueden formar con ceros y unos.

Ejemplo 29. La expresión regular $\emptyset^*$ genera el lenguaje que sólo contiene a la cadena vacía. Esto es así pues

$$\emptyset^* = \epsilon \cup \emptyset \cup \emptyset\emptyset \cup \emptyset\emptyset\emptyset \dots$$

y la única cadena que se genera es la vacía.

La expresión regular $\emptyset$ funciona como neutro ante la unión y cero ante concatenación pues si E es una expresión regular, entonces

$$E + \emptyset = E \quad (1.39)$$

$$E\emptyset = \emptyset \quad (1.40)$$

son operaciones que se consideran propiedades y deben recordarse.

Ejemplo 30. ¿Cuál es el lenguaje generado por la expresión $(0^*1^*)^*$? Primero apliquemos las reglas de cerradura y concatenación.

$$\begin{aligned} L((0^*1^*)^*) &= (L(0^*1^*))^* \\ &= (L(0^*)L(1^*))^* \\ &= (\{\epsilon, 0, 00, \dots\}\{\epsilon, 1, 11, \dots\})^* \end{aligned}$$

Hay muchas maneras de enlistar la concatenación de $L(0^*)L(1^*)$, pero hay que notar que dos cadenas que están ahí son $\epsilon 1 = 1$ y $0\epsilon = 0$, por lo que la cerradura de la concatenación resulta equivalente a la expresión del ejemplo 28, $(0+1)^*$.

Ejemplo 31. La expresión $b(ac)^*d$ genera cadenas que siempre comienzan con b, seguidas de una cantidad arbitraria de parejas ac y terminando con una letra d, como bd, bacd, bacacacd y así sucesivamente.

Ejemplo 32. Se pueden describir las cadenas generadas con la expresión regular $(1+0)10^*$, indicando que se producen cadenas que inicien con un cero o un uno, seguido siempre de un 1 y terminando con una cantidad arbitraria de ceros, como 11000, 0100, 01, 11 o 11000000.

Ejemplo 33. El lenguaje $L((01)^* + (123+0)^* + 1(23)^*)$ equivale a la unión de los lenguajes

$$(01)^* \cup (123+0)^* \cup 1(23)^*$$

que a su vez podría separarse aún más.

Ejemplo 34. En el cuadro 1.1 se mostró un AFD que acepta las cadenas que representan potencias de 2 en binario. Una expresión regular equivalente es 10^* .

Ejemplo 35. Para el lenguaje L_6 , del ejemplo 10, las cadenas de múltiplos de 5 desde 100 a 195 pueden generarse con la expresión regular $1(0+1+2+3+\dots+9)(0+5)$.

Ejemplo 36. El lenguaje $L(1^* + 2^*)$ se forma con todas las cadenas que sólo tienen unos o sólo tienen ceros. La cadena vacía pertenece a este lenguaje.

Ejemplo 37. Otro ejemplo clásico es el lenguaje de las cadenas que alternan unos y ceros, como 010, 010101, 1, 1010 y todas en las que no hay repeticiones sucesivas. Una expresión regular que las genera parece ser algo como

$$(01)^*$$

pero esta sólo incluye las que comienzan con 1, terminan con cero y tienen longitud par. Para completar el lenguaje, se requiere agregar las simétricas $(10)^*$ y las de longitud impar. La expresión regular resultante es

$$(01)^* + (10)^* + 0(10)^* + 0(01)^*$$

que puede simplificarse usando la cadena vacía como:

$$(1+\epsilon)(01)^*(0+\epsilon)$$

de donde se ve que las expresiones regulares no son únicas para cada lenguaje.

Muchas expresiones regulares son equivalentes, en es sentido que definen el mismo lenguaje. Por ejemplo, como $(0+1)^*$ son todas las posibles cadenas de ceros y unos, el lenguaje $(0+1)^* + 1^*$ no aporta nada pues $1^* \subset (0+1)^*$, por lo que son iguales. Debe ser claro que $1+11+111+\dots=11^*$. Existe toda un algebra de expresiones regulares.

1.3.1. Conversión de expresiones regulares a AFD

Aunque es posible construir un autómata que acepte el lenguaje generado por una ER, no se requiere gran pericia o astucia. Es posible hacerlo a ojo, pero

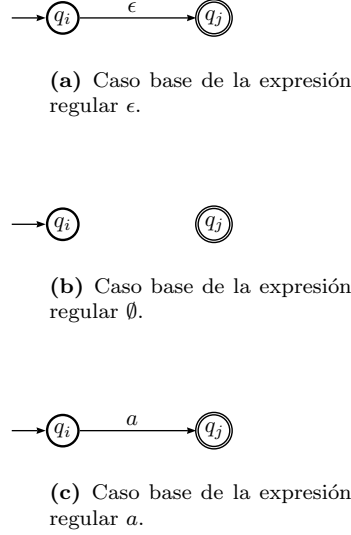


Figura 1.12: Casos base de la conversión de ER a AFN- ϵ .

existe un algoritmo seguro para hacer la conversión cualquiera que sea la ER. La idea es construirla por bloques, de manera inductiva.

Hay tres casos base, que consisten en las expresiones regulares ϵ , $\emptyset$ y a , con $a \in \Sigma$ un símbolo del alfabeto Σ . Los autómatas correspondientes a estos casos base se muestran en la figura 1.12.

A partir de estos simples autómatas como casos base, es posible construir autómatas más complejos, mediante construcciones inductivas. Sean E y F expresiones regulares. Para cada uno de los operaciones de suma, concatenación y cerradura se tiene un paso inductivo, esquematizados en la figura 1.13.

Adición de expresiones regulares. Para la suma de expresiones regulares $E + F$, en la que se aplica la suma de lenguajes, definida en la página 38 (punto 7), se construye un nuevo autómata a partir de los autómatas de E y F . Un nuevo primer estado es conectado con transiciones ϵ a los anteriores estados iniciales de los autómatas de E y F . De los estados finales de E y F se colocan transiciones ϵ al nuevo único estado final. Ver figura 1.13a.

$$L(E + F) = L(E) + L(F) \quad (1.41)$$

Concatenación de expresiones regulares. Se aplica la concatenación de lenguajes, definida en la página 38 (punto 6). Para la concatenación de las expresiones E y F , como EF , el estado final de E se conecta con el estado inicial de F y ambos dejan de ser estados especiales. Ver figura 1.13b.

$$L(EF) = L(E)L(F) \quad (1.42)$$

Cerradura de una expresión regular. Se aplica la cerradura de lenguajes, definida en la página 38 (punto 5). La cerradura E se construye haciendo un nuevo estado inicial y otro nuevo estado final. El estado inicial se conecta con el anterior estado inicial de E y con el estado final. El estado final de E se conecta con su estado inicial original. Ver figura 1.13c.

$$L(E^*) = (L(E))^* \quad (1.43)$$

Ejemplo 38. Sea la expresión regular $1 + 10$, que genera un lenguaje con dos cadenas: $L(1 + 10) = \{1, 10\}$. La expresión emplea los símbolos 1 y 0, cuyos autómatas corresponden al caso base 1.12c. Además de ese caso base, están dos operaciones: primero la concatenación de 1 con 0 y luego la suma de 1 y 10, que se realiza como dicen los casos inductivos. Se ilustra en la figura 1.14.

1.3.2. Conversión de AFD a expresiones regulares

Se ha mencionado que AFDs y Expresiones Regulares son dos formalismos equivalentes para definir Lenguajes Regulares. Esta proposición es suficientemente fuerte como para requerir una prueba mas detallada. Para mostrar la equivalencia, son necesarias dos cosas:

- Partir de un AFD arbitrario y construir una expresión regular que genere el mismo lenguaje que el AFD acepta.
- Dada una expresión regular E cualquiera, construir un AFD que solo acepte $L(E)$.

La gran ventaja de la Teoría de la Computación es que ofrece pruebas constructivas, que nos dan algoritmos para resolver problemas prácticos. La equivalencia entre AFD y expresiones regulares es también constructiva.

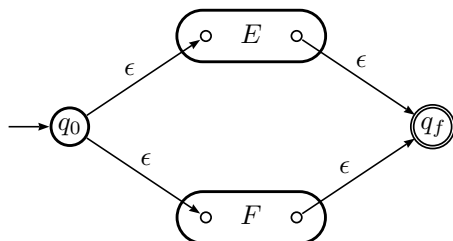
Sea $A = (Q, \Sigma, \delta, q_0, F)$ un AFD, con n estados. A partir de A se construirá una expresión regular R tal que $L(A) = L(R)$. Se entenderá por $R_{ij}^{(k)}$, la expresión regular que etiqueta el camino entre los estados q_i y q_j y que pasa por estados cuyo índice no es mayor que k . Se construirá inductivamente aumentando k de uno en uno.

Caso base: No hay estados intermediarios, $k = 0$. En ese caso, $R_{ij}^{(0)}$ solo son las expresiones de las rutas directas del nodo q_i al nodo q_j . Hay dos posibles casos:

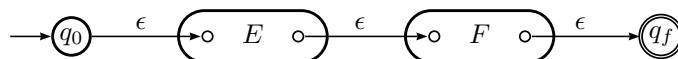
1. Un arco del estado i al estado j , con $i \neq j$.
2. Un camino de longitud 0, que consiste sólo del estado q_i .

Si $i \neq j$ entonces sólo el caso 1 es posible. Debe revisarse entonces el autómata A para buscar transiciones de q_i a q_j con algún símbolo a y hay tres posibilidades, que nos dan las primeras 3 reglas de construcción:

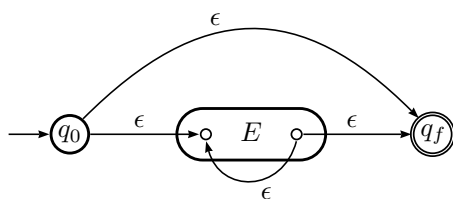
- (a) Si no hay transición posible, entonces $R_{ij}^{(0)} = \emptyset$.
- (b) Si hay tal transición, entonces $R_{ij}^{(0)} = a$.



(a) La suma $E + F$. Las transiciones ϵ conectan a los autómatas de E y F .



(b) La concatenación EF .



(c) La cerradura E^* .

Figura 1.13: Pasos inductivos para la construcción modular de autómatas a partir de expresiones regulares.

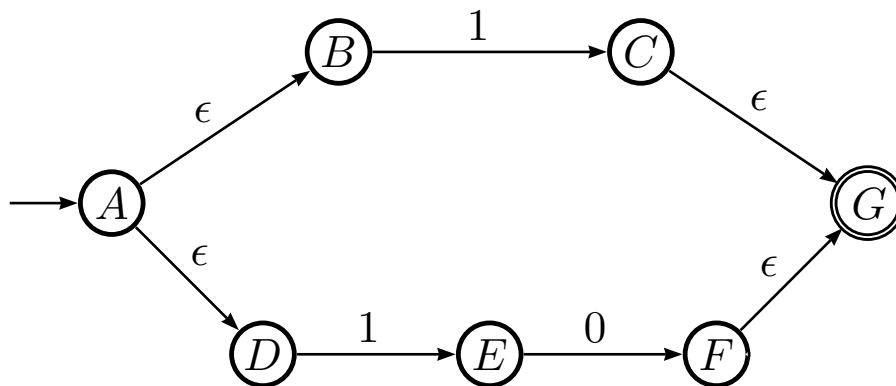


Figura 1.14: Conversión de $1 + 10$ a AFN.

- (c) Si hay varias transiciones, con símbolos $a_1, a_2, \dots, a_r$ entonces $R_{ij}^{(0)} = a_1 + a_2 + \dots + a_r$.

En caso del camino de longitud cero y cuando $i = j$, entonces se trata de todas las transiciones de i sobre sí mismo. En este caso, el camino de longitud cero se representa por la expresión ϵ , la cadena vacía. Por eso, es necesario agregar ϵ a todas las expresiones que ya se tienen del caso base y construir ϵ , $a + \epsilon$ y $\epsilon + a_1 + a_2 + \dots + a_r$.

Caso inductivo: Sea $R_{ij}^{(k)}$ la expresión que etiqueta las rutas del nodo i al nodo j y que no pasan por un nodo mayor que k . Hay dos casos que deben considerarse:

1. El camino no pasa por el estado q_k , así que la etiqueta está en el lenguaje de $R_{ij}^{(k-1)}$ por lo que ya se tiene.
2. El camino pasa por el estado k al menos una vez. Esta cadena se subdivide en 3 partes, primero del nodo i al k , luego los posibles caminos de k a k y por último de k a j .

Considerando los dos casos anteriores se obtiene la relación con la que se genera la expresión que se necesita (la cuarta regla de construcción):

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} + R_{ik}^{(k-1)} (R_{kk}^{(k-1)})^* R_{kj}^{(k-1)} \quad (1.44)$$

para la etiqueta de los caminos que van del estado i al j sin pasar por estados con índice mayor a k .

Con las 4 reglas anteriores se pueden comenzar a construir expresiones partiendo de los índices más pequeños, de tal forma que cada una dependa sólo de expresiones con índices menores y que estén ya disponibles.

Finalmente, la expresión regular que se desea, equivalente al AFD A se forma con la unión de los lenguajes $R_{1j}^{(n)}$ para cada estado final $q_j \in F$.

El problema con este algoritmo es que genera cadenas (expresiones regulares) indeseablemente largas, por lo que es propenso a errores.

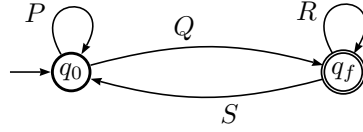


Figura 1.15: El caso al que se desea llegar en la reducción de estados. La expresión regular es $P^*QR^*(SP^*QR^*)^*$.

Eliminación de estados

Otra técnica para obtener la expresión regular a partir de un autómata es conocida como *eliminación de estados*.

En este caso, cada que se elimina un estado s es necesario revisar todas las rutas que pasan por s , de tal manera que si hay una ruta del estado q al p pasando por s , entonces se sustituye el estado s por una transición etiquetada por las expresiones regulares equivalentes a todas las rutas involucradas.

Al final de la eliminación de estados se espera tener el estado inicial y un único estado final, por lo que el caso más complicado será el de la figura 1.15. Se entiende que las etiquetas de las transiciones son expresiones regulares construidas a partir de la eliminación de estados. La expresión final es

$$P^*QR^*(SP^*QR^*)^*$$

Un autómata con tres estados se presenta en la figura 1.16, donde el estado s se elimina.

Obsérvese que es posible que los estados p y q (en la misma figura 1.16) sean el mismo estado. Hay dos cosas que considerar. Primeramente, no hay un orden determinista en cuanto a qué estados deben eliminarse primero. Por otra parte, puede ser complicado visualizar todas las rutas que pasan por un estado que se desea eliminar. Los detalles de la conversión se muestran en los siguientes ejemplos, que ilustran los casos posibles. Refiérase el lector a la figura 1.16 cada que necesite.

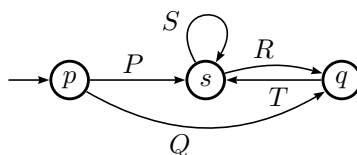
Ejemplo 39. La figura 1.17 muestra un autómata y la secuencia de sus transformaciones. En la figura 1.17b se ha eliminado el estado q_2 , lo que requiere poner la ruta $q_3 - q_2 - q_1$ (símbolos 01) en una transición de q_3 a q_1 .

El segundo estado que se elimina es q_1 , el único posible pues el estado inicial y el final son intocables. Al eliminar este estado, las rutas $q_0 - q_1 - q_3$ (10^*1) y $q_3 - q_1 - q_3$ (010^*1) deben agregarse a las transiciones $q_0 - q_1$ y $q_3 - q_3$ respectivamente.

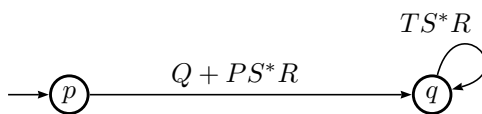
Del autómata final se lee la expresión regular

$$(0 + 10^*1) (010^*1)^* (1(0 + 10^*1)(010^*1))^*$$

de las 3 transiciones resultantes.



(a) Autómata original.
El estado s va a desaparecer.



(b) Autómata ya transformado, luego de eliminar el estado s . La expresión entre paréntesis corresponde a la transición de q a s y el retorno a q .

Figura 1.16: Eliminación del estado s . Las etiquetas P , R , S y Q representan expresiones regulares. La expresión final es $(P + PS^*R(TS^*R)^*)$.

Ejemplo 40. En la figura 1.18 se tiene un autómata como el del ejemplo anterior, pero con otra configuración de aristas, que hacen más interesante la eliminación de estados. La figura 1.18a es el autómata original y 1.18a es el autómata cuando se elimina el estado q_1 .

Por q_1 pasan varias rutas:

1. $q_0 - q_1 - q_2$: 00 que se agrega a $q_0 - q_2$,
2. $q_0 - q_1 - q_3$: 01 que forma la transición $q_0 - q_3$ y
3. $q_2 - q_1 - q_2$: 10 que se agrega a $q_2 - q_2$.

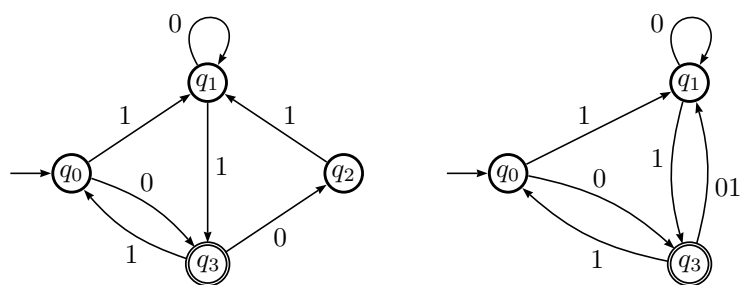
Eliminar q_2 requiere menos operaciones, agregando un ciclo $q_3 - q_3$. La expresión regular final es

$$(01 + (1 + 00)(0 + 10)^*(1 + 11))(01 + 00(0 + 10)^*(1 + 11))^*$$

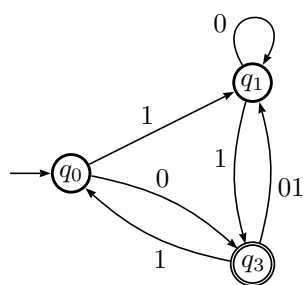
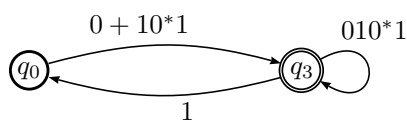
que se obtiene directamente de la figura 1.18c.

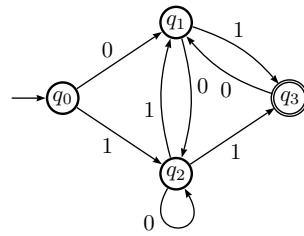
Ejemplo 41. La figura 1.19 muestra un autómata y la secuencia paso a paso de su transformación a expresión regular. En la figura 1.19a está el autómata original. Como se busca terminar con un autómata con transición a un único estado final, se agrega tal estado en 1.19b y q_3 y q_4 dejan de ser estados finales.

Siendo el orden de eliminación arbitrario, en la figura 1.19c se ha eliminado el estado q_3 . Las rutas afectadas son del estado q_1 a q_5 y de q_1 a q_2 , que están en las nuevas transiciones dibujadas. Al eliminar q_4 el proceso se repite, lo que se muestra en la figura 1.19d.

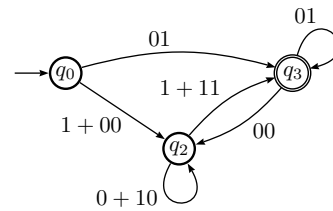


(a) Autómata original.

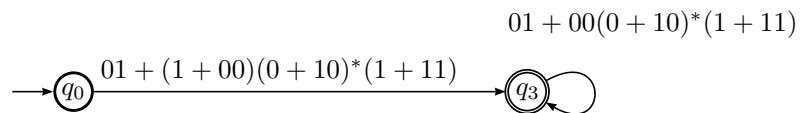
(b) Se elimina el estado q_2 . Aparece ruta de q_3 a q_1 .(c) Se elimina q_1 .**Figura 1.17:** Transformación de un autómata a expresión regular por eliminación de estados. Ver texto para más detalles.



(a) Autómata original.



(b) Se elimina el estado q_1 . Aparece ruta de q_0 a q_3 y todas las otras son modificadas.



(c) Se elimina q_2 .

Figura 1.18: Transformación de un autómata a expresión regular por eliminación de estados. Ver texto para más detalles.

En la figura 1.19e, la eliminación de q_1 modifica las rutas $q_0 - q_1 - q_2$ (10^*10), $q_0 - q_1 - q_5$ (10^*1), $q_2 - q_1 - q_2$ ($1 + 010^*10$) y $q_2 - q_1 - q_5$ ($0 + 010^*1$), por lo que estas nuevas transiciones deben suplir tales caminos.

Al final, la eliminación de q_2 ya no es problema. En la figura 1.19f se ha sustituido R por la subexpresión 10^*1 para ahorrar espacio.

Cuando estos ejemplos se realizan con JFLAP hay dos cosas que tomar en cuenta. Primero, JFLAP guiara al usuario paso a paso en la transformación, impidiendo operaciones que no se necesitan. No hay problema si se hace algo y no se puede deshacer, siempre es posible cerrar la ventana e iniciar desde cero. Por otra parte, para aplicar el algoritmo de reducción, JFLAP requiere que haya transiciones entre cada pareja de estados, por lo que se agregarán transiciones etiquetadas con $\emptyset$ donde falte. Esto crea algo de confusión visual, pero siempre es posible mover los estados para leer adecuadamente.

1.3.3. Simplificación de expresiones regulares

El algoritmo para formar expresiones regulares a partir de un AFD genera expresiones muy largas y es conveniente simplificar durante el proceso, ya sea que se haga a mano o se programe por computadora. A continuación se muestran algunas reglas algebraicas de las expresiones regulares que son útiles para este propósito.

Sean R , E_1 , E_2 y E expresiones regulares sobre un alfabeto Σ y $a \in \Sigma$. Se cumplen las siguientes operaciones.

$$R + R = R \quad (1.45)$$

$$E_1 + E_2 = E_2 + E_1 \quad (1.46)$$

$$E + (E_1 + E_2) = (E + E_1) + E_2 \quad (1.47)$$

$$E(E_1E_2) = (EE_1)E_2 \quad (1.48)$$

$$R(E_1 + E_2) = RE_1 + RE_2 \quad (1.49)$$

$$a + R^*a = R^*a \quad (1.50)$$

$$(\epsilon + R)^* = R^* \quad (1.51)$$

$$(\epsilon + R)R^* = R^* \quad (1.52)$$

$$\epsilon + R^* = R^* \quad (1.53)$$

$$\emptyset^* = \epsilon \quad (1.54)$$

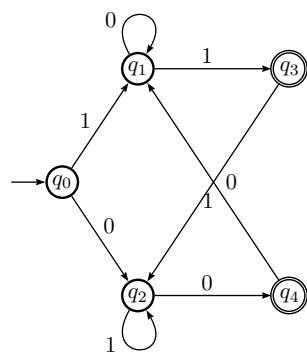
$$(R + E)^* = (R^*E^*)^* \quad (1.55)$$

$$R + RE = R(\epsilon + E) \quad (1.56)$$

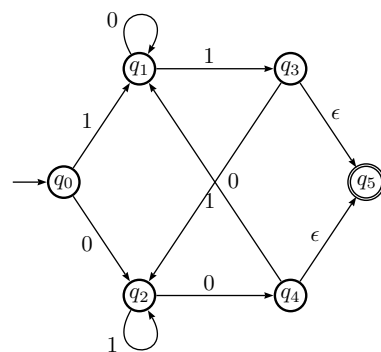
$$\emptyset + R = R + \emptyset = R \quad (1.57)$$

$$\emptyset R = R\emptyset = \emptyset \quad (1.58)$$

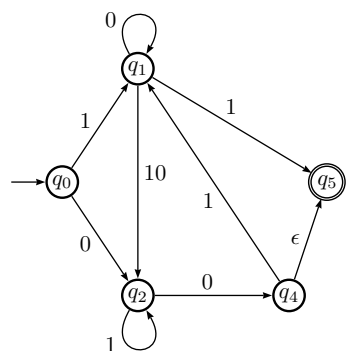
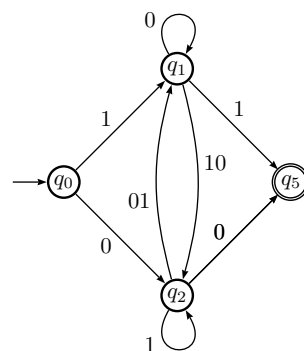
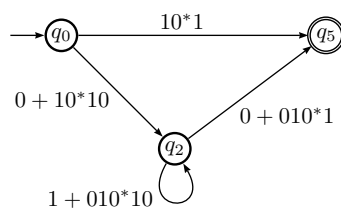
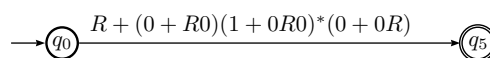
Algunas de estas reglas son más evidentes que otras, pero todas (y las que se deriven) pueden ser útiles. La regla (1.58) es particularmente útil para las enormes expresiones generadas por la ecuación (1.44).



(a) Autómata original.



(b) Se agrega un estado final único.

(c) Se elimina q_3 .(d) Se elimina q_4 .(e) Se elimina q_1 .(f) Se elimina q_2 . $R = 10^*1$ y se ha sustituido por cuestión de espacio.**Figura 1.19:** Transformación de un autómata a expresión regular por eliminación de estados. Ver texto para más detalles.

1.4. Propiedades de los lenguajes regulares

Aunque los ejemplos y operaciones ya presentados en secciones previas son en sí mismos propiedades de los lenguajes regulares, en este apartado se incluyen algunos casos notables, para introducir una clasificación básica de las propiedades de dichos lenguajes.

1.4.1. ¿Cómo saber que un lenguaje es regular?

Si un lenguaje es regular debiera ser simple probarlo pues basta mostrar el autómata que lo acepta. Pero no poder construir un autómata o una expresión regular no significa que no sea regular: la ausencia de prueba no es prueba de ausencia. Por eso se requiere un criterio más fuerte. Tal criterio es el lema del bombeo. Esto se aplica al caso de lenguajes infinitos, pues todo lenguaje finito es regular.

Proposición 6 (Lema del bombeo para lenguajes regulares). *Supongamos un lenguaje L que sea regular e infinito. Entonces existe una constante n , que depende del lenguaje L , tal que para toda cadena $w \in L$, con $|w| \geq n$, se puede descomponer w en tres cadenas $w = xyz$ que cumplen con*

1. $y \neq \epsilon$,
2. $|xy| \leq n$ y
3. para toda $k \geq 0$, la cadena xy^kz también pertenece a L .

El lema dice que para todo autómata que acepta un lenguaje infinito, en cada cadena hay una subcadena que es posible repetir arbitrariamente y la cadena resultante también pertenecerá al lenguaje.

Ejemplo 42. Sea el lenguaje $L = L(10^*)$, formado por cadenas de un uno y ceros (las potencias de 2 en binario). Tomando $n = 2$, una cadena que pertenece a ese lenguaje es $w = 100$ y cumple con $|w| \geq 2$. Se puede separar dejando un símbolo para cada subcadena $w = xyz$: $x = 1$ y $y = z = 0$. Así, para cualquier k , se cumple que $xy^kz \in L$ pues

$$xy^0z = 1\epsilon 0 = 10$$

$$xy^1z = 100$$

$$xy^2z = 1000$$

$$\vdots$$

y es clara la pertenencia.

Para un ejemplo de un lenguaje no regular, vaya a la página 55.

Sin embargo, este es un resultado un tanto débil pues sólo es condición necesaria, no suficiente: existen lenguajes regulares que no cumplen con el lema. En otras palabras: si cumple el lema entonces es regular. Si no lo cumple, no se sabe. Un resultado más fuerte es el siguiente. Primero, es necesario definir una relación de equivalencia entre cadenas.

Definición 16. Sea L un lenguaje. Las cadenas x y y de L están relacionadas, denotado por $x \equiv_L y$ y si para toda cadena $z \in \Sigma^*$ ocurre que $xz \in L$ siempre y cuando $yz \in L$. Si están relacionadas se dice que son indistinguibles.

Debe ser claro que las cadenas se *distinguen* si existe al menos una cadena z tal que xz y yz no estén al mismo tiempo en L .

Proposición 7 (Teorema de Myhill-Nerode). *Un lenguaje L es regular si y sólo si la relación $\equiv_L$ determina una cantidad finita de clases de equivalencia. Más aún, la cantidad de clases es igual al número de estados del AFD mínimo que acepta L .*

Para aplicarlo es necesario construir todas las clases de equivalencia para mostrar que es una cantidad finita. Otra aplicación útil y común de este resultado es la minimización de AFD.

1.4.2. Propiedades de cerradura

Estas propiedades son de la forma siguiente: dados dos lenguajes regulares, determinar si cierta operación entre ellos resulta en un lenguaje L que también es regular. Aunque en algunos casos esto sea obvio o intuitivo, es necesario comprobar formalmente la veracidad de cada cerradura.

Como se trata de lenguajes regulares, para cada uno hay un AFD que lo acepta, así que basta ver la manera de construir un AFD para el lenguaje L resultante. Esto se hace partiendo de los AFD originales.

Algunos ejemplos a continuación, planteados como proposiciones. Los detalles o las pruebas enteras son excelentes ejercicios para estudiantes diligentes.

Proposición 8. *La unión de dos lenguajes regulares es regular.*

Demostración. Sean L_1 y L_2 dos lenguajes regulares. Se trata de demostrar que el lenguaje $L = L_1 \cup L_2$ es también regular. Siendo regulares, existe un AFD que los acepta, digamos A_1 y A_2 respectivamente. El autómata que acepta a L se construye creando un nuevo estado inicial y conectándolo mediante transiciones épsilon a los estados iniciales de A_1 y A_2 , que pierden tal condición. También se agrega al nuevo AFD su único estado final, al que se conectan los estados finales de A_1 y A_2 y también dejan de ser estados finales.

Otra manera equivalente es partir de las expresiones regulares que generan a cada lenguaje. Sean E_1 y E_2 tales expresiones. La expresión $E_1 + E_2$ genera a L , es decir,

$$L_1 \cup L_2 = L(E_1) \cup L(E_2) = L(E_1 + E_2)$$

□

Proposición 9. *La intersección de dos lenguajes regulares es regular.*

Demostración. Esta demostración requiere un poco más de trabajo. Sean L_1 y L_2 dos lenguajes regulares. Se trata de demostrar que el lenguaje $L = L_1 \cap L_2$ es también regular.

Siendo regulares, existen AFDs que los aceptan, digamos $L_1 = L(A_1)$ y $L_2 = L(A_2)$. Sean $A_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ y $A_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$. En caso de que los alfabetos de fueran distintos, tomaríamos Σ como la unión de los alfabetos. Construiremos un autómata cuyos estados sean parejas de estados de A_1 y A_2 respectivamente, que simule el comportamiento de ambos autómatas al mismo tiempo.

El AFD A que acepta L se construye como

$$A = (Q_1 \times Q_2, \Sigma, \delta, (q_1, q_2), F_1 \times F_2)$$

y $\delta((p, q), a) = (\delta_1(p, a), \delta_2(q, a))$. Debe ser claro que si una de las transiciones $\delta_1(p, a)$ o $\delta_2(q, a)$ no existe, la transición entera $\delta((p, q), a)$ no existe.

Primeramente, debe extenderse δ para que procese cadenas, pues está definida sólo para símbolos de Σ . Puede probarse por inducción que $\hat{\delta}((q_1, q_2), w) = (\hat{\delta}_1(q_1, w), \hat{\delta}_2(q_2, w))$.

El AFD A aceptará la cadena w si y sólo si $\hat{\delta}((q_1, q_2), w)$ es una pareja de estados de aceptación, es decir, w se acepta siempre y cuando A_1 y A_2 la acepten, lo que significa que está en la intersección del lenguaje. $\square$

Para ilustrar esta construcción, va un ejemplo.

Ejemplo 43. La figura 1.20 muestran dos autómatas y su intersección. Las etiquetas que se han dado a los estados describen el resultado del producto cartesiano, solo cambiando la notación (p, r) por pr .

La intersección tiene una transición del estado pr a qs porque hay transiciones de p a q en el primer AFD (subfigura 1.20a) y de r a s en el segundo (subfigura 1.20b). No así con la transición del estado p sobre sí mismo. No aparece en la intersección porque no hay una equivalente de r a r .

El complemento de un lenguaje regular también es regular y la construcción es bastante simple. Es de gran utilidad para determinar si dos expresiones regulares generan el mismo lenguaje, un problema algo más complejo de lo que parece.

Proposición 10. El complemento de un lenguaje regular es regular.

Demostración. Sea L un lenguaje regular, aceptado por el AFD A definido como $A = (Q, \Sigma, \delta, q_0, F)$ y δ completa. Si δ no es completa, es necesario completarla agregando un estado adicional y las transiciones necesarias para que cada estado tenga una transición para cada símbolo. La idea es construir un autómata $B = (Q \cup \{q_f\}, \Sigma, \delta, q_0, Q - F)$ donde a cada estado final se le quita esa condición y los estados originales normales son ahora finales. Así, las cadenas aceptadas por A , al ser procesadas por B terminan en un estado que ya no es final y por tanto no son aceptadas.

El estado $\{q_f\}$ es un nuevo estado final accesible desde cada estado final de A , con transiciones para cada símbolo de Σ para aceptar cadenas del tipo xy donde el prefijo x es una cadena aceptada por A . $\square$

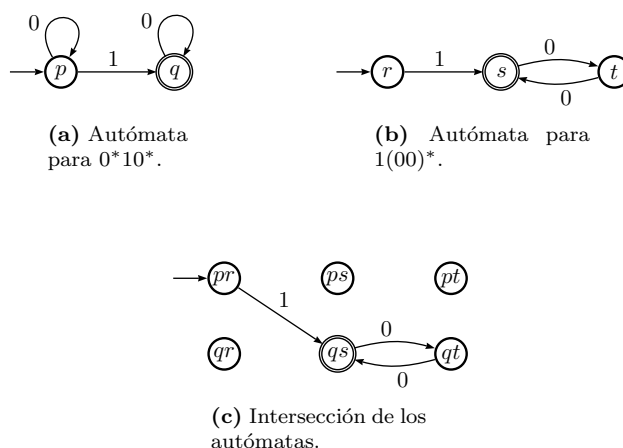


Figura 1.20: Dos autómatas y su intersección. Se presentan todos los estados generados, pero solo las transiciones coincidentes.

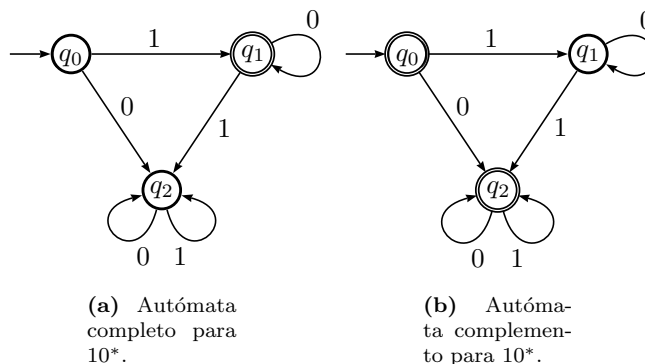


Figura 1.21: Un autómata y su complemento.

El siguiente ejemplo ilustra la construcción del complemento de un lenguaje regular, dado su AFD. Se trata del mismo ejemplo del lenguaje 10^* que aparece en la página 16 como AFD incompleto y que se completa en la página 18.

Ejemplo 44. La figura 1.21 muestran tres autómatas, el original (subfigura 1.21a) que acepta el lenguaje L , uno intermedio (subfigura 1.21b), que no altera el lenguaje L , para completar la función δ y finalmente su complemento, que acepta L^C .

La construcción anterior no es válida en caso de que se trate de un AFN. Esto es debido a que δ no se comporta como función en el caso del no determinismo.

Otras operaciones que también son cerradas para los AFD son:

1. Diferencia de lenguajes $L_1 - L_2$
2. Reflexión, invirtiendo cada cadena

3. Cerradura de Kleen L^*
4. Concatenación
5. Homomorfismo, que es la sustitución de cada símbolo de Σ por una cadena de otro alfabeto.

Se sugiere al lector analizar y discutir en equipo estas propiedades. Su demostración requiere recursos similares a las de las que ya se mostraron.

1.4.3. Propiedades de decisión

Las preguntas típicas de decisión son problemas de la forma ¿la cadena w está en el lenguaje L ? Esta es precisamente la definición de problema. A esta se suman dos preguntas que también son de decisión, primero si cierto que el lenguaje es vacío o si es infinito. De otra forma, una pregunta (sobre una propiedad) es de decisión si se responde con un SI o un NO.

Estas últimas dos preguntas de decisión son válidas cuando se describe un lenguaje o se trata de el resultado de una o más operaciones entre lenguajes y no está claro si el conjunto resultante es vacío o infinito.

En términos generales, para comprobar si un lenguaje regular es no vacío basta con encontrar una ruta del estado inicial a un estado final. Si lo que se tiene es una expresión regular, el único riesgo es cuando el conjunto vacío forma parte de tal expresión, por sus propiedades como lenguaje, por lo que se requiere simplificar la expresión.

Para saber si un lenguaje es infinito, basta encontrar un ciclo en su diagrama de transiciones o una cadena de longitud mayor o igual que la cantidad de estados.

1.5. Limitaciones de los lenguajes regulares

Las expresiones regulares y su equivalente en máquina abstracta, los AFD, son el modelo de cómputo más simple y por tanto, como es de esperarse, no son capaces de reconocer o aceptar todos los posibles lenguajes. Si bien es una cantidad infinita, esta es enumerable.

Ejemplo 45. *El lenguaje $L = \{01, 0011, 000111, 00001111, \dots\}$ no es regular. Esto significa que no hay AFD que lo acepte o expresión regular que produzca tales cadenas.*

Demostración. Es claro que no encontrar un autómata que acepte el lenguaje $0^n 1^n$ no significa que no exista: la ausencia de prueba no es prueba de ausencia.

Una forma de probarlo es emplear el lema de bombeo, tomando $n = 3$ por ejemplo la cadena $w = 0011$, que cumple pues $|w| = 4 > 3$. Como sea que se separe en subcadenas como $w = xyz$ fallará. Por ejemplo $x = 00$, $y = 1$ y $z = 1$.

Las cadenas

$$\begin{aligned}xy^0z &= 00\epsilon 1 \notin L \\xy^1z &= 0011 \in L \\xy^2z &= 00111 \notin L \\xy^3z &= 001111 \notin L \\&\vdots\end{aligned}$$

y sólo para $k = 1$ (la cadena original) se tiene que la cadena pertenece al lenguaje. Esto prueba que L no es regular. $\square$

El ejemplo anterior es el clásico que se emplea para limitar las posibilidades de los AFD. La razón por la que un AFD no puede aceptar tal lenguaje es que no tiene forma de saber cuántos ni cuáles símbolos ha procesado en la cadena de entrada. Es decir, no tiene memoria.

Así como ese lenguaje no es regular, muchos otros no lo son. Otros lenguajes no regulares son por ejemplo 0^n10^n , los palíndromos y una infinidad más. Más ejemplos se presentan en el resto de las notas.

Esta limitante es la que lleva al estudio del siguiente modelo abstracto de computadora, que es el que acepta los Lenguajes Libres de Contexto, en la siguiente sección.

Capítulo 2

Lenguajes Libres de Contexto

Introducción

En el caso de los lenguajes regulares, estos fueron definidos en términos de una máquina abstracta (el autómata finito y sus variantes) y luego se mostró la equivalencia con un método algebraico para definirlos (las expresiones regulares). Claro, la presentación pudo ser distinta: primero el modelo algebraico y luego el de la máquina.

Para la siguiente categoría de lenguajes, se invertirá el camino, presentando primero el modelo algebraico, que en este caso es en forma de gramática, y luego el modelo computacional (el autómata de pila). Los lenguajes regulares también se pueden definir en términos de una gramática muy simple, pero las expresiones regulares son el mecanismo preferido por su simplicidad y por estar más cercanas a lo que en la práctica se hace uso.

Como se mostrará, los lenguajes regulares son un subconjunto propio de los lenguajes libres de contexto. Ya se mostró un lenguaje (0^n1^n) que no es regular. Aquí se mostrará que es libre de contexto, junto con otros ejemplos.

Los lenguajes regulares permiten definir y modelar los lenguajes de programación. Son un elemento esencial en el diseño de compiladores y muchas componentes de software interactivo (aquel donde el usuario hace algo más que sólo dar CLICK).

2.1. Gramáticas Libres de Contexto

Una gramática es un conjunto de reglas con las que se pueden producir cadenas a partir de un alfabeto base. Las reglas se escriben haciendo uso de variables, el símbolo de producción $\rightarrow$ y símbolos terminales. Por ejemplo, si se dice que X es una variable y que puede sustituirse con un par de letras a ,

entonces la regla de producción asociada es

$$X \rightarrow aa.$$

En este caso, la letra a es un símbolo del alfabeto base Σ . La X es la parte izquierda de la producción (una variable) y aa la parte derecha de la producción. La parte derecha puede contener tanto variables como símbolos terminales.

De manera formal, una *Gramática Libre de Contexto* (GLL) G consiste en 4 entidades $G = (V, T, P, S)$, donde

1. V son las variables,
2. T los símbolos terminales, elementos de un alfabeto fijo Σ ,
3. P las reglas de producción, de la forma

$$X \rightarrow \alpha \quad (2.1)$$

con $X \in V$ y $\alpha \in (V \cup T)^*$ y

4. S el símbolo inicial.

La aplicación de reglas de producción para generar una cadena se llama *derivación* y se distingue de la producción al utilizar una flecha de dos líneas $\Rightarrow$ a la que se puede agregar como subíndice la gramática empleada. Si no hay confusiones entre gramáticas, como en este caso, el subíndice en $\Rightarrow_G$ se puede suprimir de la derivación y escribir sólo $\Rightarrow$. En este caso, haciendo analogía con los lenguajes regulares, el conjunto de símbolos terminales T equivale al alfabeto Σ .

Veamos esto con ejemplos.

Ejemplo 46. Sea $G = (\{S\}, \{0, 1\}, P, S)$ y las producciones P las reglas:

$$S \rightarrow 0S1 \mid 01 \quad (2.2)$$

Partiendo del símbolo inicial, es posible generar cadenas como

$$S \Rightarrow 01 \quad (2.3)$$

$$S \Rightarrow 0S1 \Rightarrow 0011 \quad (2.4)$$

$$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000111 \quad (2.5)$$

y de esta forma se ve que el lenguaje generado es $0^n 1^n$, el Ejemplo 45 de un lenguaje que no es regular.

Es un lenguaje libre de contexto pues se obtiene a partir de una gramática libre de contexto.

Se emplea $\Rightarrow^*$ para sintetizar una producción, quitando pasos intermedios y el $*$ significa que se aplican tantas derivaciones como sea necesario.. En el ejemplo anterior, puede escribirse $S \Rightarrow^* 000111$

Ejemplo 47. Un palindroma es una cadena simétrica con respecto a su centro, es decir, se lee lo mismo al derecho que al revés. En español algunas comunes son «Anita lava la tina» o «Dábale arroz a la zorra el abad». Claro, son palindromas una vez que no nos fijamos en espacios ni acentos. Para facilitar las cosas pensemos en los palindromas con los símbolos 0 y 1. Sea $G = (\{S\}, \{0, 1\}, P, S)$ y las producciones P las reglas:

$$S \rightarrow 0S0 \quad (2.6)$$

$$S \rightarrow 1S1 \quad (2.7)$$

$$S \rightarrow 0 \quad (2.8)$$

$$S \rightarrow 1 \quad (2.9)$$

$$(2.10)$$

Iniciando con el símbolo S , se pueden generar cadenas aplicando producciones en un orden arbitrario. Por la forma de las producciones, es claro que las cadenas que se producen son simétricas, por lo que esta gramática produce palindromas. Además, todas las cadenas son de longitud impar, pues para producir palindromas de longitud par se requiere la producción adicional $S \rightarrow \epsilon$ que genera la cadena vacía.

De esta manera, podemos derivar el palindroma 11011 aplicando 3 producciones de la gramática, lo que se escribe:

$$S \Rightarrow_G 1S1 \Rightarrow_G 11S11 \Rightarrow_G 11011$$

usando una flecha doble $\Rightarrow$ para distinguir la derivación de las producciones. La derivación puede resumirse escribiendo $S \Rightarrow^* 11011$.

Las cadenas intermedias entre el símbolo inicial y la cadena derivada, que combinan símbolos terminales con variables, son llamadas *formas sentenciales*. Se puede decir que una forma sentencial es un elemento del conjunto $F = (V \cup T)^*$. Para una gramática fija, sus formas sentenciales posibles son un subconjunto de F .

Cuando una misma variable posee varias producciones distintas, es posible escribirlas como una sola, pero separando los lados derechos con una barra vertical. En el ejemplo anterior, las producciones pueden reescribirse como

$$S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \quad (2.11)$$

sin que la gramática cambie, sólo se trata de ahorrar líneas de texto.

El *lenguaje de una gramática* es el conjunto de cadenas que pueden derivarse a partir del símbolo inicial, esto es, dada la gramática $G = (V, T, P, S)$, el lenguaje de G es

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}. \quad (2.12)$$

2.1.1. Derivaciones izquierdas y derivaciones derechas

Dada una producción de la forma $X \rightarrow AB$, la definición de gramática no indica cuál de las variables sustituir primero, es decir, en la derivación, puede

sustituirse A y luego B o al revés. Eso ocasiona que con para una misma cadena del lenguaje se pueden tener varias derivaciones distintas. Para evitar este problema es posible restringir la manera como se realizan las derivaciones.

Lo más sencillo es sustituir la variable que está más a la izquierda, en lo que se conoce como *derivación derecha*.

Ejemplo 48. Sea G una gramática con producciones

$$S \rightarrow AB \mid Bb \quad (2.13)$$

$$A \rightarrow aA \mid aa \quad (2.14)$$

$$B \rightarrow aBb \mid b \quad (2.15)$$

y debe entenderse que las mayúsculas son las variables y las minúsculas los terminales. Obsérvese las dos derivaciones siguientes:

$$1. S \Rightarrow AB \Rightarrow aAB \Rightarrow aaaB \Rightarrow aaab$$

$$2. S \Rightarrow AB \Rightarrow Ab \Rightarrow aAb \Rightarrow aaab$$

$$3. S \Rightarrow Bb \Rightarrow aBbb \Rightarrow abbb$$

Las primeras dos producen la misma cadena, en un orden distinto de derivaciones. La primera derivación se obtiene sustituyendo siempre la variable más a la izquierda, por lo que es una derivación izquierda. De la misma forma, la segunda derivación es una derivación derecha.

2.1.2. Árboles de derivación

Una manera de ver gráficamente una derivación completa es con su árbol de derivaciones. Consiste en un diagrama en forma de árbol, con las sustituciones de variables como ramificaciones y los símbolos terminales como las hojas.

Para introducir esta idea, se puede partir de la definición de árbol como estructura de datos. Sea cada dato almacenado en una estructura básica llamada *nodo*.

Definición 17. Un árbol T es un conjunto no vacío n de nodos, donde uno de los nodos es llamado raíz y el resto están organizados en subconjuntos disjuntos que son a su vez árboles, llamados subárboles o hijos de la raíz.

Esta definición es en esencia recursiva pues se ha definido el tipo abstracto de datos árbol de n 4 nodos en términos de árboles de menos de n nodos. Los nodos que ya no tiene hijos se llaman hojas.

Un árbol de derivación es una representación, preferentemente gráfica, del conjunto total de las derivaciones gramaticales que producen una cadena. Para el ejemplo 48, la cadena $aaab$ tiene el árbol de derivaciones de la figura 2.1.

En ocasiones se prefiere dibujar simples segmentos de recta en vez de flechas. En los árboles de derivación, las hojas siempre son símbolos terminales de la gramática o incluso la cadena vacía. Las variables siempre son nodos intermedios.

Las derivaciones izquierda o derecha significan recorrer el árbol de un lado o de otro, lo que se conoce como preorden o postorden.

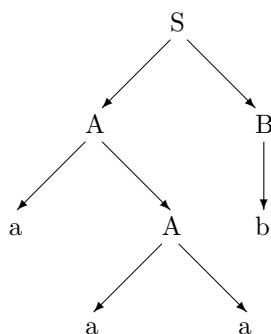


Figura 2.1: Árbol de derivación de la cadena *aaab*.

2.1.3. Formas normales

Es de gran utilidad práctica que cada gramática tenga reglas de producción con el mismo formato, para facilitar su análisis y uso. Las siguientes son las más importantes, aunque en la literatura especializada hay más.

Forma Normal de Chomsky

Una gramática está en forma normal de Chomsky (FNC) cuando en sus producciones, cada variable se transforma en otras dos variables o en un terminal, es decir, las producciones son de la forma

$$A \rightarrow BC \quad (2.16)$$

o

$$A \rightarrow a \quad (2.17)$$

y en los casos necesarios, $A \rightarrow \epsilon$.

Toda gramática que no genere la cadena vacía se puede poner en FNC. Su uso tiene varias ventajas. Los árboles sintácticos son binarios y son fáciles de utilizar. Hay muchos algoritmos para convertir toda gramática en FNC.

Proposición 11. *Cada LLC tiene una gramática en Forma Normal de Chomsky que la genera. Toda gramática en Forma Normal de Chomsky genera un LLC.*

Demostración. La base de esta demostración es ver cómo transformar una GLC cualesquiera en una con Forma Normal de Chomsky. La manera de transformar la producción genérica

$$X \rightarrow Y_1 Y_2 \cdots Y_n$$

en un conjunto de producciones con la forma (2.16) es generando variables

intermedias:

$$\begin{aligned}
 X &\rightarrow Y_1 Z_1 \\
 Z_1 &\rightarrow Y_2 Z_2 \\
 Z_2 &\rightarrow Y_3 Z_3 \\
 &\vdots \\
 Z_{n-2} &\rightarrow Y_{n-2} Z_{n-1} \\
 Z_{n-1} &\rightarrow Y_{n-1} Y_n
 \end{aligned}$$

En el caso de producciones que generan varios símbolos terminales, cada símbolo terminal se convierte en una variable y de nuevo se aplica el paso anterior. $\square$

Ejemplo 49. Para la gramática del ejemplo 47, sólo hay que cambiar las dos primeras reglas de producción, pues las dos últimas ya están en Forma Normal de Chomsky. La regla $S \rightarrow 0S0$ genera las producciones

$$\begin{aligned}
 S &\rightarrow CC_1 \\
 C_1 &\rightarrow SC
 \end{aligned}$$

y la regla $S \rightarrow 1S1$ genera las producciones

$$\begin{aligned}
 S &\rightarrow UU_1 \\
 U_1 &\rightarrow SU
 \end{aligned}$$

finalizando con las producciones $C \rightarrow 0$ y $U \rightarrow 1$.

Forma Normal de Greibach

Una gramática está en forma normal de Greibach cuando cada producción es de la forma

$$A \rightarrow a\alpha,$$

donde $a \in T$ y α es un elemento de V^* , es decir, una concatenación de variables que incluye la producción $A \Rightarrow \epsilon$.

Como cada regla de producción genera un símbolo, cada derivación procesa un símbolo de la cadena de entrada. Por ello, la cantidad de derivaciones empleadas se conoce por anticipado pues es la longitud de la cadena que se desea producir.

La conversión a esta forma es mucho más laboriosa y requiere primero la conversión a la Forma Normal de Chomsky.

2.1.4. Simplificación de gramáticas

En esta sección veremos la manera de eliminar de una gramática aquello que no es útil, ya sea variables o terminales que no ayudan a generar cadenas.

2.1.5. Ejemplos de aplicaciones

Las GLC tienen una gran importancia en el campo de los lenguajes de programación o los lenguajes de marcado (como html o $\text{\LaTeX}$). Cada lenguaje es definido en términos de una gramática. Además se emplean en todo tipo de motores de búsqueda, a veces en forma de expresiones regulares (estas emplean una forma simple de gramáticas llamadas gramáticas regulares).

Ejemplo 50. *La gramática del ejemplo 47 se extiende fácilmente para aceptar cadenas que acepten delimitadores correctamente emparejados. Para un lenguaje como C o derivados, los delimitadores son los paréntesis, las llaves y los corchetes. Las reglas de producción son muy simples:*

$$S \rightarrow \{S\} \mid [S] \mid (S) \mid \epsilon$$

Ejemplo 51. *las expresiones aritméticas son otro ejemplo sencillo. El caso de operaciones con un sólo dígito se obtiene con la siguiente gramática.*

$$\begin{aligned} E &\rightarrow E + E \mid E - E \\ E &\rightarrow E * E \mid E / E \\ E &\rightarrow EE \mid 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

Es fácil modificarla para que los números no tengan restricciones.

Estos ejemplos caen directamente en lo que se conoce como analizadores sintácticos. Se estudiarán unas secciones más adelante.

2.1.6. Gramáticas ambiguas

Aunque expresiones como $a + b \times c$ y $(a + b) \times c$ en el ejemplo 51 tienen distinto significado, sus derivaciones son equivalentes:

$$E \rightarrow E * E \rightarrow E * E + E \quad (2.18)$$

$$E \rightarrow E + E \rightarrow E * E + E \quad (2.19)$$

El árbol de derivación para cada una de estas expresiones tiene su propia forma, que se muestra en la figura 2.2.

Esto significa que la gramática no proporciona medios para diferencias significados. Estos casos son conocidos como gramáticas ambiguas y su ambigüedad se debe a la existencia de más de un árbol de derivación distinto para una misma cadena, lo que no es conveniente en la construcción de compiladores.

Para eliminar la ambigüedad es necesario poder indicar la precedencia de las operaciones, o de las producciones, en general. Esto se hace agregando variables o reglas que obliguen tal precedencia. La gramática se puede construir como

$$\begin{aligned} E &\rightarrow E + F \mid E - F \\ F &\rightarrow E * E \mid E / E \\ F &\rightarrow (F) \mid D \\ D &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

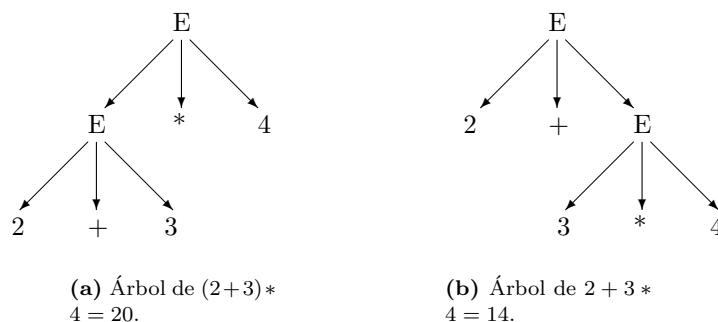


Figura 2.2: Árboles sintácticos que generan una ambigüedad aritmética. Siempre debe ser claro el significado de las operaciones.

Agregar variables o reglas no es la única manera de resolver este problema, en ocasiones se puede especificar la forma de agrupar las operaciones.

Cuando no es posible eliminar la ambigüedad de una gramática es porque existen lenguajes para los que todas sus gramáticas son ambiguas, llamados *inherentemente ambiguos*.

Ejemplo 52. La gramática

$$\begin{aligned} S &\rightarrow 0A \mid 1B \\ A &\rightarrow 0AA \mid 1S \mid 1 \\ B &\rightarrow 1BB \mid 0S \mid 0 \end{aligned}$$

es ambigua pues la cadena 001101 puede derivarse de dos maneras distintas:

$$\begin{aligned} S &\Rightarrow 0A \Rightarrow 00AA \Rightarrow 001S1 \Rightarrow 0011B1 \Rightarrow 001101 \\ S &\Rightarrow 0A \Rightarrow 00AA \Rightarrow 0011S \Rightarrow 00110A \Rightarrow 001101 \end{aligned}$$

Claro, si se puede encontrar una gramática equivalente que no sea ambigua, no hay problema. Casos irresolubles no tienen esta opción:

Ejemplo 53. Un ejemplo es el lenguaje

$$L = \{a^n b^n c^m d^m \mid n \geq 1, m \geq 1\} \cup \{a^n b^m c^m d^n \mid n \geq 1, m \geq 1\}$$

para la cuál no hay ni una gramática que no sea ambigua.

Por fortuna, la gran mayoría de lenguajes útiles en la práctica no son inherentemente ambiguos, por lo que no ofrecen problemas para encontrar una gramática adecuada para la construcción de compiladores.

Se sabe que si una cadena con dos árboles de derivación distintos, también tiene dos derivaciones izquierdas distintas y viceversa.

Hay mucha más teoría desarrollada con respecto a las gramáticas libres de contexto, pero rebasa los alcances de este curso. Para mayor información, remítase el lector a libros especializados en el tema.

2.2. Autómatas de Pila

Un Autómata de Pila (AP) es un autómata al que se le da la posibilidad de llevar una memoria sencilla en forma de pila, estructura de datos tipo lista donde se agrega y retira información siempre por el mismo extremo.

De manera similar que con los AFD (página 15), un AP es una tupla de conjuntos, función y elementos $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ que consiste en lo siguiente:

1. Q un conjunto de estados,
2. Σ el alfabeto de la cinta de entrada,
3. Γ el alfabeto de la pila,
4. $\delta : Q \times \Sigma \cup \{\epsilon\} \times \Gamma \rightarrow Q \times \Gamma^*$ la función de transición,
5. $q_0 \in Q$ el estado inicial,
6. $Z_0 \in \Gamma$ el símbolo del fondo de la pila y
7. $F \subset Q$ el conjunto no vacío de los estados finales o de aceptación.

De esta manera, un AFD es una tupla $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ junto con una cinta de entrada donde se escribe una cadena de caracteres que será procesada. La pila inicia con Z_0 como único elemento. La transición que se emplea queda completamente determinada por el símbolo que se lee en la cinta de entrada y el que está en el tope de la pila, que se saca para revisar.

Las transiciones tienen la forma

$$\delta(q, a, x) = (q, w) \quad (2.20)$$

que indica que, estando en el estado p , el AP procesa el carácter a de la entrada al mismo tiempo que saca el símbolo x del tope de la pila. La transición hace pasar al AP al estado q y enviar la cadena w a la pila. Si el símbolo a no está al inicio de la entrada que se está procesando o x no está en el tope de la pila, entonces la transición no se puede llevar a cabo.

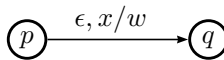
Una pregunta que es importante hacer es si la pila debe quedar vacía cuando el autómata termine de procesar una cadena como requisito para aceptarla. Este es un detalle importante que se estudiará en breve.

Hay varias estrategias que se siguen para procesar cadenas y trabajar con la pila. Dos de gran utilidad son las siguientes.

1. Una transición de la forma

$$\delta(p, \epsilon, x) = (q, w)$$

que actúa sobre la pila (retira x del tope y agrega la cadena w) sin procesar la cadena de entrada. Es esta forma la que requiere que la función δ tenga como parte de su dominio a $\Sigma \cup \{\epsilon\}$ y no sólo a Σ .



manc

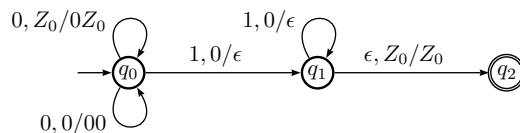
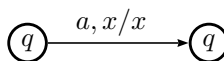


Figura 2.3: Autómata de pila que reconoce el lenguaje $0^n 1^n$. Se entiende que al inicio la pila sólo contiene a Z_0 . Este autómata es completamente determinístico.

2. La transición

$$\delta(q, a, x) = (q, x)$$

procesa el símbolo a de la cadena de entrada y deja intacta la pila, pues saca x del tope y lo regresa.



Las anteriores dos transiciones aparecen comúnmente en diversas etapas de los autómatas de pila. Veamos estas estrategias en uso con algunos ejemplos.

Ejemplo 54. *El lenguaje no regular $0^n 1^n$ puede ser generado por una gramática muy sencilla, mostrada en el ejemplo 46 (pág. 58). Cada cadena producida por esa gramática puede ser aceptada por un autómata de pila. Basta que el autómata procese todos los símbolos 0 agregándolos a la pila y cuando encuentre los símbolos 1, saque un 0 por cada 1 para verificar que sean la misma cantidad. Este autómata se muestra en la figura 2.3.*

Es importante notar que cada transición del autómata de pila del ejemplo anterior es determinística. En ningún momento hay forma de tomar dos rutas distintas. No es el caso del autómata siguiente.

Ejemplo 55. *retomando los palindromos generados por la gramática del ejemplo 47, también pueden ser reconocidos por un autómata de pila que guarde la mitad de la cadena en la pila y luego vaya sacando cada símbolo para comprobar que es igual al que corresponde en los que quedan en la cinta de entrada. Esto se hace con el autómata de pila del diagrama 2.4, solo que para palindromos de longitud par.*

El estado q_0 guarda en la pila la primera mitad de la cadena, el estado q_1 comprueba si lo guardado en la pila es idéntico a lo que resta en la cinta de entrada.

En el caso del ejemplo 2.4, la transición que se selecciona depende al mismo tiempo del símbolo de la cinta de entrada y del que está en el tope de la pila, como se dijo en la definición. Con este autómata, hay que notar lo siguiente.

Supongamos que estamos en el estado q_0 y que ya procesamos los dos primeros símbolos de la cadena 00011000. En la cinta de entrada hay un 0 (el tercero)

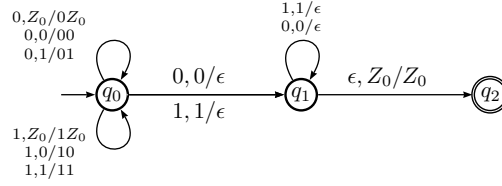


Figura 2.4: Autómata de pila que acepta palíndromos de longitud par del alfabeto $\Sigma = \{0, 1\}$. Este autómata es no determinístico, pues el autómata no sabe dónde es la mitad de la cadena.

en la cabeza de lectura y en el tope de la pila otro 0 (el que se acaba de procesar). Se pueden usar dos transiciones, una que lo mantiene en q_0 y otra que lo lleva a q_1 . Quedarse en q_0 es lo correcto, pues aún no se llega a la mitad.

Esto significa que el autómata de pila es intrínsecamente no determinista. Más adelante se verá la manera como esto limita la cantidad de lenguajes que el autómata de pila puede aceptar.

2.2.1. Descripción instantánea de los AP

Lo que ocurre con el AP en un momento arbitrario puede ser descrito con la tripleta (q, a, α) , llamada descripción instantánea del AP, donde

1. $q \in Q$ es el estado actual,
2. $w \in \Sigma^*$ es lo que resta de la cadena que se procesa y
3. $\alpha \in \Gamma^*$ el contenido de la pila.

De esta manera, con la descripción instantánea es posible determinar el estado exacto del autómata y los pasos sucesivos en el procesamiento de la cadena de entrada. Debe ser claro que al inicio, todo autómata de pila tiene la descripción instantánea (q_0, w, Z_0) , es decir, el estado inicial, toda la cadena original w que se procesará y el símbolo Z_0 como contenido único de la pila.

Para indicar que se pasa de una descripción estándar a otra se emplea el símbolo $\vdash$.

La descripción instantánea del final del procesamiento dependerá de la cadena y si se tiene o no éxito. Para un autómata que logra procesar toda su cadena y vaciar la pila, tal descripción queda (q_f, ϵ, Z_0) , donde se supone que q_f es un estado final.

Para la cadena 000111, la sucesión de descripciones instantáneas del autómata de pila del ejemplo 2.3 son

$$(q_0, 000111, Z_0) \vdash (q_0, 00111, 0Z_0) \vdash (q_0, 0111, 00Z_0) \vdash (q_0, 111, 000Z_0) \vdash (q_1, 11, 00Z_0) \vdash (q_1, 1, 0Z_0) \vdash (q_1, \epsilon, Z_0) \vdash (q_2, \epsilon, Z_0)$$

por lo que la cadena es aceptada.

Una cadena que no pertenece al lenguaje no alcanzará el estado final. Por ejemplo, 001 falla pues cuando se procesa el 1, se saca un cero de la pila, pero queda uno más, por lo que la última transición no se puede realizar.

En el ejemplo anterior, también es válido resumir la sucesión de descripciones instantáneas como

$$(q_0, 000111, Z_0) \vdash^* (q_2, \epsilon, Z_0) \quad (2.21)$$

donde el símbolo $\vdash^*$ indica que se aplican varias transiciones para pasar de la primera a la última.

Ejemplo 56. *En todo lenguaje de programación procedimental existen las sentencias de control **if-else**. Se entiende que debe haber un **if** para cada **else**, pero no puede haber **else** sin **if**. Claro sentencias **if** sin **else** sí son permitidas. Si sólo se emplean las iniciales **ie** en vez de **if else**, se puede pensar en construcciones de sentencias de control representadas con cadenas de símbolos $\{i, e\}$, dando lugar a el lenguaje L_{ie} que representa las construcciones correctas de sentencias de control **if-else**.*

Ejemplos de cadenas en este lenguaje son

$$\{i, ie, ii, iie, iei, ieie, iiee, iieie, iiiieieie, \dots\}$$

y cadenas inválidas son

$$\{e, ei, iee, ieei, iiiiee, \dots\}.$$

*Una manera de describir este lenguaje es decir que consta de todas las cadenas de símbolos **i** y **e** que comienzan con **i** y siempre hay al menos la misma cantidad de símbolos **i** que **e** cuando se cuenta desde el inicio de la cadena. Un autómata de pila que acepte este lenguaje es fácil de construir pues sólo requiere enviar a la pila cada **i** que llega y sacar una **i** cada vez que se procesa una **e**.*

En este caso, una cadena se acepta sin importar si la pila está vacía.

En el ejemplo anterior, se da un caso indeseable en la práctica pues al término de su ejecución el AP no deja vacía la pila, lo que podría ocasionar que la memoria se fuera acabando poco a poco al usar una y otra vez el algoritmo.

El remedio es agregar un estado que se encargue de vaciar la pila y cuando esté vacía pasar al nuevo único estado de aceptación. Esta solución implica que el autómata debe usar el no determinismo.

2.2.2. Lenguaje de un Autómata de Pila

Con las definiciones anteriores, ya podemos definir cuál es el lenguaje de un Autómata de Pila. Dado el autómata $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, el lenguaje que acepta es

$$L(P) = \{w \in T^* \mid \delta(q_0, w, Z_0) \vdash^* (p, Z_0) \text{ con } p \in F\} \quad (2.22)$$

En esta definición se está suponiendo que se acepta la cadena al llegar a un estado final y estando la pila vacía, pero también se puede relajar la

condición, aceptando cadenas exclusivamente por alcanzar un estado final o al vaciar la pila, claro, en ambos casos al procesar la cadena. De esta forma, se tienen lenguajes diferentes para un AP dependiendo de la restricción impuesta.

2.2.3. Equivalencia entre GLL y AP

Ahora falta probar que las GLC son equivalentes a los AP, y de esta forma comprobaremos que el lenguaje de los AP son los Lenguajes Libres de Contexto (definidos como en la ecuación (2.12). Tal y como las LR equivalen a los AFD, en el sentido de que para cada LR hay un AFD y viceversa, también para cada GLC hay un AP y viceversa.

Primero, a partir de una gramática G se puede construir un Autómata de Pila P que simule la gramática. Es realmente sencilla la construcción. Sea la gramática $G = (V, T, P, S)$. El autómata se construye como

$$P = (\{q_0\}, T, T \cup V, \delta, S, \{q_0\})$$

un autómata de un sólo estado. El símbolo inicial S se convierte en el símbolo de pila vacía Z_0 . Las transiciones se forman de la siguiente manera. Para cada producción

$$X \rightarrow \alpha$$

con α una combinación de variables y terminales, es decir, $\alpha \in V \cup T$, se construye la transición que no procesa ningún caracter

$$\delta(q_0, \epsilon, X) = (q_0, \alpha)$$

lo que significa que si el autómata P encuentra la variable X en la pila, la retira y la sustituye con uno de sus lados izquierdo α .

Debe recordarse que aquí está implícito el no determinismo, pues se sustituye por cada uno de los lados derechos, tomando múltiples ejecuciones paralelas, aunque sea más fácil pensar que el autómata hace la sustitución conveniente. Estas transiciones generan derivaciones de lado izquierdo.

Para cada terminal a , se construye la transición

$$\delta(q_0, a, \epsilon) = (q_0, \epsilon)$$

que procesa el símbolo a de la entrada al tiempo de retirarlo de la pila.

Ejemplo 57. Retomando el ejemplo del lenguaje $a^n b^n$, generado por la gramática

$$S \rightarrow aSb \mid ab$$

el autómata que lo simula se presenta en la figura 2.5.

Para la cadena $aaabbb$, el autómata procede con las descripciones instantáneas siguientes:

$$\begin{aligned} (q_0, aaabbb, S) &\vdash (q_0, aaabbb, aSb) \vdash (q_0, aabbb, Sb) \\ &\vdash (q_0, aabbb, aSbb) \vdash (q_0, abbb, Sbb) \\ &\vdash (q_0, abbb, abbb) \vdash (q_0, bbb, bbb) \\ &\vdash (q_0, bb, bb) \vdash (q_0, b, b) \vdash (q_0, \epsilon, \epsilon) \end{aligned}$$

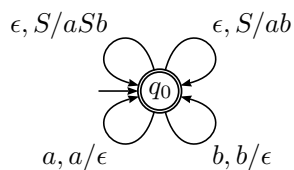


Figura 2.5: Autómata de pila que simula la gramática del lenguaje $a^n b^n$.

Con esta sucesión de descripciones instantáneas, es evidente que lo que simula el autómata es la derivación (por la izquierda):

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aaabbb.$$

2.2.4. Limitaciones de los AP

Así como se presentó el lema de bombeo para lenguajes regulares, existe un resultado equivalente para LLC, que se emplea para mostrar que algunos lenguajes no son libres de contexto.

Proposición 12 (Lema del bombeo para lenguajes libres de contexto).¹ *Supongamos un lenguaje L que sea un LLC. Entonces existe una constante n , que depende del lenguaje L , tal que para toda cadena $z \in L$, con $|z| \geq n$, se puede descomponer z en cinco cadenas $z = uvwxy$ que cumplen con*

1. $yvx \neq \epsilon$,
2. $|vwx| \leq n$ y
3. para toda $k \geq 0$, la cadena uv^kwx^ky también pertenece a L .

Ejemplo 58. *El lenguaje $L = \{a^n b^n c^n\}$ no es regular. Puede ser un tanto comprensible a partir del ejemplo 45, donde se presentó un lenguaje que no es regular pero sí libre de contexto. Ya se vio en la página 66 cómo un autómata de pila acepta el lenguaje $0^n 1^n$.*

Como en el caso del lema del bombeo para lenguajes regulares, hay lenguajes que no son libres de contexto y satisfacen el lema del bombeo.

Existen herramientas más poderosas para probar que un lenguaje es LLC, pero pese a los esfuerzos ninguna tiene garantías completas. El siguiente resultado es más fuerte que el lema del bombeo.

Proposición 13 (Lema de Ogden). *Si un lenguaje L es libre de contexto, entonces existe un número k tal que para cualquier cadena $w \in L$ de longitud mayor o igual que k y cualquier manera de marcar k posiciones en w , la cadena puede separarse como $w = xyzv$ y las subcadenas cumplen*

1. La cadena xy tiene al menos una posición marcada,
2. xyz tiene al menos k posiciones marcadas y
3. $ux^i y z^i v \in L$ para cada $i \geq 0$.

¹También conocido como el lema Bar-Hillel.

2.2.5. Autómatas de Pila Determinísticos (APD)

El lenguaje $L = \{x^n y^n, n > 0\} \cup \{x^n y^{2n}\}$ no es aceptado por ningún autómata de pila determinista.

Los lenguajes aceptados por los APD se denominan *Lenguajes Independientes del Contexto Deterministas*. Es un conjunto más reducido que el de los Lenguajes Libres de Contexto, pero aún así, útil para diseñar rutinas de análisis sintáctico.

Aún más, el conjunto de lenguajes se reduce si se exige que el APD vacíe su pila al terminar de procesar una cadena.

2.3. Lenguajes y compiladores

Los lenguajes de programación y los compiladores con que trabajamos son dos importantísimas aplicaciones, por demás evidentes, de la teoría de lenguajes formales, particularmente de las gramáticas libres de contexto. Es la razón principal de su estudio.

La manera como se definen la mayoría de las características de un lenguaje es mediante gramáticas. Por ejemplo, en lenguaje C, las sentencias de iteración se declaran con las reglas de producción como

```
while ( Expr ) Sentencia
do Sentencia while ( Expr );
for ( Decl Expr ; Expr ; Expr ) Sentencia
```

A su vez, *Expr* y *Sentencia* son variables de la gramática que tiene sus propias producciones.

Lo que hay en un compilador es un analizador sintáctico, cuyo objetivo es resolver un problema: determinar si la cadena w (el programa que escribimos) pertenece al lenguaje generado por la gramática del lenguaje. Esto se responde encontrando una derivación con las producciones de la gramática del lenguaje que lleven a obtener w , lo puede hacerse de varias maneras, ya sea partiendo del símbolo inicial S y llegando hasta la cadena w , lo que se conoce como *análisis descendente*. Otra forma es en sentido contrario, partiendo de la cadena y encontrando la derivación en sentido inverso, conocido como *análisis ascendente*. También es posible combinar ambas estrategias.

Como la derivación se puede encontrar derivando por la izquierda, por la derecha o leyendo la cadena en ambos sentidos, esto da origen a diversos analizadores sintácticos, cada uno con características que lo hace deseable para propósitos específicos o restricciones de eficiencia.

Así como la tabla de transiciones de un AFD permite construir con gran facilidad un analizador léxico que reconoce cierto lenguaje, para un AP es posible definir una tabla que facilita el trabajo de un analizador sintáctico. El mayor problema en la práctica es la existencia de no determinismo en el AP, lo que no puede simularse eficientemente.

Por lo general se parte de la gramática y se la da una forma adecuada para que el proceso de análisis sea eficiente. El conflicto mayor es llamado recursividad izquierda, pero en toda gramática válida siempre hay forma de eliminarla. Se mostrará la manera de hacerlo y luego se presentan dos de los analizadores más simples, que hacen uso de una gramática donde no hay recursividad izquierda.

Hay otros tipos de analizadores, como los LR que son muy eficientes, pero no se presentarán en estas notas.

2.3.1. Eliminación de recursividad izquierda

Se dice que una producción es recursiva por la izquierda cuando la variable que la produce aparece en la primera posición en la forma sentencial producida, es decir, cuando son de esta forma:

$$X \rightarrow X\alpha \quad (2.23)$$

con α cualquier combinación de variables y terminales. Es claro que la intención de una producción de esa naturaleza es repetir α cuantas veces se requiera, pero a la hora de programarla, la variable X (que se convierte en una función del programa, se llama a sí misma de inmediato, lo que ocasiona un ciclo recursivo del que no se puede salir².

La producción (2.23) no puede estar sola, debe haber otra producción para la variable X . De no ser así, se estaría sustituyendo a sí misma recursivamente sin parar. Debe haber otra producción cuyo lado derecho no comienza con X , de la forma $X \rightarrow \beta$. De esta forma, la sustitución de X puede ser algo como

$$X \Rightarrow X\alpha \Rightarrow X\alpha\alpha \Rightarrow X\alpha\alpha\alpha \Rightarrow \beta\alpha\alpha\alpha$$

y se han aplicado 3 veces la producción (2.23) y $X \rightarrow \beta$ una vez al final. Esta derivación sugiere la idea de producir primero lo no recursivo y luego realizar las repeticiones. Primero se crea una nueva variable T y se agregan las producciones

$$X \rightarrow T\alpha \quad (2.24)$$

$$T \rightarrow \beta T \quad (2.25)$$

$$T \rightarrow \beta \quad (2.26)$$

y de esta forma se tiene una gramática equivalente a $X \rightarrow X\alpha \mid \beta$ que no es recursiva por la izquierda.

Incluso en el caso de que una variable tenga una producción como

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid A\alpha_3 \mid \beta_1 \mid \beta_2$$

será necesaria solo una variable extra para eliminar la recursividad:

$$A \rightarrow \beta_1 T \mid \beta_2 T \mid \beta_1 \mid \beta_2$$

$$T \rightarrow \alpha_1 T \mid \alpha_2 T \mid \alpha_3 T$$

²Hasta que la pila del sistema se llena y el programa es botado de la memoria.

Código 2.1: La función que procesa la producción 2.27.

```

1 void A()
2 {
3     if ( Cad[Pos] == 'a' ) {
4         Procesar( 'a' );
5         B();
6         Procesar( 'c' );
7         A();
8     } else if ( Cad[Pos] == 'b' ) {
9         Procesar( 'b' );
10        C();
11        A();
12    } else {
13        B();
14        Procesar( 'a' );
15    }
16 }

```

2.3.2. Analizadores Recursivos Descendentes

Los analizadores recursivos descendentes son los más sencillos de todos y se construyen casi de manera directa a partir de la gramática. Cada variable se convierte en una función y las producciones en el cuerpo de cada función. Considérese la producción

$$A \rightarrow aBcA \mid Ba \mid bCA \quad (2.27)$$

que son solo tres producciones de una gramática mayor. Esta variable se convierte de manera directa en una función que se muestra en el código 2.1, donde la cadena que se revisa está en el arreglo `Cad` y la variable `Pos` tiene la posición del símbolo que se está revisando. La función `Procesar` verifica que el símbolo enviado sea el que corresponde y avanza en la revisión de la cadena.

Es por ello que programar analizadores recursivos descendentes es tan sencillo, una vez que la gramática no tiene recursividad. Si la gramática es muy grande, se requiere de programas adicionales para comprobar que no sea recursiva o convertirla en caso de serlo.

2.3.3. Analizadores LL(1)

El nombre de estos analizadores significa Left-Left y se debe a que buscan una derivación izquierda, al tiempo de analizar la cadena de entrada (el programa) de izquierda a derecha. Como en ciertas situaciones, se puede aplicar más de una regla de derivación, observa un símbolo de la entrada por adelantado, es decir, sin procesarlo, para que sirva en la decisión de cuál regla utilizar. Es por eso del 1 entre paréntesis, porque se revisa un símbolo.

Es importante aclarar que se presentan este tipo de analizadores por su simplicidad, no por su generalidad. Un lenguaje como el C tiene analizadores LL(1),

pero el lenguaje C# no los admite debido a ciertas instrucciones semánticas, como las expresiones lambda.

Para poder encontrar la derivación que genere la cadena w a partir de la GLC, se requiere que no haya recursividad por la izquierda, es decir, producciones de la forma

$$A \rightarrow A\alpha|B$$

donde la variable del lado derecho aparece como primer símbolo de la forma sentencial. α puede ser cualquier combinación de variables y terminales. Esta producción puede sustituirse agregando una nueva variable T y definiendo

$$A \rightarrow BT$$

$$T \rightarrow \alpha T$$

$$T \rightarrow \epsilon$$

que es equivalente y libre de recursividad izquierda.

En el tema de análisis léxico, sección 1.2.7, página 22, el proceso de análisis es guiado por la tabla de transiciones. En este caso hay una equivalencia pues se construye una tabla que permite ir decidiendo por la producción que será empleada al ir construyendo la derivación izquierda.

Para construir esta tabla guía, se necesita primero definir dos conjuntos, llamados comúnmente First y Follow. Consisten en los primeros símbolos por los que se puede sustituir una variable luego de las derivaciones necesarias y los que pueden seguir de manera inmediata. Se requiere First y Follow para cada variable de la gramática.

El algoritmo para encontrar los conjuntos First y Follow de una variable cualquiera X es el siguiente. Primeramente el conjunto First.

1. Si X fuera un símbolo terminal, entonces $\text{First}(X) = \{X\}$.
2. Si $X \rightarrow \epsilon$ es una producción, entonces ϵ se agrega a $\text{First}(X)$.
3. Si existe la producción $X \rightarrow Y_1 Y_2 \dots Y_k$, entonces el símbolo terminal a se agrega a $\text{First}(X)$ si para algún i , a está en $\text{First}(Y_i)$ y ϵ está en $\text{First}(Y_1), \text{First}(Y_2), \dots, \text{First}(Y_{i-1})$.
Si ϵ se produce por $Y_1 Y_2 \dots Y_k$, entonces ϵ se agrega a $\text{First}(X)$.

Para construir el conjunto Follow,

Lo central es que para construir $\text{First}(X_1 X_2 \dots X_n)$, se agregan los conjuntos First de cada variable X_i y la cadena vacía siempre que alguna de las variables la produzca.

A partir de estos conjuntos, se puede construir la tabla de rastreo o tabla guía, en inglés *parsing table*, que nos permite encontrar de manera sencilla la derivación izquierda.

2.4. Propiedades de los LLC

2.4.1. Propiedades de decisión

Las dos preguntas de decisión más importantes son si un LLC L está vacío y si cierta cadena w pertenece a L .

La primera pregunta se responde encontrando una derivación a partir del símbolo inicial hasta una cadena de símbolos terminales. Por lo general se eliminan primero las variables inútiles y al final del algoritmo se ve si sigue existiendo el símbolo inicial.

La segunda pregunta es la que interesa más pues es el proceso que llamamos *compilación*. Encontrar la derivación que produce una cadena es equivalente a saber que el programa pasado a un compilador no tiene errores de sintaxis. Si el programa puede ser generado a partir del símbolo inicial de su gramática, significa que fue construido siguiendo correctamente las reglas del lenguaje. Para ello hay diversos algoritmos.

Otra pregunta es si un LLC es infinito. Lo que se hace es construir el grafo de dependencias de las variables y determinar si existe algún ciclo, lo que da respuesta positiva a esta pregunta.

2.4.2. Propiedades de cerradura

Como ocurre con los lenguajes regulares, los libres de contexto son cerrados bajo ciertas operaciones, pero no las mismas que en el caso de los regulares.

Proposición 14. Sean L , L_1 y L_2 lenguajes libres de contexto. Se cumple que

- $L_1 \cup L_2$ también es libre de contexto.
- $L_1 L_2$ también es libre de contexto.
- L^* es libre de contexto.

Como contra parte, hay varias operaciones que en general no son cerradas. Por ejemplo la intersección de LLC. El ejemplo recurrente es el lenguaje $L(a^n b^n c^n)$ que no es libre de contexto, pero sí lo son los lenguajes $L(a^n b^m c^m)$ y $L(a^n b^n c^m)$ cuya intersección es el primero.

El complemento de un LLC no necesariamente es libre de contexto. Significa que la la capacidad para aceptar cadenas no es igual a la capacidad para rechazarlas, es decir, no es simétrica.

Como caso curioso, la intersección entre un lenguaje regular y uno libre de contexto sí es libre de contexto.

2.5. Limitaciones de los lenguajes libres de contexto

Como ya se observó antes, el lenguaje $a^n b^n c^n$ no es libre de contexto y de hecho hay una infinidad de lenguajes que no lo son. Para efectos prácticos, una de las limitaciones importantes de las LLC es que es imposible determinar si los tipos de datos enviados a una función corresponden a los tipos puestos en su definición.

Por ello las gramáticas requieren un poco más de ayuda al momento de determinar la validez de un programa.

Entre los más sencillos se encuentran los LL y LR. Los LL intentan leer la entrada de izquierda a derecha y generar una derivación izquierda. Los LR generan una derivación por la derecha. En general los LR son más poderosos, pero para ambos tipos hay LLC que no son reconocibles por ninguno de estos analizadores.

Capítulo 3

Lenguajes Recursivamente Enumerables

De los Lenguajes Regulares, aceptados por AFD o generados por ER, se pasó a los Lenguajes Libres de Contexto, aceptados por AP o generados por GLC. Ahora pasamos a un conjunto de lenguajes aún más amplio, aceptados por los autómatas más poderosos, llamados Máquinas de Turing, que equivalen a gramáticas de gran versatilidad: las gramáticas sensibles al contexto.

El enfoque cambia un poco pues las MT pueden verse no sólo como aceptadoras de cadenas de un lenguaje, sino como calculadoras de funciones, lo que las hace equivalentes a cualquier computadora moderna.

3.1. Máquinas de Turing

Las Máquinas de Turing¹ son el nivel más complejo de modelo abstracto de computadora. Al AFD se le agregó memoria de lectura en forma de pila y se consiguió una máquina mucho más poderosa, el autómata de pila. Ahora, al AFD no se le agrega una memoria tipo pila, sino que se le permite escribir en la cinta de entrada y avanzar en ambas direcciones, con lo que de nuevo gana mucho poder. De hecho, cualquier cálculo que se puede realizar por la computadora más poderos de la actualidad es posible con una MT.

Una Máquina de Turing M consiste en los siguiente conjuntos:

¹Propuestas por Alan Mathison Turing en 1936. Marvin Minsky consideraba el artículo de 1936 de Turing la presentación de la computadora moderna y algunas técnicas de programación.

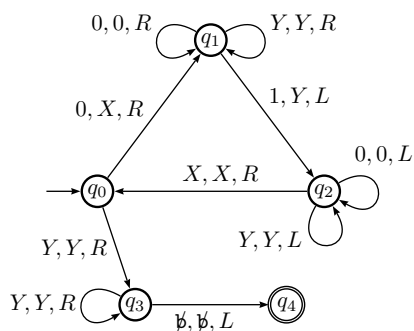


Figura 3.1: Máquina de Turing que acepta el lenguaje $0^n 1^n$. Su tabla de transiciones se muestra en la tabla 3.1.

1. Q un conjunto de estados,
2. $\Sigma \subset \Gamma$ el alfabeto de la cinta de entrada,
3. Γ el alfabeto de la máquina,
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ la función de transición,
5. $q_0 \in Q$ el estado inicial,
6. $B \in \Gamma$, $B \notin \Sigma$ el espacio en blanco y
7. $F \subset Q$ el conjunto de estados de aceptación.

De esta manera, un AFD es una tupla $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$ junto con una cinta de entrada donde se escribe una cadena de caracteres que será procesada. Las transiciones tienen la forma

$$\delta(p, a) = (q, b, Y) \quad (3.1)$$

que indica que, estando en el estado p , la MT procesa el símbolo a de la cinta de entrada, pasando al estado q (que puede ser el mismo estado p), escribiendo el símbolo b en la cinta y avanzando en la dirección Y (L = izquierda, R = derecha).

A diferencia de los anteriores autómatas, cuando la MT llega a un estado de aceptación $q \in F$, debe detener su ejecución, aceptando la cadena, no siendo válidas las transiciones a partir de q . Por eso, transiciones como la descrita en la ecuación (3.1), debe cumplirse que $p \notin F$. Por obvias razones, no puede ocurrir que $q_0 \in F$.

Ejemplo 59. Consideremos el lenguaje $L = \{0^n 1^n\}$, que siendo libre de contexto y por tanto aceptable con un AP, nos es útil para ilustrar el comportamiento de una MT. Sea la MT

$$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0, 1\}, \{0, 1, X, Y, \emptyset\}, \delta, q_0, \emptyset, \{q_4\})$$

dada por la figura 3.1 y las transiciones dadas en la tabla 3.1.

El algoritmo funciona así: al inicio, la cabeza de lectura lee el primer 0, lo sustituye con una X y pasa al estado q_1 , que va en busca de un 1, saltándose los

	0	1	X	Y	$\emptyset$
q_0	(q_1, X, R)	—	—	(q_3, Y, R)	—
q_1	$(q_1, 0, R)$	(q_2, Y, L)	—	(q_1, Y, R)	—
q_2	$(q_2, 0, L)$	—	(q_0, X, R)	(q_2, Y, L)	—
q_3	—	—	—	(q_3, Y, R)	$(q_4, \emptyset, L)$
q_4	—	—	—	—	—

Tabla 3.1: Tabla de transiciones de la Máquina de Turing que acepta el lenguaje $0^n 1^n$. La construcción es similar a el caso de los AFD, cada renglón corresponde a un estado y cada columna a un posible símbolo en la cadena de entrada. Su diagrama de transiciones se muestra en la figura 3.1.

caracteres necesarios, que pueden ser un 0 o una Y . Al encontrar un 1, pasa al estado q_2 , lo sustituye el 1 con una Y . El estado q_2 retrocede hasta encontrar la X más cercana. Al encontrarla pasa al estado q_0 y avanza esperando encontrar un 0 para repetir este procedimiento. De no encontrar ni un cero, es de esperarse que se encuentre con una Y , pasando al estado q_3 , que se encarga de saltarse todas las Y . Al encontrar un $\emptyset$, la máquina M se detiene en q_4 .

El lector diligente puede probar con las cadenas 0011, 011 y 001 para estudiar el comportamiento en cada caso. ¿En qué estado se queda la MT en cada caso?

El ejemplo anterior sirve para mostrar la capacidad nueva de esta máquina abstracta, es fácil extender el ejemplo para cadenas de la forma $a^n b^n c^n$ o con tantos símbolos como se desee.

Escribir sobre la cinta de entrada y moverse en ambas direcciones le otorga un poder superior al de la pila como única memoria. Todo lo que se calcula con una computadora moderna es posible calcularlo con una MT. Pasemos a otro ejemplo, un lenguaje que no sea aceptado por un autómata de pila.

Ejemplo 60. El lenguaje $\{0^n 1^n\} \cup \{0^n 1^{2n}\}$ no es aceptado por ningún autómata de pila determinista. Una MT no tiene problemas. El autómata se muestra en la figura 3.2. En su primera parte trabaja idéntico que el de la figura 3.1.

Los estados 1, 2 y 3 se encargan de revisar que los ceros sean la misma cantidad que los unos, como en las cadenas 01 o 000111. Al terminar esta etapa, el estado 4 verifica que no queden unos extras. Si los hay, pasa a los estados 5 y 6, que verifican que los 1 que queden sean tantos como los 0 que se marcaron con x . El estado 7 comprueba que ya no hay más símbolos x .

Es posible que una MT siga realizando operaciones de manera indefinida, quedando en un ciclo eterno, como ocurre en algunas ocasiones con los programas de computadora de los principiantes. Por otra parte, también es posible que por un error de diseño, la MT intente moverse fuera del extremo izquierdo de la cinta, lo que da lugar al abandono de los cálculos por una operación anormal.

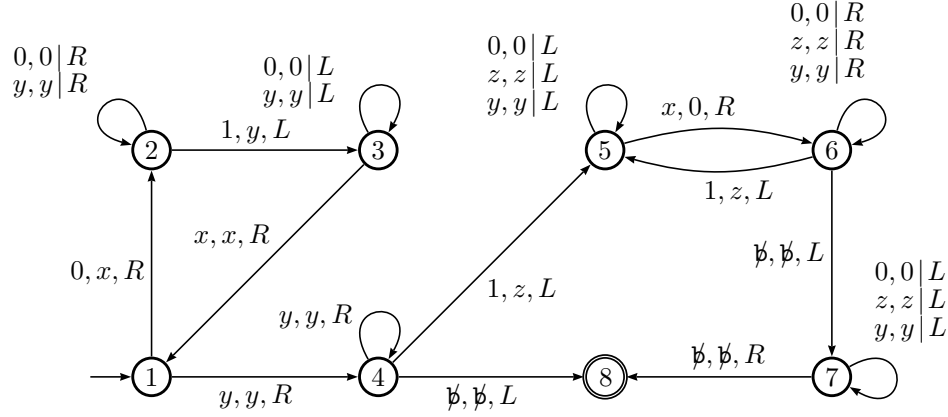


Figura 3.2: Máquina de Turing determinística que acepta el lenguaje $\{0^n 1^n\} \cup \{0^n 1^{2n}\}$.

3.1.1. Descripción instantánea de una MT

Como en el caso de los autómatas de pila, es muy conveniente una manera de describir el estado que guarda la MT en un momento arbitrario de su ejecución. Para los AP, tal descripción se dio en la sección 2.2.1.

Las MT pueden ser descritas mediante el contenido de su cinta, insertando el estado actual entre los símbolos, a la izquierda del símbolo que sigue por procesarse, es decir, queda de la forma

$$a_1 a_2 \dots a_{i-1} q_j a_i a_{i+1} \dots$$

que indica que la MT está en el estado q_j y que procesará el símbolo a_i . Debe ser claro que al inicio de su ejecución, al procesar la cadena $a_1 a_2 \dots a_n$, la MT comienza con la descripción instantánea

$$q_0 a_1 a_2 \dots a_n$$

y no es necesario describir el resto de la cinta, que contiene símbolos $\emptyset$. Utilizaremos el símbolo $\mapsto$ para indicar que de una descripción instantánea la MT ejecuta una transición y pasa a otra descripción instantánea.

Ejemplo 61. Para la MT del ejemplo 59, que acepta el lenguaje $0^n 1^n$, la sucesión de descripciones instantáneas para la cadena 0011 es como sigue:

$$\begin{aligned} q_0 0011 &\mapsto X q_1 011 \mapsto X 0 q_1 11 \mapsto X q_2 0Y1 \mapsto q_2 X 0Y1 \\ &\mapsto X q_0 0Y1 \mapsto X X q_1 Y1 \mapsto X X Y q_1 1 \mapsto X X q_2 Y Y \\ &\mapsto X q_2 X Y Y \mapsto X X q_0 Y Y \mapsto X X Y q_3 Y \mapsto X X Y Y q_3 \emptyset \\ &\mapsto X X Y q_4 Y \end{aligned}$$

y la cadena es aceptada, ya que la ejecución termina con la MT en q_4 , el estado de parada. El símbolo de espacio en blanco ($\emptyset$) se escribe para hacer explícito qué símbolo está por procesar la MT, pero sabemos que todo el resto de la cinta está lleno de tal símbolo.

3.1.2. Lenguaje de una MT

En el caso de las MT, como extensión de las otras máquinas abstractas, también se considera que una cadena en la cinta de entrada es aceptada cuando la MT llega a alguno de los estados finales. Sin embargo, por la forma como una MT opera, hay que considerar que una MT puede quedar en un ciclo infinito, además de detenerse en un estado no final, como les puede ocurrir a los AFD y AP.

Por lo general se prefiere que no haya transiciones que salgan de los estados finales. Se supone que al llegar a un estado final o de aceptación, la MT se detiene. Si una MT se detiene en un estado NO final, entonces se interpreta que la cadena ha sido rechazada, pero si queda ciclada, ignoramos la respuesta. Esto se reduce a los siguientes casos: Si w es la cadena en la cinta de entrada al inicio de la ejecución de la MT M , ocurre uno de los siguientes casos.

1. Si M se detiene en un estado final, $w \in L(M)$.
2. Si M se detiene en un estado que no es final, $w \notin L(M)$.
3. Si M se queda ciclada, entonces no tenemos una decisión.

Considerando lo anterior, el lenguaje de la MT M

$$L(M) = \{w \in \Sigma^* \mid \delta(q_0, w) \vdash^* (p, \cdot, \cdot) \text{ con } p \in F\} \quad (3.2)$$

y los puntos se refieren a cualquier decisión, pues lo importante es el estado final p .

Ejemplo 62. Consideremos ahora el lenguaje $L = \{0^n 1^n 2^n\}$, que NO es libre de contexto, por lo que no hay un AP que lo acepte. Una manera de conseguir que una MT revise si una cadena pertenece a este lenguaje es con el siguiente algoritmo. La cabeza de lectura se supone apuntando al primer símbolo, que debe ser un cero, así que se usará la transición de q_0 a q_1 , cambiando el 0 por a y moviendo la cabeza a la derecha. Este es el algoritmo:

1. En q_0 , se cambia el primer símbolo (debe ser un 0) por a, se va al estado q_1 y se avanza a la derecha.
2. Si símbolo fuera una b, se va al paso 6, en el estado q_4 .
3. En q_1 , cada 0 o b que se lee se vuelve a reescribir hasta encontrar un 1. Tal 1 se sustituye por b, se va al estado q_2 y la cabeza de lectura se mueve a la derecha.
4. En q_2 , cada 1 o c que se lee se vuelve a reescribir hasta encontrar un 2. Ese dos se sustituye por c, se va a q_3 y se mueve a la izquierda.
5. En q_3 , la cabeza de lectura se mueve a la izquierda, hasta encontrar una a, es decir, cada símbolo que se lee se vuelve a reescribir. Al encontrar la a, se reescribe, se va a q_0 y ahora se mueve a la derecha, regresando al paso 1.

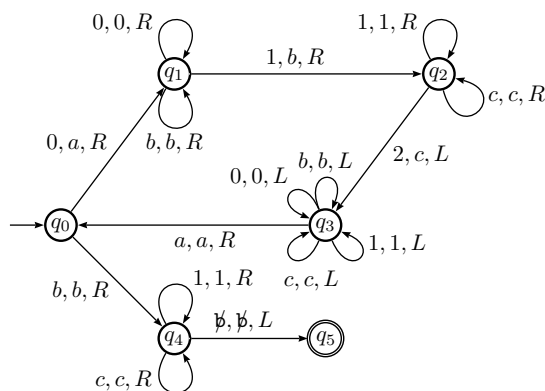


Figura 3.3: Máquina de Turing que acepta el lenguaje NO libre de contexto $0^n 1^n 2^n$.

6. Cada Y que se lee se reescribe y se mueve a la derecha, hasta encontrar una Z .
7. Cada símbolo b o c que se lee se reescribe y se mueve a la derecha, hasta encontrar un $\emptyset$.

Los pasos finales (a partir de 6) son para comprobar que no quedó ningún 1 o 2 en la cadena. La MT que aplica este algoritmo es la de la figura 3.3.

Debe ser claro que es posible extenderse a lenguajes como $\{0^n 1^n 2^n 3^n \dots k^n\}$.

3.1.3. MT como calculadoras de funciones

Además de una máquina para aceptar cadenas, es decir, una máquina abstracta que resuelve un problema en el sentido de la ecuación (1.10) en la página 13, es conveniente ver a las MT como evaluadoras de funciones, de tal manera que para cierta entrada produzcan una salida distinta que la aceptación.

Pueden verse entonces como funciones $MT : \mathbb{N} \rightarrow \mathbb{N}$. En este caso, si se calcula $f(a) = b$, el argumento a se coloca en la cinta al iniciar la ejecución y al terminar, en la cinta sólo aparece el resultado b .

De la misma manera, se puede pensar que una MT recibe m argumentos, separándolos con algún símbolo especial y que con ellos calcula un resultado que incluye varios valores. Así, se modelan funciones de varias variables, en general el caso $MT : \mathbb{N}^m \rightarrow \mathbb{N}^n$. Veamos algunos ejemplos.

Ejemplo 63 (Contador). La idea es tener una MT que sume 1 a su entrada, dada en binario. Esa es la base de un contador, sumar uno en cada iteración.

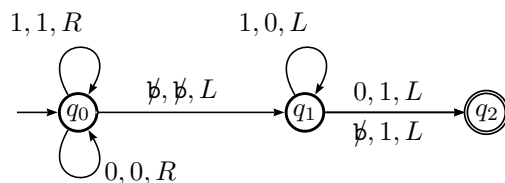


Figura 3.4: Máquina de Turing que suma 1 a su entrada escrita en sistema binario.

La idea es simple si se recuerda la numeración en binario:

0
1
10
11
100

Lo que se requiere es comenzar desde el final de la cadena y sustituir cada 1 por cero hasta encontrar un 0 que debe cambiarse por uno. Si tal 0 no se encuentra, entonces se agrega un 1 al inicio de la cadena, como al pasar de 11 a 100. La idea es simple y la MT que lo hace es la de la figura 3.4.

Ejemplo 64. Máquina de Turing que desplaza toda su cadena de entrada una celda a la derecha. Si el alfabeto de entrada consta sólo de un símbolo, el problema se reduce a eliminar el primer carácter y escribirlo al final de la cadena, pero nos interesa más el caso de $|\Sigma| > 1$.

La idea esencial del algoritmo que se propone es tener estados suficientes para cada símbolo de el alfabeto. Se requiere un estado para ir acarreando a la derecha cada símbolo. Es la misma idea de hacerlo a mano: la cadena $|1101|$ debe transformarse en $|blank1101|$, así que al inicio se lee el primer símbolo (1), se escribe $blank$ y se pasa a el estado que sabe que se tiene que escribir ese símbolo, digamos q_1 , avanzando a la derecha. Ahora, como sigue un 1, no se requiere sobrecribir, solo saltárselo hasta encontrar un símbolo distinto, aunque realmente se debiera leer el segundo 1 y escribir el que se leyó primero, pero no es necesario.

Al llegar al tercer símbolo, el 0, es momento de escribir un 1 en esa posición, pasar al estado que sabe que hay un 0 pendiente, digamos q_2 , y avanzar en la cadena. El siguiente símbolo es un 1, así que se escribe el 0 y se pasa al estado q_1 de nuevo. Ahora, al final de la cadena, encontrarse con el primer espacio en blanco, se escribe el 1 final y se pasa al estado de parada. El diagrama se muestra en la figura 3.5.

Ejemplo 65. Construyamos una MT que duplica la cadena de entrada, es decir, dada la cadena $w \in \Sigma^*$, al término de su ejecución la cinta tiene la cadena

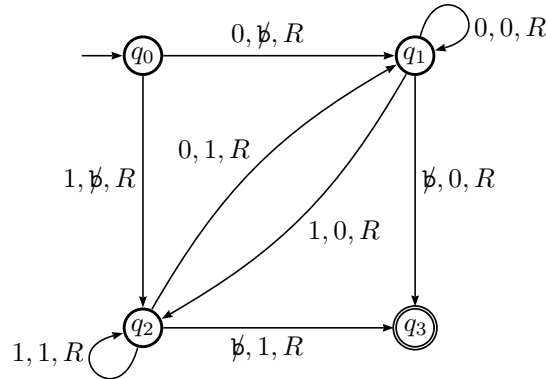


Figura 3.5: Máquina de Turing que desplaza la cadena de entrada una posición a la derecha.

ww. Para duplicar la cadena, se requiere que la MT posea un alfabeto de cinta, digamos $\Sigma = \{0, 1\}$. La idea de ejecución es duplicar la cadena codificada. Cada 0 se copia como x y cada 1 se copia como y . Para distinguir los símbolos ya copiados se sustituyen con a y b respectivamente. Una secuencia de pasos para la cadena

<u>1</u> 001	Cadena original
b00 <u>1</u> x	Primer 1 se copia
ba0 <u>1</u> xy	Segundo símbolo copiado
baa <u>1</u> xyy	Tercer símbolo copiado
baabxyyx	Ahora falta sustituir símbolos
10011001	

Una vez diseñado el algoritmo que realizará la tarea, hacer la MT es mero trámite.

Como puede verse de los ejemplos anteriores, lo primero para poder hacer una MT que realice alguna tarea se requiere tener idea precisa de cómo se realizará, es decir, un algoritmo que haga lo que queremos. Con cada detalle ya pensado, hacer la MT es bastante más sencillo.

Por otra parte, cuando poco a poco se tiene el diseño de MT que realizan tareas sencillas, es común combinarlas, empleando una MT como rutina de otra. De esta manera, se realiza lo que se llama construcción modular de MT. El siguiente ejemplo lo ilustra.

Ejemplo 66. Ahora construiremos una MT que suma dos números dados en binario. La entrada será un par de números separados por el símbolo $+$. El terminar la ejecución, en la cinta debe estar el resultado y la cabeza de lectura al inicio de la suma. Primeramente, desarrollemos la idea de cómo llevar a cabo la suma.

Tenemos ya una MT que suma de 1 en 1. De manera similar, hacer una que reste uno también es fácil, por lo que aprovecharemos que se cumple la siguiente

propiedad:

$$m + n = (m + 1) + (n - 1).$$

De esta manera, dadas las dos cantidades, mientras la segunda no sea cero, iremos sumando 1 a la primera y restando 1 a la segunda. Cuando la segunda sea 0, borraremos el signo + y los ceros del segundo número para dejar solo el resultado en la cinta, con la cabeza de lectura en el primer bit del resultado.

Por ejemplo: $2 + 3 = 3 + 2 = 4 + 1 = 5 + 0 = 5$. En binario:

$$\begin{aligned} 10 + 11 &= 11 + 10 \\ &= 100 + 01 \\ &= 101 + 00 \\ &= 101 \end{aligned}$$

y 101 es lo único que aparece al final en la cinta. Tendiendo esta idea clara, se puede construir la MT.

Para hacerlo de manera aún más modular, pensemos que esta MT que suma puede ser empleada por otra MT, así que hay que utilizar la cinta con cuidado. Pensemos que a la derecha de la suma tenemos todo el espacio necesario disponible y que a la izquierda, la cinta está ocupada por otra información, por lo que una suma que haga aumentar el tamaño del primer sumando (nuestro acumulador) corre el riesgo de sobre-escribir en la cinta.

Por ejemplo con la suma $111 + 1111 = 10110$, en la que el primer sumando ha crecido dos posiciones a la derecha. Para evitar este problema, cada que se requiere un espacio adicional, toda la cadena de suma se recorre una posición a la derecha, empleando para ello una MT como la del ejemplo 64. La diferencia es que ahora aparece también el símbolo + como otro que hay que desplazar. Además, el cursor debe regresarse al inicio de la cadena para continuar con el proceso.

Una MT como la diseñada es la de la figura 3.6. El estado etiquetado con S_{10} representa la MT que desplaza una posición a la derecha la cadena, regresando al inicio.

3.1.4. Máquinas de Turing no determinísticas

La definición de MT dada en el apartado anterior especifica un comportamiento determinístico. Como en los autómatas anteriores, es posible definir Máquinas de Turing No determinísticas. Ahora, las transiciones serán de la forma

$$\delta(p, a) = \{(q_1, b_1, Y_1), (q_2, b_2, Y_2), \dots, (q_k, b_k, Y_k)\} \quad (3.3)$$

dando diversos movimientos para el mismo par (p, a) . Se puede ir a cualquiera de los estados q_i , escribiendo el símbolo b_i en la cinta y moviéndose en la dirección indicada por Y_i . Es adecuado pensar que la MT toma todos estos caminos al mismo tiempo.

A diferencia de los Autómatas de Pila, las Máquinas de Turing no ganan poder con el no determinismo, tal y como ocurre con los AFD. Claro, una MT

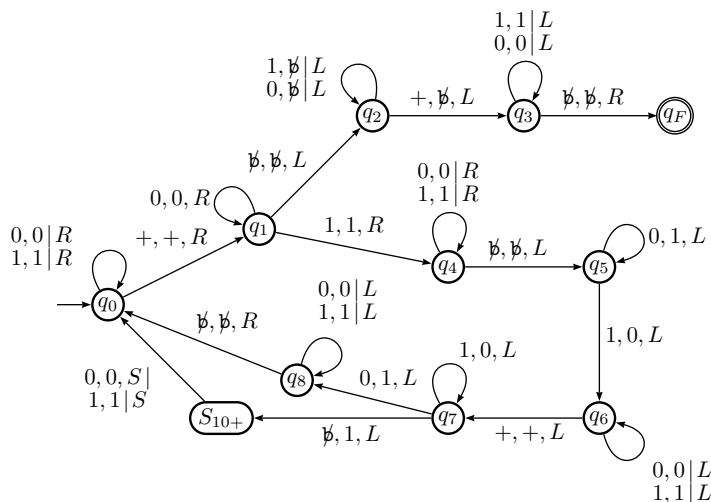


Figura 3.6: Máquina de Turing que suma dos números en binario, empleando una rutina de desplazamiento a la derecha para cuidar el espacio de la cinta. El estado etiquetado con S_{10+} es una MT que desplaza toda la cadena una posición a la derecha y regresando al inicio de la cadena.

tarda más en hacer el mismo trabajo que una MT no determinística, pero lo hace.

Ejemplo 67. Si retomamos el lenguaje $\{0^n 1^n\} \cup \{0^n 1^{2n}\}$, podemos diseñar una MT que acepte el lenguaje empleando no determinismo. Lo que se hace es comenzar como la MT de la figura 3.2, pero en el estado 2 hay dos transiciones posibles al encontrar el siguiente 1. Un camino es cambiarlo por y y pasar directo al estado 4. El otro camino es ir cambiarlo por y , pasar al estado 3 y cambiar el siguiente 1 de nuevo por y antes de ir al estado 4.

De esta manera, se puede pensar que el autómata marca uno o dos símbolos dependiendo de la cadena. La MT se muestra en la figura 3.7. El NO determinismo está en el estado dos, pues al encontrarse un 1, el autómata lo cambia por y y avanza a la izquierda y también a la derecha.

En general, el no determinismo reduce el tiempo de ejecución de muchas MT y puede incluso reducir su tamaño, como en el ejemplo anterior.

3.1.5. Máquinas de Turing con múltiples cintas

Agregar cintas a la MT permite organizar mejor los algoritmos y el procesamiento de los datos. Por ejemplo, se puede emplear una cinta para la entrada, otra para la salida y otras más para los cálculos intermedios.

El comportamiento de las transiciones es un poco más complejo pues en este caso hay que ver el símbolo que está en cada una de las k cintas de entrada para decidir qué hacer, es decir, para decidir si pasar a otro estado, escribir qué

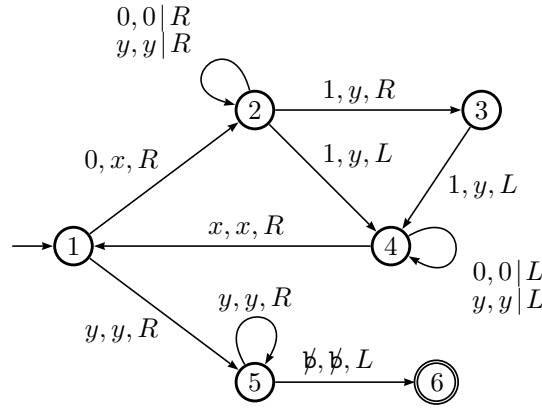
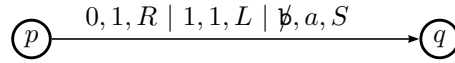


Figura 3.7: Máquina de Turing NO determinística que acepta el lenguaje $\{0^n 1^n\} \cup \{0^n 1^{2n}\}$.

y moverse en una u otra dirección en cada cinta. Por ello las transiciones ahora tienen la forma

$$\delta(p, a_1, a_2, \dots, a_k) = (q, (b_1, D_1), (b_2, D_2), \dots, (b_k, D_k))$$

donde las a_i son símbolos que se leen en la respectiva cinta i , b_i es lo que se escribe y D_i la dirección que toma la cabeza de lectura en cada caso. En el caso de $k = 3$ cintas, la transición representada por un diagrama es la siguiente.



Lo que ocurre en cada cinta queda separado por líneas verticales. Esta transición se comporta así: estando en el estado p y estando las cabezas de lectura por procesar los símbolos 0, 1 y $\emptyset$, se mueve al estado q y escribe un 1 en la primera cinta, deja la segunda sin cambios y el símbolo a en la tercera.

Así como el no determinismo, varias cintas facilitan el diseño de algoritmos. Un ejemplo ilustra esto.

Ejemplo 68. La figura 3.8 muestra una versión con 3 cintas de la MT de una sola cinta de la figura 3.3, que acepta el lenguaje $a^n b^n c^n$. Lo que hace el estado 1 es copiar los símbolos a en la segunda cinta hasta encontrar la primera b y comenzar a copiarlas en la tercera cinta en el estado 2. Al encontrar una c , se queda sobre ella y pasa al estado 3. Para el caso de la cadena $aaabbbccc$ el estado de las cintas y las cabezas de lectura, aquí mostrados como cursores, sería:

Cinta 1: aaabbbccc
 Cinta 2: aaa
 Cinta 3: bbb

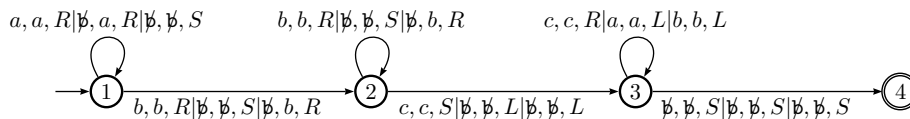


Figura 3.8: Máquina de Turing que acepta el lenguaje $a^n b^n c^n$ con tres cintas. La reducción en tamaño y tiempo de ejecución es notable.

Con los cursores en posición, basta moverlos de manera sincronizada, un símbolo a la vez para verificar que se trata de la misma cantidad, procesando la c de la primera cinta, la a de la segunda y la b de la tercera, avanzando (a la derecha) en la primera y retrocediendo (a la izquierda) en las otras dos. El proceso termina cuando se alcanza al mismo tiempo un espacio en blanco y pasando al estado de aceptación.

Como en el caso del no determinismo, las múltiples cintas no le dan poder adicional a las Máquinas de Turing. Cada MT multicinta puede ser simulada con una MT de una sola cinta. Lo que se hace es simular cada cinta en una porción distinta y cuando se requiere más espacio se utiliza una rutina que redistribuya la información, separando los contenidos de cada cinta simulada.

3.1.6. Lenguajes sensibles al contexto

Si vemos a las MT como aceptadoras de lenguajes, estos lenguajes son los llamados Lenguajes Sensibles al Contexto (LSC) y son generados por Gramáticas Sensibles al Contexto.

En una GSC, también hay variables, terminales, producciones y un símbolo inicial. Lo que cambia es la forma de las producciones. En las GLC, el lado izquierdo de cada producción constaba sólo de una variable. Ahora es posible que la variable esté acompañada de otras variables o símbolos terminales, como en

$$\begin{aligned} Ab &\rightarrow aaAba|Bc \\ cAa &\rightarrow acA|c \end{aligned}$$

donde para poder sustituir una variable, se ponen requisitos a su contexto: la variable A puede cambiarse por Bc sólo si A tiene enseguida una b .

Ejemplo 69. La gramática con las producciones

$$\begin{aligned} A &\rightarrow aABc|abc \\ cB &\rightarrow Bc \\ bB &\rightarrow bb \end{aligned}$$

genera el lenguaje $\{a^n b^n c^n\}$ que no es libre de contexto. Un ejemplo de derivación izquierda es el siguiente.

$$\begin{aligned}
 A &\Rightarrow aABc \\
 &\Rightarrow aaABcBc \\
 &\Rightarrow aaabcBcBc \\
 &\Rightarrow aaabBccBc \\
 &\Rightarrow aaabbccBc \\
 &\Rightarrow aaabbcBcc \\
 &\Rightarrow aaabbBccc \\
 &\Rightarrow aaabbbccc
 \end{aligned}$$

Es evidente cómo el contexto define la producción que se empleará.

Ejemplo 70. Otra gramática para el mismo lenguaje es

$$\begin{aligned}
 A &\rightarrow aAB \mid abc \\
 bBc &\rightarrow bbcc \\
 cB &\rightarrow Bc
 \end{aligned}$$

y lo genera con secuencias distintas, como puede constatarse intentando con la misma cadena $aaabbbccc$.

Estas gramáticas son más versátiles pero también mucho más complicadas de emplear y analizar. No se estudiarán en este curso.

3.2. Problemas de decisión

Con problemas de decisión se hace referencia a problemas donde se requiere tener siempre una respuesta, positiva o negativa. Una MT que llega a un estado de aceptación y termina acepta la cadena de su entrada, pero una MT que nunca termina, es decir que queda ciclada, jamás da una respuesta, por lo que la pertenencia de la entrada queda en una incertidumbre.

Todo problema puede plantearse como un problema de decisión. Según se definió en la página 13, en la definición 9, un problema es determinar si una cadena w pertenece a un lenguaje L y es precisamente un problema de decisión. Las MT pueden emplearse para este fin. Más aún, pueden ser empleadas para simular el comportamiento de otra MT.

Los lenguajes mismos se clasifican en dos tipos: los decidibles y los reconocibles.

Definición 18. *Un lenguaje es reconocible si existe una MT que siempre se detiene aceptando las cadenas del lenguaje.*

Observe que esta definición sólo habla de las cadenas del lenguaje. Es posible que con cadenas que no son del lenguaje la MT nunca se detenga.

Definición 19. *Un lenguaje es decidible si existe una MT que siempre se detiene aceptando las cadenas del lenguaje y rechazando aquellas que no pertenecen.*

En este caso, un lenguaje es decidible cuando existe al menos una MT que siempre se detiene en todas sus entradas. Estos lenguajes también son llamados recursivos.

Varias propiedades son sencillas de probar, en el caso de esta clasificación de lenguajes.

Proposición 15. *Si L_1 y L_2 son lenguajes decidibles, entonces también los son $L_1 \cup L_2$, $L_1 \cap L_2$, $L_1 L_2$.*

Demostración. Las pruebas son sencillas y consisten construir una nueva MT M a partir de las MT M_1 y M_2 que deciden a los lenguajes. Se deja tal construcción como ejercicio. $\square$

El caso del complemento se deja para más adelante, cuando esté en un contexto más conveniente. Los lenguajes reconocibles cumplen las mismas propiedades, excepto el complemento.

3.2.1. Máquinas de Turing Universales

Las transiciones de una MT pueden codificarse como cadenas de símbolos, estableciendo una numeración binaria para los estados y símbolos.

La transición $\delta(q_2, a) = (q_3, b, L)$ pudiera codificarse como la cadena

0010100010010

donde los símbolos 1 separan cadenas de ceros que representan cada elemento de la transición. a se codifica con 0 y b con 00. $L = 0$ y $R = 00$, aunque no se emplea.

Una transición genérica $\delta(q_i, a_m) = (q_j, a_n, L)$ puede codificarse como cadenas de ceros separadas por unos como $0^i 1 0^m 1 0^j 1 0^n 1 0$

$$\underbrace{00 \dots 0}_i 1 \underbrace{00 \dots 0}_m 1 \underbrace{00 \dots 0}_j 1 \underbrace{00 \dots 0}_n 1 0$$

donde la dirección L se codifica con un cero y R con dos ceros. De esta manera cada transición de una MT puede ser codificada con ceros y unos, colocando todas las transiciones en una misma cadena, separando el código de cada transición con dos unos consecutivos para distinguirlas.

Definición 20. *Con tal cadena formada por la concatenación de transiciones (o de cadenas que codifican transiciones, para ser exactos), lo interesante es que esa cadena representa por sí sola a la MT original. Si M es tal Máquina de Turing, sea $\langle M \rangle$ su codificación, o descripción canónica.*

Ejemplo 71. La MT que suma 1 a un número en binario, en la figura 3.4, se codifica como

01000101000100110100101001001101010010101100100010010010110010010001000101100101000100010

Definición 21. Si al lado de esta representación se coloca la cadena de entrada w para la MT, toda esta gran cadena $\langle M \rangle w$ se puede dar como entrada a una Máquina de Turing U cuya tarea sea simular el comportamiento de M . Tal MT U se llama Máquina de Turing Universal.

Para simular otra MT, requiere varias cintas para facilitar el trabajo. En una están las transiciones codificadas, es decir, la descripción canónica de la MT simulada. En otra la entrada de dicha MT. Otra cinta es de control y se escribe el estado en el que se está en cada momento. La máquina universal avanza al parejo en la transición actual y en la cinta de control para verificar que se trata del mismo estado. De no ser así, busca la siguiente transición. Lo mismo para el símbolo que se lee.

Un trabajo arduo, pero esto modela de manera clara la ejecución de programas en memoria por parte de una computadora.

3.2.2. Máquina de Turing Universal restringida

Aunque la descripción canónica anterior es universalmente aceptada, una MTU que la emplea para simular cualquier MT es bastante sofisticada. Se puede construir una MTU que imponga ciertas restricciones para que sea más sencilla. Tal MTU se describe en esta sección.

La MTU la llamaremos M_R y simulará el comportamiento de MT con pocos estados, etiquetados con letras minúsculas de la a a la f . La cadena de entrada constará sólo del alfabeto binario además del espacio en blanco, codificado aquí como la letra B, pero durante la ejecución se permite el uso de los símbolos 2, x y y. Solo se considera el caso de MT determinísticas con un solo estado final.

Esto permite codificaciones más cortas para las transiciones. Por ejemplo, la transición $\delta(a, 0) = (b, 1, R)$ queda representada con la cadena a0b1R. Las transiciones se separan con guiones. Para la simulación, M_R emplea tres cintas. La primera cinta contiene los códigos de las transiciones encerrados entre dos símbolos #. Es una cinta de lectura, excepto al final, cuando se coteja el estado final.

La segunda cinta contiene la cadena que será la entrada, encerrada entre dos letras B, que se interpreta como los espacios en blanco alrededor de la cadena. En esta cinta ocurre el comportamiento de la MT simulada. Si durante la simulación se requiere más espacio al final de la cadena, deben agregarse suficientes símbolos B. La cadena no puede crecer hacia el otro extremo de la cinta.

La tercera cinta contiene dos símbolos, que representan el estado inicial y el estado final. Durante la simulación, la cabeza de lectura está fija en este símbolo y sólo se mueve hasta el final. El estado inicial es cambiado cada que se pasa de un estado a otro de la MT simulada. Al terminar la ejecución se espera que ambos símbolos sean iguales, lo que significa que la MT llegó al estado final.

Durante su ejecución, M_R lee una y otra vez las transiciones codificadas en la cinta 1. Cada que encuentra la transición correcta, la simula cambiando de estado en la cinta 3, escribiendo en la cinta 2 y moviendo la cabeza en la cinta 2. Luego regresa la cabeza de la cinta 1 al inicio para comenzar de nuevo buscando alguna transición para el nuevo estado y el nuevo símbolo de lectura.

Ejemplo 72. Empleando la MT M_{++} que suma 1 a su entrada en binario, mostrada en la figura 3.4, su descripción queda de la siguiente manera.

$$\langle M_{++} \rangle = a0a0R-a1a1R-aBbBL-b1b0L-b0c1L-bBc1L$$

Para esta codificación se ha sustituido $\emptyset$ por B, pues la MTU M_R en JFLAP así lo requiere. Si se simulará la suma de 1 a la cadena 1011, las cintas de M_R comenzarían con la entrada y durante la simulación podrá verse cómo la cinta

Cinta 1: #a0a0R-a1a1R-aBbBL-b1b0L-b0c1L-bBc1L#
 Cinta 2: B1011B
 Cinta 3: ac

representa la cinta de la MT simulada. Como es de esperarse, las cabezas de lectura inician en el primer símbolo de cada cinta.

La MTU M_R consta de 10 estados con suficientes transiciones para procesar el lenguaje binario y la tabla de transiciones de la MT simulada. De manera general, los estados q_0 y q_2 buscan una transición que comience con el estado actual y se pasa a q_1 donde se comprueba que el símbolo de la transición corresponde con el que está por procesarse en la cinta dos. Si no es ese regresa a q_0 y si lo es, pasa a q_3 , en la ruta que procesa la transición terminando en q_6 , donde se regresa a la primera transición para iniciar de nuevo.

En la tabla 3.2 se explica con detalle lo que hace cada transición. La figura 3.9 es un esquema general de la M_R , que puede descargarse en la liga <https://goo.gl/xr1CDg>.

¿Cuántas Máquinas de Turing hay?

Con la codificación presentada de máquinas de Turing de la sección previa, es fácil responder esta pregunta. Basta escribir en binario todos los números naturales. Algunas de las cadenas serán representaciones correctas de alguna máquina de Turing. De esta manera, descartando las codificaciones que no corresponden a MT alguna, tendremos una lista de todas las MT posibles. Evidentemente, esta es una cantidad infinita y enumerable, aún cuando hay una infinidad de números cuya representación en binario no corresponde a ninguna MT.

¿Cuántas funciones hay?

Esta sección está tomada de las notas Fundamentos Matemáticos y repetida aquí por conveniencia.

Transición	Objetivo
S a q_0	Se salta los delimitadores de las cintas 1 y 2
q_0 a q_1	Si el estado de inicio de la transición coincide con el que indica la cinta 3
q_0 a q_2	Si el estado de inicio de la transición NO coincide con el que indica la cinta 3
q_2 a q_2	Avanza hasta encontrar el símbolo de nueva transición (—)
q_1 a q_3	Comprueba que el símbolo de lectura de la transición sea el indicado en la cinta 2, si coinciden se ha encontrado la transición correcta
q_3 a q_4	Cambia el estado actual de la cinta 3 por el nuevo estado indicado en la transición que se procesa
q_4 a q_5	Escribe en la cinta 2 el símbolo indicado en la transición que se lee
q_5 a q_6	Mueve la cabeza de lectura de la cinta 2 conforme se indica en la transición de la cinta 1
q_6 a q_6	Regresa la cabeza de lectura de la cinta 1 hasta la primera transición, para comenzar de nuevo
q_2 a q_7	Si ya no hay transición adecuada, el estado de la cinta 3 se copia en la cinta 1
q_7 a q_8	Si el estado copiado coincide con el estado final, la ejecución termina.

Tabla 3.2: Explicación de las transiciones de la Máquina de Turing Universal M_R .

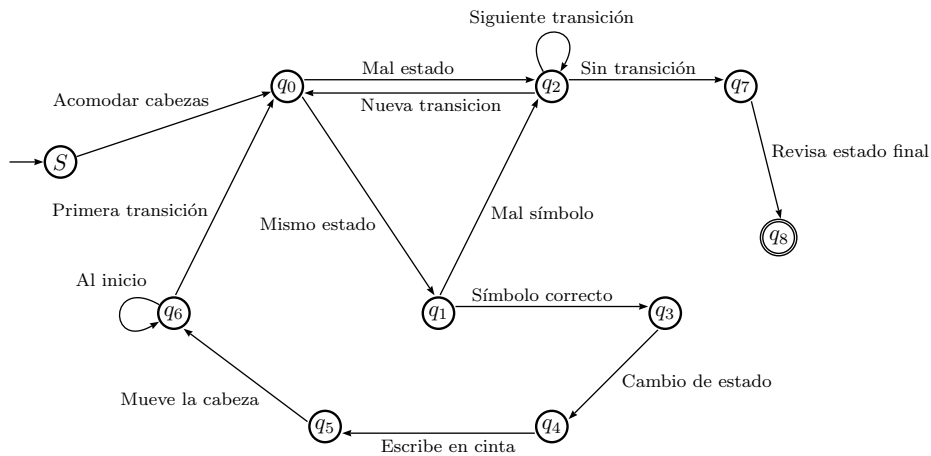


Figura 3.9: Esquema de la Máquina de Turing universal M_R .

Un programa puede verse como una función que recibe una entrada y produce una salida. Tanto entrada como salida pueden codificarse como cadenas de bits, por lo que cada programa se interpreta como una función de $\mathbb{N}$ en $\mathbb{N}$.

¿Cuántas funciones hay? ¿Cuántos programas se pueden programar? Estas dos preguntas son de gran importancia en el mundo de la computación pues si son la misma cantidad, entonces es posible hacer un programa para cada función que se pueda definir, independientemente de si son 1 a 1, sobre, biyectivas o con cualquier estructura especial.

Considérese cualquier lenguaje de programación. Está basado en una cantidad finita de palabras reservadas y operadores. Con esos elementos, se pueden comenzar a enumerar los programas desde el más pequeño construible, aumentando la cantidad de bytes uno por uno de ser posible. Esto nos da una numeración para los posibles programas, por lo que son tantos programas como números naturales. Cada programa resuelve un único problema, aunque claramente la relación (*programa*, *problema*) no es inyectiva.

Por otra parte, contar las funciones posibles de $\mathbb{N}$ en $\mathbb{N}$ requiere algo más de esfuerzo. Una función no tiene restricciones de dominio y rango, por lo que se puede comenzar con las funciones que van de $\mathbb{N}$ en $\{0, 1\}$. Con ese dominio y rango se definen las *funciones características*. Sea $A \subset \mathbb{N}$ y su función característica $f_A : \mathbb{N} \rightarrow \{0, 1\}$ como

$$f_A(x) = \begin{cases} 1 & \text{si } x \in A \\ 0 & \text{si } x \notin A \end{cases}$$

que caracteriza a los elementos del conjunto A marcándolos con 1. ¿Cuántas funciones características hay? Tantas como subconjuntos se pueden definir para los números naturales, por lo que hay al menos tantas funciones como $|\mathbb{N}| = |\mathbb{R}|$. Y eso que sólo se está contando esa familia de funciones.

★

Debido a que $|\mathbb{N}| < |\mathbb{R}|$, eso significa que hay más funciones que programas. Este resultado es de gran importancia pues indica que hay una infinidad de problemas que no tiene solución computable, o sea que hay problemas para los que no hay programas que los resuelvan. Se acaba así la intención de que la computadora resuelva todo. Todo lo anterior es la prueba de la contundente

Proposición 16. *Hay más problemas que posibles programas de computadora.*

Pero, ¿cuáles son esos problemas para los que no hay solución? ¿aparecen en la vida cotidiana? Son muchas las preguntas que falta responder y son de gran importancia tanto teórica como práctica.

3.2.3. Lenguajes recursivos

En una MT M con lenguaje $L(M)$, una cadena w es aceptada cuando M llega a un estado de aceptación al procesar w y se detiene. Pero si M procesa w y no llega a un estado de aceptación, se tiene dos casos: que M se detenga en un estado que no es de aceptación o que no se detenga jamás, es decir, que se quede en un ciclo infinito.

Definición 22. *Un Lenguaje Recursivo es aquel para el cual hay una MT que siempre se detiene con cualquier entrada, ya sea en un estado de aceptación o no.*

3.2.4. Lenguajes diagonal y universal

Consideremos el alfabeto $\{0, 1\}$. Las cadenas que se pueden generar son las combinaciones de 0 y 1 que representan números enteros positivos en binario. Claro, faltan ahí las cadenas que inician con ceros, pero es fácil incluirlas, basta comenzar agregando un cero al inicio de las cadenas de la potencia de dos anterior y luego continuar la numeración. Sean $w_1, w_2, \dots$ tales cadenas.

Como también las MT pueden enumerarse, codificándolas en binario como ya se vio, sean $M_1, M_2, \dots$ tales máquinas. Ahora es posible construir una tabla donde se enumeren las cadenas (verticalmente) y las MT (horizontalmente), obteniendo una tabla de pertenencias. De esta manera, cada columna corresponde al lenguaje de una MT M_j , representando la pertenencia al lenguaje de cada cadena enumerada a la izquierda.

Lenguaje diagonal L_d

La tabla 3.3 muestra esta construcción. Cada fila i representa la cadena w_i e indica si esta cadena es aceptada (1) o no (0) por la MT M_j de la columna que corresponde, el problema ¿ $w_i \in L(M_j)$? En la tabla se han marcado los elementos de la diagonal, es decir, la pertenencia de la cadena w_i al lenguaje $L(M_i)$.

El *lenguaje diagonal*, representado como L_d es un lenguaje que no está en esa tabla y que se construye invirtiendo las pertenencias de la diagonal. De esta forma, la cadena w_i sí pertenecerá a L_d si no pertenece a $L(M_i)$ y no pertenecerá a L_d si pertenece a $L(M_i)$.

$$L_d = \{ w_i \in (0 + 1)^* \mid w_i \notin L(M_i) \} \quad (3.4)$$

Proposición 17. *L_d NO es un lenguaje recursivamente enumerable.*

Demostración. Supongamos que existe una MT M tal que $L_d = L(M)$. Como las MT son enumerables, M debe ser una de las máquinas M_i de la tabla 3.3, donde están enlistadas todas las MT por lo que $M = M_j$ para alguna j .

Pero, ¿ $w_j \in L_d$? Si w_j está en L_d , entonces eso significa que $w_j \in L(M_j)$, lo que es una contradicción, por la definición (3.4), que dice que $w_i \notin L(M_i)$.

Si w_j no está en L_d , entonces debe pertenecer a $L(M_j)$, por la definición (3.4) lo que de nuevo es una contradicción porque $M = M_j$. $\square$

Lenguaje universal L_u

A partir de la misma tabla 3.3, es posible formar el llamado *lenguaje universal*, L_u . Este se construye con aquellas cadenas w_i que representan la codificación de la MT M_i y que aceptan su propio código como cadena de entrada.

$w_i \backslash M_j$	M_1	M_2	M_3	M_4	M_5	M_6	M_7	M_8
0	1	0	1	0	1	0	1	1
1	1	1	1	0	1	0	0	1
00	0	1	0	1	1	0	1	0
01	0	0	0	1	1	0	1	0
10	0	1	0	0	0	0	0	1
11	0	1	1	1	1	1	1	1
000	1	0	0	1	0	0	0	0
001	0	0	1	0	0	1	0	1 ...
010	1	0	1	1	0	1	1	1
011	0	1	0	0	0	0	0	1
					$\vdots$			

Tabla 3.3: Tabla de pertenencia de todas las cadenas de $(0+1)^*$ y todas las MT. Los elementos marcados sirven para definir el lenguaje diagonal L_d , que no es recursivamente enumerable.

En otras palabras, L_u es el conjunto de cadenas que representan MT que se aceptan como entrada. ¿Este lenguaje es reconocible o es decidible? Analicemos cada caso.

Sea w una cadena arbitraria de L_u . Una MT universal U puede simular el código $\langle w \rangle$ procesando la cadena w y como las máquinas representadas por cadenas de L_u aceptan su propio código, esta simulación terminará exitosamente. Así, la máquina U siempre termina la simulación de cadenas de L_u , por lo que el lenguaje es al menos reconocible.

Claro, esto no significa que L_u también sea decidible, pues no sabemos si U puede quedarse ciclada con otras cadenas que no estén en L_u . La respuesta a esta pregunta nos la da la siguiente proposición.

Proposición 18. *El complemento de un lenguaje decidible es también decidible.*

Demostración. Sea L un lenguaje decidible. Eso significa que hay al menos una MT, digamos M , que decide L , es decir, se detiene aceptando las cadenas de L y las que no son de L no las acepta pero también se detiene. A partir de M se puede construir una MT que decida el lenguaje complemento L^C de manera sencilla.

Sea M_C una MT que emplea a M como subrutina invirtiendo sus respuestas. Cada vez que M_C recibe una cadena de $w \in L^C$, simula a M . Como w está en el complemento de $L = L(M)$, entonces M termina y no acepta a w , por lo que M_C informa que la cadena es aceptada. Si a M_C se le da una cadena que no está en L^C , entonces M la aceptará y M_C responderá que esa cadena no es aceptada. $\square$

De esta manera, si L_u fuera decidible su complemento también lo sería. ¿Quién es el complemento de L_u ? Es otro lenguaje, compuesto por las cadenas

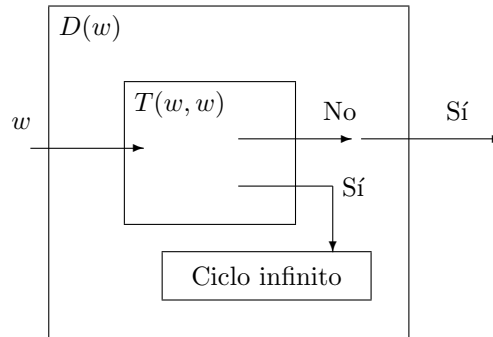


Figura 3.10: Esquema de la Máquina de Turing para el problema de la parada. Se comprueba que es imposible la construcción de la MT T .

que representan MT que no aceptan su propio código, es decir, L_d , el lenguaje diagonal, que no es recursivamente enumerable. Si L_u fuera decidable, también L_d lo sería, pero ni siquiera es reconocible. Por lo tanto, L_u no es decidable. Se trata de un ejemplo de un lenguaje reconocible, con complemento no reconocible.

3.2.5. El problema de la parada

A partir de que hay más problemas que programas, se entiende que hay una infinidad (incontable) de problemas que no se pueden resolver con computadora. Sin embargo, encontrar tales problemas no es sencillo.

El problema de la parada es el primer problema que se demostró que no es posible resolver con una MT y por tanto por ninguna computadora. Aparece en el artículo original de Turing de 1936.

Supongamos que existe una máquina de Turing T que recibe como entrada dos cadenas iguales w , el código de cualquier MT. Lo que hace T es decidir si la MT codificada como w termina su ejecución utilizando como entrada la misma cadena w . De esta forma, $T(w, w)$ siempre sabe si toda otra MT termina o no cuando recibe como entrada su propio código.

A partir de T , se puede construir otra MT D que utiliza a la MT T para decidir qué hacer. La entrada de D se alimenta con w , el código de alguna MT. D copia w y pasa las dos cadenas a T , que hace su trabajo determinando si la máquina codificada con w termina al procesar su propio código. Con la respuesta de T , la máquina D hace lo siguiente. Si T dice que w termina, entonces D entra en un ciclo infinito. Pero si T dice que w NO termina, entonces D responde que Sí.

Si para cualquier MT T , su código en binario es $\langle T \rangle$, entonces ¿qué responde D cuando su entrada es $\langle D \rangle$? Solo hay dos casos.

Caso 1: Si T responde Sí cuando $w = \langle D \rangle$, entonces D entra en un ciclo infinito y T se habrá equivocado pues D no para.

Caso 2: Si T responde que D No para, entonces D termina de inmediato respondiendo que Sí, lo que hace que T se haya equivocado de nuevo.

Con los dos razonamientos anteriores, es evidente que la existencia de una MT como T es imposible. Como implicación práctica inmediata, esto significa que no es posible saber si un programa arbitrario terminará su ejecución ante una entrada arbitraria. La MT D se muestra como esquema en la figura 3.10.

Este es un ejemplo académica e históricamente importante, que nos da muestra de limitaciones que tienen las MT y por tanto las computadoras que empleamos.

Entendiendo que hay una frontera tras la cual están los problemas que no se pueden resolver, uno de los objetivos de la Teoría de la Computación es estudiar los problemas que sí tienen solución, clasificándolos por su dificultad, en términos del consumo de recursos. Esa es el área conocida como Computabilidad y es motivo de otra asignatura.